

TECHNISCHE UNIVERSITÄT
CHEMNITZ

Integrating Open Large Language Models into the Jupyter Notebook Reproducibility Pipeline

Master Thesis

Submitted in Fulfilment of the
Requirements for the Academic Degree
M.Sc. Web Engineering

Dept. of Computer Science
Chair of Computer Engineering

Submitted by: Erisa Zaimi

Student ID:

Date: [Submission Date]

Supervising tutor: Dr. Sheeba Samuel

Abstract

Jupyter notebooks are widely used to share computational research, but many published notebooks fail to re-execute in a different environment. The FAIR Jupyter pipeline containerises each repository before execution, yet it still only detects and logs the dependency-related failures that remain. This thesis adds a layer that explains and repairs them.

The working dataset is the residual set of 214 dependency-related failures recorded by the Docker-based pipeline. Of these, 200 fall inside a pip-only repair scope, split into 13 development records and 187 evaluation records reserved from prompt and repair-strategy tuning. A deterministic classifier decides each failure’s subtype and repair eligibility before any language model is consulted. An explanation component produces a plain-language, schema-validated description of every classified failure, whether or not it can be repaired. A repair component proposes a fix with an open language model whose suggestion is grounded in live metadata from the Python Package Index and validated against that evidence, **so a package name or version is never invented**. A fix-application component applies the proposal in a Docker environment rebuilt from the original recipe, re-executes the whole notebook, and classifies the result. When re-execution exposes a new error, the layer classifies and explains it too and, if it is repairable, attempts one bounded second repair round. Every repair-agent invocation is logged in a SQLite benchmark table, and the final run’s table was materialised through an RML mapping as RDF linked to the notebooks of the FAIR Jupyter knowledge graph. The implementation is covered by 711 automated tests.

The results were generated by the final frozen implementation over the 187 evaluation records with Gemma-2 9B. The explainer completed 160 of the 187 original-failure explanations and all 50 explanations of newly exposed errors; every completed response was schema-valid, and the 27 missing ones were runtime timeouts or a service error of the local model deployment. Of the 73 real repair attempts made across both rounds, 62 (84.9%) removed the specific dependency error they targeted, and all 73 repair proposals were schema-valid and PyPI-grounded. No notebook re-executed cleanly from start to finish within the two-round budget (0 of 187). Two limits explain this gap: the static import-to-distribution mapping let only 73 of 235 repair-agent invocations reach the model, the rest abstaining, and resolving one dependency error frequently exposed a further one.

Repeating the evaluation with Qwen3.6-35B-A3B-MLX-8bit in place of Gemma, with the pipeline held fixed and only model-specific serving settings changed, left every aggregate repair figure identical. The two models differed on 13 notebooks in which compatible NumPy release they selected to pin: Gemma chose 1.26.3 and

Qwen 1.26.4 from the same grounded candidate set, with no effect on any downstream repair outcome. Qwen completed all 237 explanation records under its serving environment, whereas the local Gemma run experienced runtime timeouts; since model and serving setup changed together, this does not isolate the model as the cause.

In a separate human study, 15 participants each rated six explanations drawn from a frozen pool of 12 original-failure explanations, on a six-item explanation-satisfaction questionnaire (90 explanation evaluations, 540 ratings): 88.9% were Agree or Strongly Agree, every item reached a majority-agree median, and satisfaction was the lowest item, matching free-text requests for more detail about the likely cause.

Keywords: Jupyter notebooks, computational reproducibility, large language models, retrieval-augmented generation, dependency repair

Contents

Contents	4
List of Figures	8
List of Tables	10
List of Abbreviations	12
1 Introduction	13
1.1 Motivation	13
1.2 The FAIR Jupyter Pipeline and Its Remaining Gap	13
1.3 Contribution of This Thesis	14
1.3.1 Objectives	15
1.3.2 Research Questions	15
1.3.3 Summary of Results	16
1.4 Thesis Structure	16
2 Background and Related Work	18
2.1 Reproducibility of Computational Notebooks	18
2.2 Restoring Executability	18
2.2.1 Static Analysis and Knowledge Graphs	19
2.2.2 LLMs for Dependency Resolution	19
2.2.3 LLM-Based Program Repair and Notebook-Specific Work	20
2.3 FAIR Principles and Knowledge Graphs	21
2.4 Research Gap	22
3 Problem Analysis and Dataset	24
3.1 Scale of the Problem	24
3.2 The Working Dataset	25
3.2.1 Why the Docker Pipeline, Not the Conda Run	25
3.2.2 Source and Filtering	25
3.2.3 Repair Scope: Why pip-Only	25
3.2.4 Explanation Scope Is Wider Than Repair Scope	27
3.2.5 The Most Common Failing Modules	27
3.3 Summary	28

CONTENTS

4	System Architecture	29
4.1	Design Overview	29
4.2	Integration Point	30
4.3	Component Architecture	31
4.3.1	ErrorClassifier	31
4.3.2	LLMExplainer	32
4.3.3	RAGRepairAgent	33
4.3.4	FixApplicator	33
4.3.5	ResultLogger	33
4.4	Orchestration and Control Flow	34
4.5	Data Contracts	35
4.5.1	Fix Object	35
4.5.2	The <code>repair_attempts</code> Table	36
4.5.3	Knowledge-Graph Enrichment	37
4.6	Demonstration Interface	38
5	Methodology and Implementation	41
5.1	Implementation Overview	41
5.2	Dataset Preparation	42
5.3	Context Extraction and Error Classification	43
5.3.1	Root-Cause Hints	43
5.3.2	Recovering Source Context	43
5.3.3	Limitations	44
5.4	LLM Explanation Component	44
5.4.1	Role and Separation from Repair	44
5.4.2	Model and Runtime Selection	45
5.4.3	Structured Explanation Output	45
5.4.4	Prompt, Validation, and Retry	46
5.4.5	A Real Explanation Record	47
5.5	PyPI-Grounded Repair Generation	47
5.5.1	Overall Flow	47
5.5.2	Layer 1: Deterministic Retrieval	48
5.5.3	Why PyPI Metadata Is Not Enough	49
5.5.4	Layers 3 and 4: Validation and Command Construction	50
5.5.5	Why Repository Dependency Files Are Not Used as Repair Evidence	51
5.5.6	Runtime Configuration	51
5.5.7	The Implemented Result Shape	51
5.6	Fix Application and Notebook Re-execution	52
5.6.1	Recovering Notebook Identity	52
5.6.2	Re-validating the Persisted Command	53
5.6.3	Rebuilding the Docker Environment	53
5.6.4	Commit Pinning	54
5.6.5	Fix Application Safety	54

CONTENTS

5.6.6	Outcome Classification	54
5.6.7	Why a Real Pilot Was Necessary	55
5.6.8	Environment Isolation and What Is Out of Scope	55
5.7	Orchestration and the Bounded Second Round	56
5.7.1	One Pass per Notebook	56
5.7.2	Reclassifying and Explaining a Newly Exposed Error	56
5.7.3	Why Round 2 Reapplies Round 1’s Fix	57
5.7.4	What Is Recorded	57
5.8	Result Logging and Knowledge-Graph Enrichment	58
5.8.1	Joining Four Outputs	58
5.8.2	Building One Row	59
5.8.3	Preserving Structured Evidence	59
5.8.4	From SQLite to CSV	60
5.8.5	The RML Mapping	60
5.8.6	Why No Crosswalk Table Is Needed	61
5.9	A Worked Example: One Record Through the Pipeline	62
5.10	Design Decisions and Trade-offs	64
6	Validation and Evaluation	67
6.1	Automated Testing	67
6.2	RAGRepairAgent Against Real PyPI and a Real Model	69
6.2.1	Retrieval Against Real PyPI	69
6.2.2	Repair Proposals Against a Real Model	69
6.2.3	What This Pilot Does and Does Not Show	70
6.3	FixApplicator Against Real Docker Execution	70
6.3.1	Key Observation: Fixing One Dependency Can Reveal Another	70
6.4	Result Logging and Knowledge-Graph Enrichment	72
6.4.1	Notebook-Identity Alignment	72
6.4.2	Executing the RML Mapping with Morph-KGC	73
6.4.3	Full-Scale Materialisation of the Final Evaluation Run	73
6.5	The Orchestrator End to End	75
6.6	Final Evaluation on the Reserved Evaluation Split	77
6.6.1	Evaluation Design	77
6.6.2	Reproducibility Manifest and Evaluation Freeze	78
6.6.3	Explanation Completion and Schema Validity	80
6.6.4	Primary Repair Results	81
6.6.5	Targeted Dependency-Error Resolution	81
6.6.6	Abstention: Round 1, Round 2, and Final State	82
6.6.7	Proposal Quality and Proposal Coverage	82
6.6.8	Infrastructure Failures	84
6.6.9	Final Outcome Breakdown	84
6.6.10	End-to-End Repair Case Study: Notebook 163	86
6.6.11	Results by Subtype	88
6.6.12	Manual Validation of the Classifier	88

CONTENTS

6.6.13	Manual Validation of PyPI Distribution Resolution	89
6.6.14	What Limits Full Recovery	89
6.6.15	LLM Model Sensitivity Analysis	90
6.7	Human Evaluation of Explanation Quality	93
6.7.1	Questionnaire	94
6.7.2	Study Design and Participants	94
6.7.3	Analysis and Limitations	96
6.7.4	Results	96
6.8	Threats to Validity	99
6.8.1	Internal Validity	99
6.8.2	Construct Validity	100
6.8.3	External Validity	100
6.8.4	Conclusion Validity	101
6.9	Chapter Summary	102
7	Future Work	104
7.1	Building on the Knowledge-Graph Enrichment	104
7.2	Widening Repair Coverage	105
7.3	Baseline Comparison	106
7.4	Extending the Human Evaluation	106
8	Conclusion	107
	Bibliography	110

List of Figures

3.1	Scope boundary over the 214-row working dataset. Every record enters classification and stays within the explanation objective O1, but only <code>missing_package</code> and <code>wrong_version</code> records are repair-eligible. Those 200 records split into the development and frozen evaluation sets.	27
4.1	Component architecture of the repair layer. A residual dependency failure from the Docker pipeline enters at the left and is classified and explained. It reaches RAGRepairAgent only if it is repair-eligible, and FixApplicator only if the agent could ground a proposal in retrieved PyPI evidence; otherwise the agent abstains and the abstention is logged. The dashed arrow is the bounded second round: an error newly exposed by re-execution re-enters at ErrorClassifier, is explained as its own record, and is repaired only if it is itself eligible. Explanation never feeds into repair. Two rounds is the hard cap. ResultLogger writes the SQLite table; the repair RDF is generated from its CSV export by the RML mapping (dashed).	32
4.2	Repair Workspace, entry stage, for notebook #164, cropped to the workspace area. The recorded execution path across the top marks the stage in view. The centre panel shows the captured failure, and the Final Result panel on the right summarises the outcome of the whole bounded repair process.	39
4.3	RAGRepairAgent stage for the same notebook, cropped to the centre panel of the workspace. The chain at the top traces the import name through package mapping and retrieved release evidence to the grounded proposal, next to the schema- and grounding-validation outcomes recorded for it.	39
4.4	Re-execution stage for the same notebook, cropped to the centre panel. The targeted <code>sklearn</code> error is resolved and a different dependency error (<code>pandas</code>) surfaces in its place. The recorded Round-2 trigger decision that follows is shown with its reason.	40
6.1	Traceability of the final evaluation. The frozen source revision, configuration, and evaluation IDs are recorded before execution. The resulting raw outputs are validated before the reported metrics and tables are computed.	79

LIST OF FIGURES

6.2	Two levels of the same final evaluation, for the frozen Gemma-2 9B run. Panel A follows every repair-agent invocation; Panel B classifies the 187 notebooks by their final outcome. The panels deliberately do not share one funnel, because they count different populations: 235 invocations against 187 notebooks.	85
6.3	Distribution of participant ratings for the six explanation-quality items, over all 90 explanation evaluations.	97

List of Tables

1.1	The six objectives this thesis is organised around.	15
2.1	Thesis objectives, the gap each one fills, and the prior work it builds on.	23
3.1	Dependency-error subtypes across the 214-row working dataset. . . .	26
3.2	Split policy over the 214-row working dataset.	26
3.3	The 20 most frequent failing modules in the 214-row working dataset, in descending order down the left column and continuing down the right. Two entries (<code>libXrender.so.1</code> , <code>libxcb.so.1</code>) are shared system libraries rather than Python modules, and fall outside the pip-only repair scope.	28
4.1	The five repair components and the orchestrator, each mapped to the requirement or objective it realises.	30
5.1	Curated API-compatibility evidence used for <code>wrong_version</code> repairs.	49
5.2	FixApplicator outcome values.	55
5.3	The design decisions with the largest effect on how the evaluation should be interpreted, each stated as decision, alternative, reason, and cost.	64
6.1	Automated tests by area. Counts are the collected test functions per area; all 711 pass.	68
6.2	Real-Docker pilot cases	71
6.3	Triples generated from the final evaluation table	74
6.4	Real orchestrator pilot: both cases exercise a genuine Round-2 trigger followed by a RAGRepairAgent abstention, each for a different recorded reason.	76
6.5	Explanation completion and schema validity for the final Gemma-2 9B run, counted per encountered error. “Original” covers the 187 original failures of the evaluation records. “Round 2” covers the 50 newly exposed errors reclassified after Round 1, including the two that were explained but were not repair-eligible. A runtime failure is a call that produced no processable response within one retry; it is not a schema-invalid model output. Explanation calls are never counted as repair-agent invocations.	80

LIST OF TABLES

6.6	The second round, from the 50 newly exposed errors to the 12 second repairs actually applied. Every newly exposed error was explained; only repair-eligible ones proceeded. Round-2 eligibility does not by itself mean a second repair was applied: for three of every four eligible notebooks, package resolution abstained before RAGRepairAgent could propose anything.	83
6.7	Final classification of all 187 evaluation rows, after up to two repair rounds.	85
6.8	Evaluation-split notebook execution id 163 traced through classification, explanation, two repair rounds, and the final outcome. Both attempted repairs remove their targeted dependency error, but the notebook remains failing after the two-round budget is exhausted. . .	87
6.9	Per-subtype precision, recall, and F1 on the 49-record manual validation sample.	88
6.10	Explanation completion and schema validity for both models, counted per encountered error (187 original failures + 50 newly exposed Round-2 errors = 237 records in each run). A runtime failure is a call that returned no processable response within one retry. Explanation calls are separate from the repair-agent calls of Table 6.11.	91
6.11	Gemma-2 9B vs. Qwen3.6-35B-A3B-MLX-8bit on the same 187 evaluation notebooks, same repair pipeline, same metric definitions. Denominators: abstentions and outcomes over 187 notebooks; validity and targeted resolution over the 73 repair attempts that reached the model; coverage over the 235 repair-agent invocations. Repair-LLM calls exclude explanation calls.	92
6.12	Results for the six adapted Explanation-Satisfaction-Scale items, over all 90 explanation evaluations (15 participants $\times$ 6 rated explanations each). IQR is the interquartile range on the 1–5 scale; %4–5 is the share of that item’s 90 ratings that were Agree or Strongly Agree. . .	97
6.13	Descriptive comparison of the two error subtypes, pooling all six items. “Evaluations” counts participant–explanation pairs (67 + 23 = 90) and “Ratings” counts those times six items (402 + 138 = 540). . .	98

List of Abbreviations

API	Application Programming Interface
APR	Automated Program Repair
FAIR	Findable, Accessible, Interoperable, Reusable
JSON	JavaScript Object Notation
KG	Knowledge Graph
LLM	Large Language Model
PMC	PubMed Central
PROV-O	PROV Ontology
PyPI	Python Package Index
RAG	Retrieval-Augmented Generation
RDF	Resource Description Framework
RML	RDF Mapping Language

1 Introduction

1.1 Motivation

Jupyter notebooks combine code, explanatory text, and results in a single document. This makes them a popular way to share computational research, especially in data-heavy fields such as biomedicine. A notebook published alongside a paper is meant to let another researcher see, and re-run, exactly how a result was produced.

In practice, many published notebooks do not run at all when someone else tries them. Pimentel et al. [14] collected 1.4 million notebooks from GitHub and attempted to execute a large subset of them: only 24.11% ran without errors, and only 4.03% reproduced the results reported by their original authors. Samuel and Mitichen [18] found a similar pattern specifically for notebooks accompanying biomedical publications on PubMed Central, the domain this thesis also builds on. These numbers show that non-reproducible notebooks are not a rare exception; they are the common case.

A large share of these failures is caused by dependencies, not by mistakes in the underlying research logic. A notebook is written against a particular set of installed Python packages. When it is later run in a different environment, a package may be missing entirely, or a newer version of a package may no longer provide the exact function the notebook expects. Both situations produce an error before the notebook can finish, even though the scientific code itself may be correct.

The resulting error messages are short and technical, for example `ModuleNotFoundError: No module named 'sklearn'`. A researcher who is not a software developer may not know from the message alone that the fix is to install a package called `scikit-learn`, not `sklearn`.

1.2 The FAIR Jupyter Pipeline and Its Remaining Gap

The FAIR Jupyter project has studied this problem at scale. Samuel and Mitichen [18] built a pipeline that downloads notebooks referenced by biomedical publications, executes them, and records whether each notebook ran and, if not, what error occurred. The results were organised into a queryable knowledge graph [19] so that patterns in notebook reproducibility could be studied directly. A more recent extension of the same pipeline [20] runs every repository inside its own Docker container before execution. Containerisation removes many failures that were really

about a missing system tool or an incompatible operating system, rather than about the research code.

Even this containerised pipeline stops at the point of *detecting* a failure. When a notebook still does not run inside its container, the pipeline records the exception type and message and moves on. It does not explain the failure to a human reader, and it does not attempt a fix. **Every notebook whose only problem is a missing or outdated Python package** is left exactly as broken as it was found. Yet such problems are, in principle, straightforward to repair, by installing or pinning the right package version.

This is the gap this thesis addresses: turning a detected, dependency-related notebook failure into an explained and, where possible, repaired one, automatically and without a person in the loop for every single notebook.

1.3 Contribution of This Thesis

This thesis adds a repair layer on top of the existing Docker-based FAIR Jupyter pipeline. The pipeline’s own contribution ends at logging a failure. This thesis begins exactly there. Given a notebook execution that failed with a dependency-related error, the added layer performs five steps:

1. **Classify** the failure: is it really a dependency problem, and if so, is it a missing package or an incompatible version?
2. **Explain** the failure in plain language, so that a researcher without a programming background can understand what went wrong.
3. **Propose a repair**, using an open, locally-run large language model (LLM) whose suggestion is grounded in live package metadata from the Python Package Index (PyPI) rather than left to guess a version number from training data.
4. **Apply the repair** inside a Docker environment rebuilt from the same recipe the notebook originally ran under, and re-run the whole notebook from the beginning.
5. **Record the outcome**: whether the notebook now runs further, still fails, or could not be meaningfully tested.

When re-execution exposes a new dependency error that was hidden behind the first, the layer classifies and explains that error too and, if it is repairable, attempts one bounded second repair round. Two rounds is the cap.

Grounding the repair suggestion in live PyPI data follows PLLM, the closest prior system to this one. Bartlett et al. [2] show that retrieval-augmented generation (RAG) against live package data measurably improves an LLM’s dependency-repair accuracy, compared to letting the model rely on what it happens to remember from

training. This thesis adopts the same principle, but applies it inside a reproducibility pipeline rather than to standalone Python scripts. It also adds an explanation step and a re-execution-based validation step, **neither of which PLLM has**.

1.3.1 Objectives

The work is organised around the six objectives in Table 1.1, fixed at the start of the project and used throughout this text to relate individual components back to the overall goal.

Objective	Description
O1	Generate a plain-language explanation of a dependency-related failure.
O2	Generate an automated, PyPI-grounded fix suggestion using an open LLM.
O3	Apply the suggested fix and validate it by re-executing the notebook.
O4	Compile a structured, reusable benchmark dataset of failures, fixes, and outcomes.
O5	Integrate the above into the existing FAIR Jupyter pipeline without manual intervention.
O6	Record repair outcomes as RDF triples linked to the FAIR Jupyter knowledge graph.

Table 1.1: The six objectives this thesis is organised around.

1.3.2 Research Questions

The objectives say what was built. Three research questions say what the evaluation in Chapter 6 asks of it, and Chapter 8 answers each in turn.

RQ1. **To what** extent can an open LLM produce understandable, structured explanations of the residual dependency-related failures recorded by the FAIR Jupyter pipeline? This is answered on two levels: automatically, by how many explanation calls completed and how many completed responses satisfied the explanation schema, and by a human study of how readers rated the explanations.

RQ2. How effectively can a PyPI-grounded, bounded repair pipeline resolve the targeted dependency error, and how often does that restore complete notebook execution? The two halves are kept apart throughout: targeted-error resolution is measured per repair attempt, full notebook recovery per notebook.

RQ3. How sensitive are the structured repair decisions and the measured repair outcomes to the choice of open LLM, when the deterministic retrieval, validation, and execution pipeline is held fixed and only model- and provider-specific serving settings change?

1.3.3 Summary of Results

All six objectives were implemented. The 187-row evaluation split described in Chapter 3 was reserved from prompt and repair-strategy tuning, and the results reported in Chapter 6 were generated by the final frozen implementation over that split, with a two-round budget, a real local language model (Gemma-2 9B), the live PyPI registry, and real Docker execution throughout. Four results frame the rest of this thesis and answer the three research questions.

First (RQ1), the explainer completed 160 of the 187 original-failure explanations and all 50 explanations of errors newly exposed by a first repair; every completed response satisfied the explanation schema, and the 27 missing ones were timeouts or a service error of the local model deployment. A separate human study found the original-failure explanations generally well received: 15 participants gave 540 ratings, of which 88.9% were Agree or Strongly Agree. Second (RQ2), 62 of the 73 repair attempts actually made (84.9%) removed the specific dependency error they targeted, and every repair proposal the model produced was schema-valid and grounded in retrieved PyPI evidence. Third (RQ2 again), no notebook in the evaluation split re-executed cleanly within the two-round budget. End-to-end recovery was limited by the repair agent’s package mapping, which resolved the failing import often enough to reach the model in under a third of its invocations, and by the fact that many evaluated notebooks exposed further dependency errors in sequence. Fourth (RQ3), repeating the evaluation with a different open model, Qwen3.6-35B-A3B-MLX-8bit, left every aggregate repair figure unchanged; the two models differed on 13 notebooks in which compatible NumPy release they selected to pin, with no effect on any measured repair outcome. Chapter 6 reports all four in full.

1.4 Thesis Structure

Chapter 2 reviews prior work on notebook reproducibility, automated dependency resolution, and LLM-based program repair, and states the specific gap this thesis fills. Chapter 3 describes the scale of the reproducibility problem and how the concrete working dataset for this thesis was obtained. Chapter 4 presents the system architecture: the five components introduced above, their contracts, and the data they exchange. Chapter 5 is the core of the thesis: it explains, component by component, how each part was actually built and the decisions made along the way. Chapter 6 reports the automated test suite, the real-system pilots run during implementation, the final evaluation over the 187-row evaluation split, the model-sensitivity experiment, the human evaluation of explanation quality, and the threats

to validity that bound all of these. Chapter 7 sets out what remains open. Chapter 8 closes the thesis by answering the three research questions.

2 Background and Related Work

This chapter reviews the work this thesis builds on and positions its contribution against it. It covers four areas: measuring notebook reproducibility, restoring it automatically, using large language models (LLMs) for dependency resolution and program repair, and the FAIR data principles that motivate recording repair outcomes in a knowledge graph. The chapter closes by stating, precisely, the gap none of this prior work closes on its own.

2.1 Reproducibility of Computational Notebooks

Computational reproducibility means that a published analysis can be re-executed by someone else and produce the same result. For a Jupyter notebook, the weaker and more basic question is whether it can be *re-executed at all*: does the code still run, independent of whether the numbers come out identical. This thesis targets that weaker property, re-executability, as the necessary first step before reproducibility in the fuller sense can even be discussed.

A large-scale study by Pimentel et al. [14] measured this directly. Out of 1.4 million notebooks collected from GitHub, only 24.11% of the notebooks they attempted ran without an error, and only 4.03% produced results matching those already stored in the notebook. Samuel and Mietchen [18] repeated this measurement for notebooks linked from biomedical publications on PubMed Central and found the same low reproducibility rate in that domain. This is the dataset this thesis' own working data is derived from, described in Chapter 3. Both studies agree on the dominant cause: most failures are not bugs in the analysis logic, but missing or incompatible software dependencies.

Detecting these failures is itself useful. Samuel and König-Ries [16] built ReproduceMeGit, a visualisation tool that runs the notebooks in a GitHub repository and shows, for each one, whether it succeeded, crashed, or produced different output than before. Tools like this make the scale of the problem visible to a repository owner, but they stop at reporting: they do not explain a failure in terms a non-programmer can act on, and they do not attempt to fix anything.

2.2 Restoring Executability

A separate line of work tries to go one step further and automatically restore a broken environment rather than only reporting that it is broken.

2.2.1 Static Analysis and Knowledge Graphs

Wang et al. [25] studied how execution environments could be reconstructed by analysing a notebook’s own code. They followed this with SnifferDog [26], a tool that statically inspects a notebook’s imports and matches them against a database of known package APIs. SnifferDog correctly inferred the required environment for 92.6% of the notebooks it was tested on. This result makes an important point that motivates the present thesis: a notebook that looks broken is often not broken in its logic at all. It simply lacks the right dependency information, which can in many cases be recovered automatically.

A separate family of tools builds this dependency knowledge as an explicit knowledge graph rather than analysing one notebook at a time. Horton and Parnin’s DockerizeMe [11] introduced this idea: it encodes package-to-module mappings from Libraries.io in a graph database and uses it to infer, and then containerise, the dependencies a Python snippet needs. Ye et al.’s PyEGo [32] extended the same idea to a much larger graph covering third-party packages, the Python interpreter version, and system libraries together, reporting a 0.4–3.5 $\times$ accuracy improvement over DockerizeMe. Cheng et al. added explicit conflict detection between transitive dependencies with PyCRE [4], and later refined the approach with ReadPyE [3], which reached 79.75% successful environment reconstruction, the strongest result among the knowledge-graph approaches reviewed here.

These knowledge-graph approaches share one structural limitation: they can only recommend a package or version that is represented in their indexed data. Such a graph must be maintained and updated as new PyPI releases appear, and a package or release that has not yet been indexed is unavailable to it until the graph is rebuilt. This limitation motivates replacing a fixed package-and-version snapshot with a live query against the package registry itself, which is the design this thesis adopts (Chapter 4).

2.2.2 LLMs for Dependency Resolution

Bartlett et al.’s PLLM [2] is the closest prior system to this thesis and its main point of comparison. PLLM uses an LLM, retrieval-augmented with live PyPI data, to resolve missing or incompatible Python dependencies from execution feedback. In their experiments, an LLM using retrieval-augmented generation (RAG) against live package data outperformed the strongest knowledge-graph baselines by a wide margin (+15.97% fixes over ReadPyE, +21.58% over PyEGo). Results were especially strong on notebooks that depend on machine-learning libraries – relevant here, since the dataset in Chapter 3 is dominated by exactly such packages (`scikit-learn`, `scipy`, `tensorflow`). Gao et al.’s RAG survey [6] and Tao et al.’s survey of retrieval-augmented code generation [24] both confirm that grounding a generative model’s output in retrieved, verifiable facts, rather than the model’s own memorised knowledge, is established practice for exactly this kind of task. This is the basis for Lewis et al.’s original formulation of retrieval-augmented generation [13].

PLLM differs from this thesis in four ways that matter for the present design: it operates on standalone Python files, not on notebooks embedded in a reproducibility pipeline; it uses unbounded multi-round iteration rather than a fixed number of repair attempts; it produces no human-readable explanation of the failure; and it does not feed its results into any knowledge graph. Section 2.4 returns to each of these points directly.

2.2.3 LLM-Based Program Repair and Notebook-Specific Work

More broadly, LLM-based automated program repair (APR) is an active field. Yang et al.’s survey [31] organises 63 systems published between 2022 and 2025 into four design paradigms: fine-tuning, prompting, procedural pipelines, and agentic frameworks. Agentic frameworks, where the model plans and re-plans its own next action, perform well on complex, multi-file bugs, but at a real cost in latency and control. That trade-off directly motivates a design decision explained in Chapter 4: this thesis structures its repair layer as a fixed, procedural pipeline, not an autonomous agent, because the target workload is hundreds of independent, mostly single-dependency notebook failures processed in batch, not a small number of complex interactive sessions. The same survey also confirms that retrieval-augmented context improves repair quality across every paradigm it studied, reinforcing the PyPI-grounding decision from Section 2.2.2.

Two lines of work are specific to notebooks. Grotov et al. [8] released a dataset of buggy notebooks, together with a proposed agent design. They followed it with a full implementation [7] that observes the output of each execution attempt, decides what to change, applies it, and tries again. That second paper establishes a point this thesis relies on directly: because a notebook keeps state between cells, a candidate fix can only be judged correct by re-running the *whole* notebook from the top, not just the cell that originally failed (Chapter 4 adopts this rule for the same reason). Xia and Zhang’s ChatRepair [30] similarly feeds re-execution feedback back into the next repair attempt, in a conversational loop, achieving 162 of 337 fixes on a general Python bug benchmark. The same feedback principle underlies the bounded two-round repair mode built for this thesis (Section 4.4), with one difference: the budget here is fixed at two rounds rather than left open. Reyes et al.’s Byam [15] and Tang et al.’s SynFix [23] both target dependency-aware repair outside the notebook setting: breaking dependency updates in Java, and structured cross-file repair, respectively. Both motivate giving the repair model structured dependency context rather than the bare error message. This thesis follows that principle in a different form. The repair prompt carries the classified error (type, message, failing module, subtype, root-cause hint), the failing cell’s source where it could be recovered, and release evidence retrieved live from PyPI; a repository’s declared requirements are used later, when FixApplicator reconstructs the execution environment, not as evidence handed to the repair model (Section 5.5.5).

On the interactive side, Jupyter AI [12] lets a user ask an LLM inside JupyterLab to explain an error or generate a fix. It is a useful point of contrast: Jupyter AI

is designed for one user working on one notebook at a time and must be triggered manually after every failure. It has no route into a batch reproducibility pipeline and no connection to the structured metadata such a pipeline already records. The system built for this thesis is designed for exactly the opposite situation: no user present, hundreds of notebooks, and existing structured failure metadata to build on.

Model and prompt choice also draw on this literature. Szalontai et al. [22] find that a smaller, code-specialised model can outperform a larger general-purpose one on bug-fixing tasks, and that prompt design affects repair quality about as much as model size does. A broader empirical study of LLM-based APR [21] confirms that code-specialised models tend to lead on Python repair specifically, across more than 600,000 patches. Dinh et al.’s comparison of prompting techniques for code generation [5] informs the structured-output, few-shot prompt design in Chapter 5. Chain-of-thought prompting [27] was considered and not adopted: the prompts request a structured JSON answer with few-shot examples rather than explicit intermediate reasoning, because the benefit of additional reasoning prompts for locally deployable models of this size is uncertain and a second prompt technique would have made the comparison less controlled. Dinh et al. in particular find that including a project’s declared package list in the prompt measurably improves dependency-aware generation. The repair prompt in this thesis applies the same idea to a different, verifiable source: instead of the project’s own package list, it supplies the candidate releases retrieved live from PyPI for the failing distribution, so that every version the model can choose is one that is known to exist.

2.3 FAIR Principles and Knowledge Graphs

The FAIR Jupyter project this thesis extends is itself grounded in the FAIR principles: that data, and by extension research software, should be Findable, Accessible, Interoperable, and Reusable [28]. Barker et al. [1] extend these principles specifically to research software. They argue that the Reusable principle for software requires recording the *provenance* of its execution environment – not only the code itself, but the conditions under which it did or did not run. Samuel and Mietchen’s FAIR Jupyter knowledge graph [19, 17] already stores exactly this kind of provenance for the original execution attempts. Recording a repair attempt in the same graph, using the PROV ontology [29] for the activity/agent/time structure, extends that existing provenance record to cover repair as well as detection. This is the basis for objective O6, designed in Chapter 4, implemented in Chapter 5, and validated in Chapter 6 first on pilot data and then over the complete table of the final evaluation.

2.4 Research Gap

Taken together, the literature above shows that the individual building blocks already exist separately. Pipelines exist that detect and record reproducibility failures at scale [20, 18, 19]. Tools exist that infer or restore a missing Python environment, from static analysis [26] to knowledge graphs [11, 32, 4, 3] to live-retrieval LLMs [2]. LLM agents exist that can fix errors in notebooks specifically, interactively, one session at a time [8, 7]. FAIR-aligned provenance recording exists for the original failures [19].

Among the systems reviewed for this thesis, none combines all of the following: (1) an LLM-based repair layer that runs autonomously, without a user manually triggering each attempt; (2) inside an existing reproducibility pipeline rather than on standalone files; (3) with the repair suggestion grounded in live PyPI data specifically for the pip-based dependency errors that pipeline already detects; and (4) paired with a plain-language explanation and, eventually, provenance recorded as RDF linked to the same knowledge graph the pipeline already produces.

PLLM [2] is the closest system in mechanism, but it is not pipeline-embedded, does not explain a failure, and does not write to a knowledge graph. Grotov et al.’s agents [8, 7] are the closest for notebooks specifically, but they assume an interactive session with a live kernel, not an autonomous batch pass over a pipeline’s own residual failures. Table 2.1 summarises this positioning against the thesis objectives introduced in Chapter 1.

This is the gap the remainder of this thesis addresses: a controlled, package-registry-grounded LLM repair layer for dependency failures in a FAIR Jupyter notebook pipeline, that also validates its own repairs by re-executing the notebook, rather than trusting the model’s suggestion on faith.

Objective	Gap it fills	Builds on
O1 – Explanation	None of the reviewed pipelines explains a residual failure in plain language	Grotov [8, 7], Jupyter AI [12]
O2 – PyPI-grounded repair	None of the reviewed systems embeds PyPI-grounded RAG repair inside a reproducibility pipeline	PLLM [2], Lewis et al. [13]
O3 – Fix validation	Autonomous, batch re-execution validation is not done by the reviewed notebook agents	ChatRepair [30], Grotov [7]
O4 – Benchmark dataset	No reviewed benchmark covers dependency repair on biomedical notebooks	Grotov [8], Pimentel [14]
O5 – Pipeline integration	The FAIR Jupyter pipeline stops at logging; no repair layer exists yet	Samuel et al. [20, 18, 19]
O6 – KG enrichment	None of the reviewed systems records repair provenance as FAIR RDF triples	[19], PROV-O [29], [28]

Table 2.1: Thesis objectives, the gap each one fills, and the prior work it builds on.

3 Problem Analysis and Dataset

This chapter answers two questions and keeps them apart. How large is the reproducibility problem in general, and what exactly does this thesis’s implementation operate on? The first is answered by numbers from the original, full-scale FAIR Jupyter study. The second is answered by a much smaller, precisely defined dataset drawn from the newer Docker-based pipeline. The two are related, because the second is a residual subset of the same kind of failure, but they are not the same numbers and are never mixed here.

3.1 Scale of the Problem

The original FAIR Jupyter study executed every notebook it collected using a fresh conda environment per repository [18]. The figures below were recomputed for this thesis from the published database of that study [17], using the local snapshot of its `executions` table. A notebook counts as executed when its execution record is neither an environment-installation failure nor a skipped notebook, and the two exception counts are taken from the recorded failure reason of each execution:

- Total notebooks executed: **10,389**
- `ModuleNotFoundError`: 5,562 (53.5%)
- `ImportError`: 1,014 (9.8%)
- Combined dependency-related failures: **6,576 (63.3%)**

The published article itself reports 10,388 notebooks attempted for re-execution and 6,588 `ModuleNotFoundError` and `ImportError` exceptions, computed over a different denominator [18]; the small differences reflect the counting rule stated above, not a different dataset. In other words, roughly two out of every three executed notebooks failed for a dependency-related reason. The missing modules were dominated by biomedical and machine-learning packages, consistent with the PubMed Central origin of the dataset: `anndata` (1258 occurrences), `scanpy` (423), `pandas` (177), `cemba_data` (176), `tensorflow` (149), `Bio` (108), and `fastai` (103) were the most frequently missing modules in that snapshot.

These numbers exist to establish that dependency failure is a large-scale, systematic problem, not an edge case worth ignoring. **They are not the dataset this thesis’s implementation runs on.** They come from the earlier conda-based run of the pipeline, before the Docker-based extension described next.

3.2 The Working Dataset

3.2.1 Why the Docker Pipeline, Not the Conda Run

The newer, containerised pipeline [20] re-executes each repository inside its own Docker container. **Containerisation** resolves many of the failures counted in Section 3.1 that were really caused by a missing system tool or an operating-system mismatch, not by the notebook’s Python dependencies. What is left over after containerisation – notebooks that still fail even inside a container built specifically for them – is a cleaner, more targeted signal of a genuine dependency problem. This is why the Docker pipeline, not the original conda run, was chosen as the integration target. Its residual failures are exactly the gap this thesis is meant to close. And its result schema already tags each failure with a pre-computed `error_category`, which removes the need to re-implement failure detection from scratch.

3.2.2 Source and Filtering

The working dataset is derived from the Docker pipeline’s own result database (`notebook_executions` table). Its existing categorisation routine already maps two exception types into a bucket called `DEPENDENCY_ERROR: ModuleNotFoundError` and `ImportError`. These two types were kept, unchanged, as this thesis’s scope, for two reasons: they are exactly what the upstream pipeline already flags as dependency-related, and, taken together, they cover both a package being completely absent and a package being present but missing a specific name the notebook tries to import.

Filtering every row where `error_category = 'DEPENDENCY_ERROR'` yields **214** rows: 172 `ModuleNotFoundError` and 42 `ImportError`. This extraction and every count in this section is produced by `scripts/prepare_dependency_dataset.py`, and is written out as a CSV file plus a machine-readable statistics file so that the numbers can be independently recomputed.

3.2.3 Repair Scope: Why pip-Only

Not every one of the 214 rows can be repaired the same way. Version 1 of the repair layer is restricted, deliberately, to dependency problems that a single `pip install` command can fix: installing a missing package, or pinning a specific version. This scope decision was made for a concrete reason: it keeps the first version of the system controlled and measurable. A repair action that also has to reason about system-level packages (installed with `apt`, not `pip`) or about renaming an import path inside the notebook’s own code introduces a second, much larger kind of decision that this thesis does not attempt to automate yet.

Applying this scope splits the 214 rows into the four subtypes of Table 3.1, and those in turn into two groups.

The 10 `system_library` rows are excluded because they need `apt`-level installation, which is outside the `pip`-only scope by definition. A missing shared library

Subtype	Count	Meaning
<code>missing_package</code>	179	The imported module is genuinely absent.
<code>wrong_version</code>	21	The module is present, but the notebook imports a name that no longer exists in the installed version.
<code>system_library</code>	10	A missing shared object file (e.g. <code>libxcb.so.1</code>), installable only with <code>apt</code> , not <code>pip</code> .
<code>mapping_unknown</code>	4	An ambiguous local or path-shadowed name (e.g. <code>utils</code>) that cannot safely be mapped to a PyPI package.

Table 3.1: Dependency-error subtypes across the 214-row working dataset.

cannot be fixed by any `pip` command, however it is phrased. The 4 `mapping_unknown` rows are excluded because their failing name (for example a generic module called `utils`) cannot be safely resolved to one specific PyPI package without guessing. Guessing a package name is exactly the kind of unfounded suggestion this thesis’s PyPI-grounding design (Chapter 4) is built to avoid. This leaves **200 usable** rows and **14 excluded** rows.

Split	Count	Purpose
<code>dev</code>	13	Used to design and manually sanity-check the explanation and repair prompts (Chapter 5).
<code>evaluation</code>	187	Usable rows reserved for the final repair evaluation, reported in Chapter 6.
<code>excluded</code>	14	Within explanation scope; excluded from automated repair (Section 3.2.4).

Table 3.2: Split policy over the 214-row working dataset.

Table 3.2 shows how the 214 rows divide across the three resulting splits. The 13-row `dev` split was used exclusively during implementation, to design and manually sanity-check the explanation and repair prompts and to run the small real-system pilots reported in Chapter 6. The 187-row `evaluation` split was reserved from prompt and repair-strategy tuning: no prompt, mapping entry, or repair rule was designed against it. The results reported in Chapter 6 were generated using the final frozen implementation described in Chapter 5.

3.2.4 Explanation Scope Is Wider Than Repair Scope

One further decision affects every later chapter and is worth stating clearly. The pip-only restriction above governs *repair* eligibility only. It does not restrict *explanation*. All 214 rows, including the 14 excluded ones, stay within the explanation scope defined for objective O1. Only the 200 usable rows are additionally in scope for the repair-proposal and fix-application components (objectives O2 and O3). A researcher whose notebook fails because of a missing system library still deserves to be told what went wrong, even though the current repair layer cannot fix that case automatically.

Figure 3.1 shows this scope boundary in one place. Every one of the 214 records enters classification and stays within the explanation objective, while only 200 are repair-eligible and feed the rest of the repair layer.

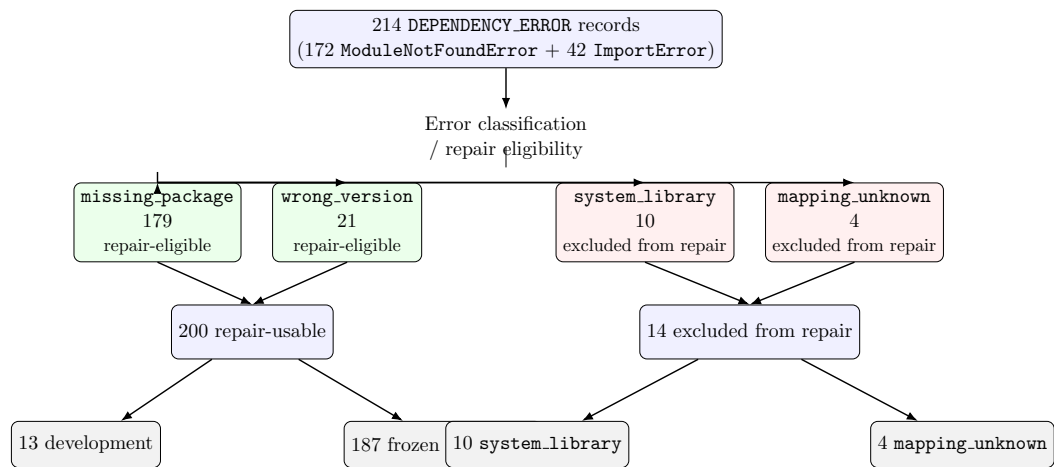


Figure 3.1: Scope boundary over the 214-row working dataset. Every record enters classification and stays within the explanation objective O1, but only `missing_package` and `wrong_version` records are repair-eligible. Those 200 records split into the development and frozen evaluation sets.

3.2.5 The Most Common Failing Modules

Several of these entries already hint at a recurring implementation problem addressed in Chapter 5: `sklearn` is not the PyPI package name (it is `scikit-learn`), and `pkg_resources` is not an installable package at all but a module bundled with `setuptools`. A repair layer that assumed a Python import name is always also its own pip package name would fail on exactly the most common cases in this dataset.

Module	Occurrences	Module	Occurrences
pkg_resources	22	google	5
sklearn	16	gnomad	5
rpy2	15	omero	5
numpy	14	dms_variants	4
cana	12	bindingcalculator	4
celloracle	12	libXrender.so.1	4
keras	7	statsmodels	4
scipy	7	openslide-bin	4
skimage	6	libxcb.so.1	3
physicool	5	bokeh	3

Table 3.3: The 20 most frequent failing modules in the 214-row working dataset, in descending order down the left column and continuing down the right. Two entries (`libXrender.so.1`, `libxcb.so.1`) are shared system libraries rather than Python modules, and fall outside the pip-only repair scope.

3.3 Summary

The reproducibility problem is large at the scale of the full conda-based FAIR Jupyter run (Section 3.1: 6,576 dependency-related failures out of 10,389 executed notebooks in the local snapshot of the published dataset). This thesis does not attempt to repair that entire set. It operates on a precisely defined, much smaller residual subset: the 214 dependency-error rows still present after the newer Docker pipeline’s containerisation, of which 200 fall inside the pip-only v1 repair scope. Of those 200 rows, 13 were used only for prompt design and small real-system pilots, and the remaining 187 were reserved for the final repair evaluation reported in Chapter 6.

Keeping these numbers distinct throughout the rest of this thesis avoids overstating what has actually been measured.

4 System Architecture

This chapter specifies the design of the LLM-based repair layer that this thesis adds to the FAIR Jupyter reproducibility pipeline [18]. The existing pipeline ends at detecting and logging notebook execution failures. Even its containerised evolution [20] stops at the point of execution: it records the residual failure, but does not explain or repair it. The components described here attach at that point. They carry a dependency-related failure through explanation, repair, re-execution, and structured logging.

Following the taxonomy of Yang et al. [31], the repair layer is built as a *procedural pipeline*, not an agentic framework: a fixed sequence of components, driven by a deterministic orchestrator. This choice is deliberate. Agentic loops achieve strong results on complex, multi-file bugs, but their latency and control overhead scale poorly across a batch of dependency-error notebooks. The interactive notebook-repair agents of Grotov et al. [8, 7] also assume a live kernel and a user in the loop – neither is available in autonomous batch processing.

Five non-functional requirements shape the design, and are referred to by these labels throughout the thesis. Components rely only on open, locally deployable models (NFR1). Explanations are written in plain language for a non-expert reader (NFR2). Everything is documented and version-controlled to support traceability and reproducibility of a run (NFR3). The layer processes its full target set without manual intervention (NFR4). And components are modular, with an injected configuration surface rather than hard-coded settings (NFR5). One functional requirement is also referred to by a label: FR1. It determines whether a detected failure is a dependency error that falls within the system’s pip-based repair scope.

The scope of this chapter is the *contracts* between components and the *data* they exchange. Prompt design, model selection, and the internals of the PyPI-grounded retrieval step belong to Sections 5.4 and 5.5. How each component was actually built is the subject of Chapter 5 as a whole.

4.1 Design Overview

The system can be stated in one sentence: *when a notebook has failed because of a dependency error, read that failure, explain it in plain language, derive a fix from live PyPI data, apply the fix and re-run the notebook to check it, and record the result.* That sentence is partitioned into the five components of Table 4.1, each realising one thesis objective, sequenced by an orchestrator that realises pipeline integration (O5).

Component	Requirement or objective	Responsibility
ErrorClassifier	FR1	Classify the dependency failure, determine repair eligibility, and identify the failing module; also reclassify an error newly exposed by re-execution.
LLMExplainer	O1	Produce a plain-language explanation of each classified error, in Round 1 and for a newly exposed Round-2 error.
RAGRepairAgent	O2	Derive a concrete fix, grounded in live PyPI metadata, for a repair-eligible record.
FixApplicator	O3	Apply the fix and re-execute the notebook to validate it.
ResultLogger	O4, O6	Store repair attempts in SQLite and export them for knowledge-graph enrichment.
Orchestrator	O5	Sequence the five components; drive the bounded second round; expose the round and configuration switches.

Table 4.1: The five repair components and the orchestrator, each mapped to the requirement or objective it realises.

4.2 Integration Point

The containerised FAIR Jupyter pipeline [20] executes the notebooks of each repository inside a per-repository Docker container. It writes one row per execution to the `notebook_executions` table, and tags each failure with an `error_category`. A pre-existing categorisation routine (`summary.py`) already maps `ModuleNotFoundError` and `ImportError` to `DEPENDENCY_ERROR`. The repair layer activates on exactly those rows: it performs a batch pass over the `notebook_executions` rows whose `error_category = DEPENDENCY_ERROR`.

Re-execution substrate. A candidate fix is validated by re-running the notebook from beginning to end inside a newly built Docker environment. The environment uses the same Python base image and installation procedure as the upstream Docker pipeline (Section 5.6.3). This reduces differences between the original and repaired executions. However, it does not reproduce the original environment exactly. The repository is cloned again when the repair is tested, and the final evaluation used

its current default branch rather than the exact revision that originally failed (Section 6.6.2).

Context available at the hook-in point. Three kinds of context are distinguished. Directly available, from the Docker pipeline’s result row, are `error_type`, `error_message`, `error_cell_index`, `notebook_id`, `repository_id`, `notebook_name`, `url`, `total_code_cells`, `executed_cells` (and `error_category`); these form the raw input to the repair layer. Additional context can be recovered when needed. A legacy execution table may provide a traceback, and the notebook source can be fetched again from GitHub (Section 5.3). These sources are treated with lower confidence because they may not correspond exactly to the original failed execution. **The original repository clone and container were not preserved.** Therefore, when a repair is tested, FixApplicator clones the repository again and rebuilds the Docker environment (Section 5.6.3).

Working set. The rows the repair layer activates on are the 214 `DEPENDENCY_ERROR` records of Chapter 3, of which 200 fall inside the pip-only repair scope. Those counts, the subtypes behind them, and the development/evaluation split of the 200 are all fixed there and are not restated in this chapter.

4.3 Component Architecture

Each component is defined below by what it does, which Table 4.1 already ties to a requirement or objective, and by its input/output contract. Implementation is deferred to Chapter 5.

4.3.1 ErrorClassifier

ErrorClassifier decides whether a failure is an in-scope dependency error eligible for pip-based repair and, if so, extracts the failing module and its subtype. It reuses the pipeline’s existing `error_category` tag as a first filter and refines the decision from `error_type` and `error_message`. The same rules are applied a second time when re-execution exposes a new error.

Input. A `notebook_executions` row (the fields listed in Section 4.2), or, in the second round, the new error type and message that FixApplicator observed after re-execution.

Output. A classification carrying three values: `failing_module`, `subtype`, and `scope_status`. For an original `DEPENDENCY_ERROR` record the subtype is one of the four dependency-related values `missing_package`, `wrong_version`, `system_library`, or `mapping_unknown`, and the scope status is `usable` or `excluded`. A newly exposed error is reclassified with the same rules; if it is not a dependency error at all (in the final evaluation, an `IndexError`), its subtype is `out_of_scope` and it is `excluded` from repair. It is still explained.

4 System Architecture

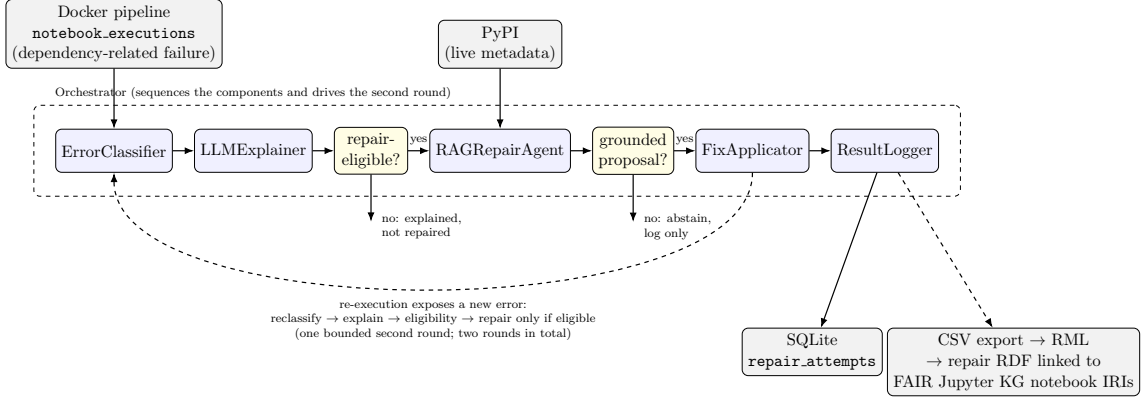


Figure 4.1: Component architecture of the repair layer. A residual dependency failure from the Docker pipeline enters at the left and is classified and explained. It reaches RAGRepairAgent only if it is repair-eligible, and FixApplicator only if the agent could ground a proposal in retrieved PyPI evidence; otherwise the agent abstains and the abstention is logged. The dashed arrow is the bounded second round: an error newly exposed by re-execution re-enters at ErrorClassifier, is explained as its own record, and is repaired only if it is itself eligible. Explanation never feeds into repair. Two rounds is the hard cap. ResultLogger writes the SQLite table; the repair RDF is generated from its CSV export by the RML mapping (dashed).

Explanation is not gated by scope. `scope_status` decides whether RAGRepairAgent and FixApplicator run on a row, not whether LLMExplainer does. Every classified row, including an `excluded` one, is still passed to LLMExplainer, for the reason given in Section 3.2.4: a researcher whose failure falls outside the pip-only repair scope still deserves an explanation of what went wrong.

4.3.2 LLMExplainer

LLMExplainer generates a plain-language explanation of the classified error for a non-expert reader (NFR2).

Input. One classified record: the error context (error type, message, failing module, relevant cell) of the original failure in Round 1, or of the newly exposed and reclassified error in Round 2.

Output. A schema-validated explanation object and the identifier of the model that produced it (`model_used`), forwarded to the ResultLogger for logging in the `explanation` and `llm_model` columns of `repair_attempts`.

Independence from repair. The explanation is produced for the reader, not for the pipeline. RAGRepairAgent never receives it, and a failed explanation call does not stop an otherwise repair-eligible record from being repaired. This holds in both rounds.

The prompt template and model are defined in Chapter 5; this component’s contract is fixed here, its prompt internals are not.

4.3.3 RAGRepairAgent

RAGRepairAgent derives a concrete repair for a classified, repair-eligible error. Following PLLM [2], the proposal is grounded in live PyPI data rather than relying only on the model’s training knowledge.

Input. The classified error record, including the error type and message, failing module, subtype, and root-cause hint, together with the PyPI evidence retrieved for the resolved package. In Round 2, the same contract is applied to the newly exposed and reclassified error.

Output. A *fix object* (Section 4.5.1), or an abstention when the available evidence is insufficient to support a repair.

The retrieval process, version selection, use of compatibility evidence, and the decision not to use repository dependency declarations as repair evidence are described in Section 5.5.

4.3.4 FixApplicator

FixApplicator applies the validated repair inside a rebuilt Docker environment and re-executes the notebook to check whether the repair worked. The notebook is always run from the beginning, because notebook state can depend on earlier cells and a partial re-run would not provide a reliable validation [7].

Input. A fix object together with the notebook and repository information needed to rebuild the execution environment. Repository dependency files such as `requirements.txt` and `setup.py`, when available, are used during this reconstruction (Section 5.6.3). In Round 2, the rebuilt environment first reapplies the validated Round-1 fix before applying the Round-2 repair, so both fixes are tested together (Section 5.7.3).

Output. An outcome $\in \{\text{fixed}, \text{still_failing}, \text{apply_error}\}$, together with the new error type and message when execution still fails, and an indication of whether the new error matches the one targeted by the repair.

4.3.5 ResultLogger

ResultLogger persists structured repair-attempt data. It is the source of the benchmark dataset (O4) and, through its CSV export, the input to the knowledge-graph mapping (O6); it does not write RDF itself.

Input. The classification, explanation, fix object, outcome, and run metadata of one repair-decision entry.

Output. One `repair_attempts` row per repair-decision entry (Section 4.5.2), whether the agent proposed a fix or abstained. Normally such an entry is a RAGRepairAgent invocation; an original out-of-scope record is represented by a synthesised

abstention entry without an agent or model call. A Round-1 row carries the explanation of the original failure; a Round-2 row carries the explanation of the newly exposed error that round targeted. A CSV export of the table is the logical source of the RML mapping (Section 4.5.3), which was pilot-validated and then run, unchanged, over the complete table of the final evaluation (Section 6.4.3).

How the four upstream outputs are joined into that single row is an implementation concern rather than a contract one, and is described in Section 5.8.

4.4 Orchestration and Control Flow

The orchestrator iterates over the target rows and runs the components in order: classify, explain, check repair eligibility, repair, apply, log. A repair-agent invocation does not always lead to a fix application: when retrieval cannot ground a candidate, RAGRepairAgent abstains, nothing is applied, and the abstention is logged as its own row. Three properties are fixed at this level.

The orchestrator calls each of ErrorClassifier through ResultLogger through that component’s own interface, rather than reimplementing any of them. It adds only two things of its own: the bounded second round described below, and a single run identifier shared by every call it makes for a given row.

The bounded second round. The default is a single repair round. In two-round mode, a second round is considered when Round 1 reached FixApplicator, the notebook came back `still_failing`, and the error it now fails on is a genuinely new one rather than the original error again. The new error is then treated as a record of its own and walks the same path as an original failure. ErrorClassifier reclassifies it. LLMExplainer explains it, so that the reader receives an account of every error the pipeline encountered, not only the first. Only then is repair eligibility evaluated: a newly exposed `missing_package` or `wrong_version` error proceeds to RAGRepairAgent and FixApplicator, while a newly exposed `system_library`, `mapping_unknown`, or non-dependency error is explained but not repaired. The explanation step is independent of the repair step in both directions: RAGRepairAgent never sees the explanation, and a failed explanation call does not prevent an eligible record from being repaired. The repair process stops after the second round. An error exposed by the Round-2 re-execution is recorded but is neither reclassified, explained, nor repaired.

This feedback structure follows the re-execution-based repair approach of ChatRepair [30], but uses a fixed two-round budget instead of an open loop. The evaluation does not run separate one-round and two-round experiments. Instead, both outcomes are taken from the same two-round execution. This avoids introducing extra variation from the language model’s non-zero sampling temperature. The Round-1 result is therefore kept unchanged, while Round 2 only extends it when a new eligible error is exposed. Chapter 6 compares the Round-1 outcome with the final outcome after up to two rounds.

Failure isolation (NFR4). Every attempt is logged, even when a component itself errors – malformed LLM output, an unreachable PyPI, or a re-execution timeout. Such cases are caught and recorded with the appropriate `outcome`; a single problematic notebook never aborts the batch.

Configuration surface (NFR5). The model, the prompt strategy, and the round count are injected per run rather than hard-coded, so they can be swapped without changing component code. The same injection points drive the model and prompt comparisons in the evaluation.

```
for row in notebook_executions with DEPENDENCY_ERROR:
    rec = ErrorClassifier(row)          # FR1: subtype, scope
    expl = LLME explainer(rec)         # 01: every record
    if not rec.repair_eligible:         # explained, not repaired
        stub = build_excluded_repair_stub(rec) # no agent call
        log(rec, expl, stub, None, round=1)
        continue
    fix = RAGRepairAgent(rec, PyPI)     # 02: proposal or abstain
    out = FixApplicator(fix, env) if fix else None # 03
    log(rec, expl, fix, out, round=1)   # row even if abstained

    if two_round and out == 'still_failing' and out.new_error:
        rec2 = ErrorClassifier(out.new_error) # reclassify
        expl2 = LLME explainer(rec2)         # explain, always
        if not rec2.repair_eligible:
            continue # trace only, no Round 2
        fix2 = RAGRepairAgent(rec2, PyPI)
        out2 = FixApplicator(fix2, env + fix) if fix2 else None
        log(rec2, expl2, fix2, out2, round=2) # cumulative env
    # hard cap: no third round
```

A failure inside `LLME explainer` is caught and recorded as a failed explanation; it does not raise past the call, so the lines that follow it run regardless.

4.5 Data Contracts

4.5.1 Fix Object

The `RAGRepairAgent` emits the *proposed* fix as structured JSON, before application. Version 1 is pip-only: `action` is one of `install`, `pin_version`, or `none`; `version` is null for a plain install; and `command` is exactly what the `FixApplicator` runs inside the container. Notebook-code edits (`replace_import`) are deferred to future work – in practice many renamed-import cases are still resolved by pinning an older version where the import was valid, which remains a pip fix.

The example below shows the object's meaningful content, not its literal persisted field names. The implemented record (Section 5.5) carries the same four values – action, resolved install name, version, and the evidence behind them – but names and nests them differently.

```
{
  "action": "pin_version",
  "import_name": "scanpy",
  "install_name": "scanpy",
  "version": "1.9.3",
  "command": "python -m pip install scanpy==1.9.3",
  "rationale": "Notebook needs an API present only up to 1.9.x;
               pinning the last compatible release.",
  "pypi_evidence": {
    "latest_version": "1.10.1",
    "chosen_version": "1.9.3",
    "requires_python": ">=3.9"
  }
}
```

4.5.2 The repair_attempts Table

The repair layer writes to a single new SQLite table, `repair_attempts`, which serves a dual purpose: it is the live validation log (O3) and the reusable benchmark dataset (O4). It records *one row per repair-decision entry*: an abstention is a row with `action = none` and no outcome, a two-round repair produces two rows, and re-running the same targets under a different model or prompt produces further rows. Normally an entry corresponds to a `RAGRepairAgent` invocation. The one exception is an original record outside repair scope: it never reaches the agent, and the orchestrator records a synthesised abstention entry for it instead, so that every original record is represented (Section 5.8.1). In the final evaluation this gives 235 rows per run (187 Round-1 and 48 Round-2 invocations), of which 73 carry an executed fix application and its outcome. The error data is never copied – each row joins back to the originating failure via `notebook_execution_id`. Filtering on `round`, `llm_model`, `prompt_strategy`, and `run_id` is what powers the ablation and the model/prompt comparisons.

```
CREATE TABLE repair_attempts (
  id                INTEGER PRIMARY KEY,
  notebook_execution_id  INTEGER NOT NULL,
                                -- FK -> notebook_executions

  -- from ErrorClassifier
  failing_module    TEXT,          -- e.g. "scanpy"
```

```

subtype                TEXT,    -- missing_package | wrong_version

-- from LLMExplainer (01): this round's explanation
explanation              TEXT,

-- from RAGRepairAgent -- the proposed fix (02)
action                  TEXT,    -- install | pin_version | none
install_name            TEXT,    -- resolved PyPI name
version                 TEXT,    -- version to pin, or NULL
command                 TEXT,    -- exact command run
rationale               TEXT,
pypi_evidence           TEXT,    -- JSON retrieval evidence

-- from FixApplicator -- the outcome (03)
outcome                 TEXT,    -- fixed|still_failing|apply_error
new_error_type          TEXT,
new_error_message       TEXT,

-- run metadata (for experiments)
llm_model               TEXT,
prompt_strategy         TEXT,    -- for prompt comparison
round                   INTEGER,  -- 1 or 2 (hard cap)
run_id                  TEXT,    -- groups one run / config
created_at              TEXT,

FOREIGN KEY (notebook_execution_id)
REFERENCES notebook_executions(id)
);

```

Section 5.8 describes how each of the eighteen non-id columns is populated from the four upstream output files, including how a `notebook_execution_id` that legitimately repeats across separate attempts is kept as separate rows rather than collapsed.

4.5.3 Knowledge-Graph Enrichment

The FAIR Jupyter knowledge graph [19] is not written by hand. It is generated from one RML mapping per table, run through Morph-KGC by the project's existing build script. Objective O6 adds a single new mapping for the `repair_attempts` table, modelled on the existing mapping for the `executions` table. It re-expresses each row as triples whose notebook IRI matches the notebook resource used by the existing FAIR Jupyter knowledge graph. When the generated repair RDF is combined with or loaded alongside the upstream graph, the repair history can therefore be queried together with the existing reproducibility data. Following the existing

CSV-based RML architecture, rather than introducing a separate RDF-generation pipeline, keeps the repair data readable by the same tooling as everything else in the graph.

The vocabulary strategy follows the earlier design notes: *reuse* existing `repr:` terms where they already fit (e.g. `repr:exception`), *add* new `repr:` terms for repair-specific facts, and use PROV-O [29] for the activity/agent/time layer, representing each repair as a `prov:Activity`. This serialisation of repair provenance directly serves the Reusable principle of FAIR [28] and the execution-environment provenance emphasised by FAIR4RS [1]. The base IRI is `repr: = https://w3id.org/reproduceme/` and the serialisation is Turtle (`.ttl`).

```
repr:notebook_512  repr:hadRepairAttempt  repr:repairattempt_001 .

repr:repairattempt_001
    a                                prov:Activity, repr:RepairAttempt ;
    repr:exception                   "ModuleNotFoundError" ; # existing term
    repr:failingModule               "scanpy" ;                # new term
    repr:fixCommand                  "python -m pip install scanpy==1.9.3" ;
    repr:fixOutcome                  "fixed" ;
    repr:llmModel                    "gemma2:9b" ;
    prov:endedAtTime                 "2026-07-15T14:22:00Z"^^xsd:dateTime .
```

4.6 Demonstration Interface

A demonstration interface was built alongside the pipeline to make its executions and repair results easier to inspect than the raw output files allow. It is a small, static web application with two views. A dashboard lists every notebook record of the final evaluation run, with filters for subtype, final outcome, and Round-2 state. Selecting a record opens a per-notebook Repair Workspace that steps through the stages this chapter defines, from the original failure through classification, explanation, repair, fix application, and re-execution to the final result, including a second round where one occurred.

This interface is a visualisation layer, not a sixth pipeline component. It only displays artefacts the pipeline has already produced, and issues no requests of its own to Ollama, PyPI, GitHub, or Docker. Nothing it does can influence a repair decision. It played no part in the quantitative evaluation of Chapter 6, and no usability evaluation of the interface itself is claimed.

Figures 4.2 to 4.4 show three successive stages of the same recorded execution, notebook #164, whose Round-1 repair of `sklearn` succeeds in removing the targeted error while the notebook as a whole continues to fail on a later, different dependency error – the distinction analysed quantitatively in Chapter 6.

4 System Architecture

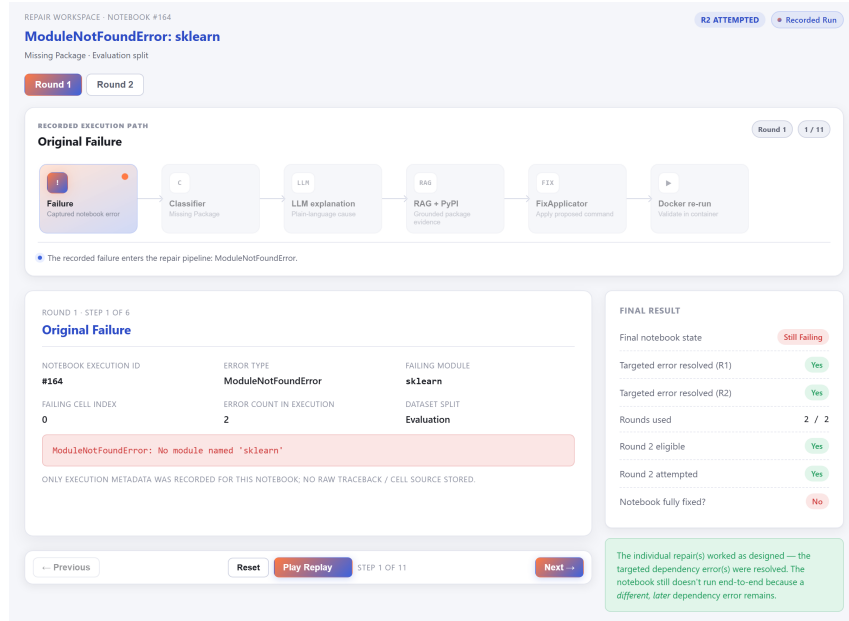


Figure 4.2: Repair Workspace, entry stage, for notebook #164, cropped to the workspace area. The recorded execution path across the top marks the stage in view. The centre panel shows the captured failure, and the Final Result panel on the right summarises the outcome of the whole bounded repair process.

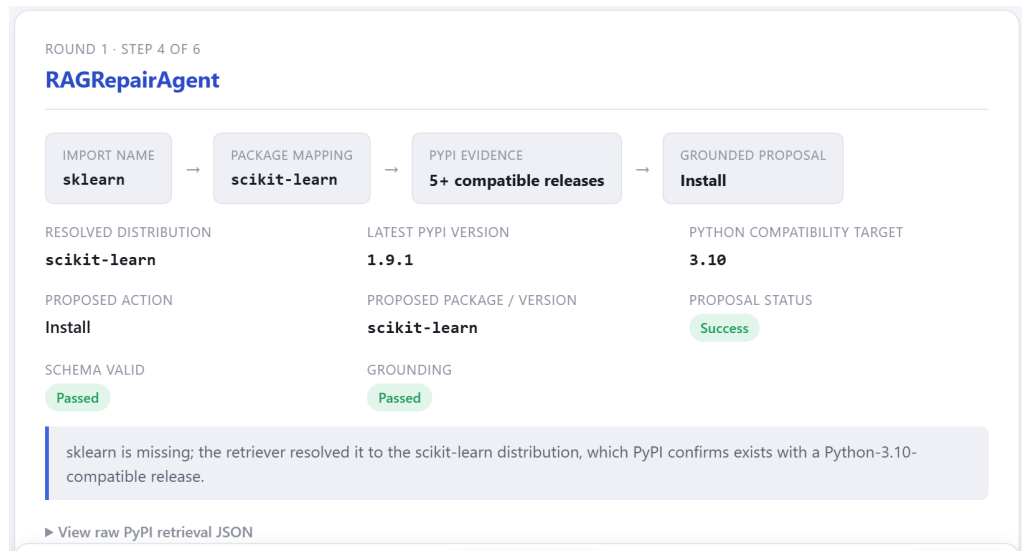


Figure 4.3: RAGRepairAgent stage for the same notebook, cropped to the centre panel of the workspace. The chain at the top traces the import name through package mapping and retrieved release evidence to the grounded proposal, next to the schema- and grounding-validation outcomes recorded for it.

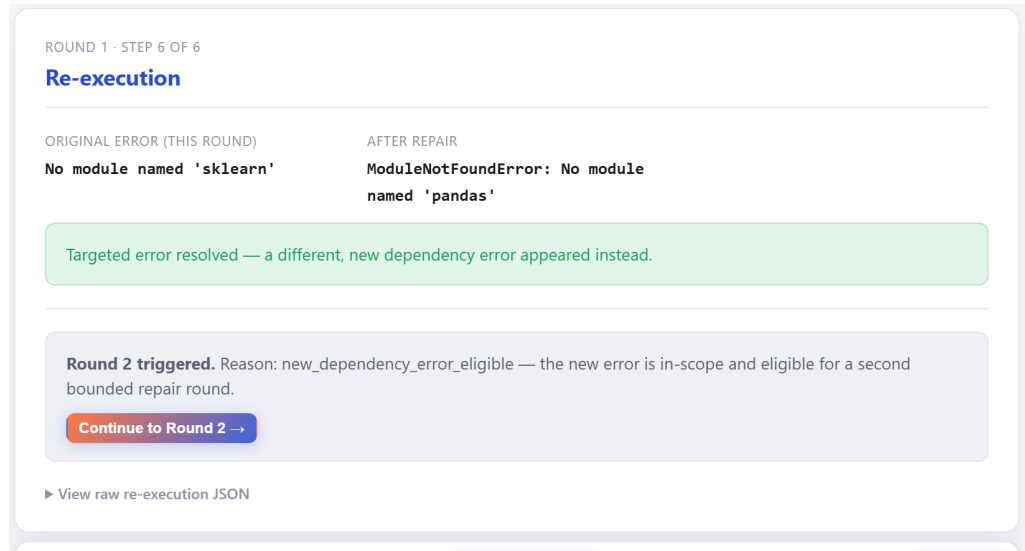


Figure 4.4: Re-execution stage for the same notebook, cropped to the centre panel. The targeted **sklearn** error is resolved and a different dependency error (**pandas**) surfaces in its place. The recorded Round-2 trigger decision that follows is shown with its reason.

Together, these components define the repair-layer architecture: classification and explanation establish what failed, RAGRepairAgent proposes a grounded repair, FixApplicator validates it through re-execution, and ResultLogger preserves the resulting evidence for evaluation and knowledge-graph integration. Chapter 5 describes how these contracts were implemented, while Chapter 6 evaluates the resulting system.

5 Methodology and Implementation

Chapter 4 fixed what each component is responsible for and what data it exchanges with its neighbours. This chapter explains how each component was built: the concrete rules and libraries used, and the reasons behind the decisions that shaped them.

The chapter follows the order data actually moves through the system. The working dataset is prepared first. Then come classification and context extraction, the explanation component, the PyPI-grounded repair component, and fix application. The orchestrator that drives these components in a single unattended pass, and that carries a newly exposed error into the bounded second round, is described next, because its behaviour is what the evaluation measures. Result logging and knowledge-graph enrichment come last, because they join the outputs of every earlier component into one persistent record.

5.1 Implementation Overview

This section gives one condensed view of how the stages connect, so that the detail in the rest of the chapter does not obscure how the pieces form a single system.

Docker-pipeline dependency-error dataset (214 rows)

```
-> dataset preparation      scripts/prepare_dependency_dataset.py
-> context extraction /
    classification          scripts/extract_error_contexts.py
-> LLMExplainer             scripts/run_llm_explainer.py
-> RAGRepairAgent           scripts/rag_repair_agent.py
-> FixApplicator            scripts/fix_applicator.py
-> new error: reclassify, explain,
    repair if eligible (round 2) scripts/run_pipeline.py
-> ResultLogger (SQLite)    scripts/result_logger.py
-> CSV export               scripts/export_repair_attempts_csv.py
-> RML mapping (Morph-KGC)  repair_attempts.rml.ttl
-> repair RDF linked to FAIR Jupyter KG notebook IRIs
```

Each arrow is a data hand-off: a plain file, a database, or a call between the components' own functions. Every stage can be invoked as its own script against the previous stage's output, which is how each was developed and tested. The same stages are also driven in sequence by a single orchestrating process (`scripts/run_pipeline.py`) within one pass, which is how the evaluation in Chapter 6 was run.

The sections below follow this order, one component at a time. Section 5.9 then traces one concrete record through every stage.

5.2 Dataset Preparation

Chapter 3 already gave the resulting numbers for this stage: 214 dependency-error rows, 200 usable and 14 excluded, split 13/187 between development and evaluation. This section explains how `scripts/prepare_dependency_dataset.py` produces those numbers, and why the rules behind them were chosen.

The script reads the Docker pipeline’s own `notebook_executions` table, and keeps every row where `error_category = 'DEPENDENCY_ERROR'`. For each kept row, it applies two independent decisions, deterministically and without any LLM involvement. The first is a *subtype*: is the module missing entirely, present but incompatible, a missing system library, or an ambiguous local name? The second is a *scope status*: is this row inside the pip-only v1 repair scope at all? Both decisions rely only on simple, inspectable rules over the `error_type` and `error_message` fields the Docker pipeline already recorded. For example, a message containing `".so"` or `"shared object file"` is classified as `system_library`. A failing module name drawn from a `short list of known-ambiguous names` (`utils`, `statistics`, `config`, `helpers`, `src`) is classified as `mapping_unknown`.

Doing this filtering with plain rules, before any language model is involved, was a deliberate choice. A decision that determines whether a row is even eligible for automated repair should be explainable and reproducible on its own terms. An LLM’s judgement about scope eligibility would be harder to audit, and could silently change between runs of the same model. Keeping this gate deterministic also means every later component can trust `scope_status` directly, instead of re-deriving eligibility itself.

The dataset is also split into three disjoint groups: 13 rows set aside for manually designing and sanity-checking the explanation and repair prompts (Sections 5.4 and 5.5), 187 usable rows reserved for a later, controlled evaluation run, and the 14 excluded rows. Separating a small development sample from the evaluation set matters for the same reason it matters in any machine-learning setting. A prompt tuned by looking at a specific row, and then tested again on that same row, would give an optimistic and misleading impression of how well the prompt generalises.

The script writes four output files: a CSV of the classified rows, a schema document describing every column, a statistics file with the counts used throughout this thesis, and a split-manifest file recording exactly which notebook execution IDs fall into which split, so the same split can be reproduced later rather than re-randomised.

5.3 Context Extraction and Error Classification

`scripts/extract_error_contexts.py` takes each dataset row prepared above and adds two things: a more detailed root-cause hint with a confidence level, and, where possible, the actual source code around the failure. Every field the dataset-preparation stage already computed (`scope_status`, `exclusion_reason`, `split`) is carried through unchanged. This stage enriches every row regardless of scope, so an excluded row still receives full context for its explanation.

5.3.1 Root-Cause Hints

The subtype produced by dataset preparation is refined into one of seven root-cause hints:

- `direct_missing_package` – the module is genuinely absent;
- `import_distribution_name_mismatch` – the import name differs from its PyPI distribution name, for example `sklearn` versus `scikit-learn`;
- `version_or_api_incompatibility` – a “cannot import name” style failure;
- `transitive_dependency` – a generic `ImportError` not matching a more specific pattern;
- `system_level_dependency` – a missing shared library;
- `local_import_or_path_issue` – an ambiguous local module name; and
- `insufficient_context` – not enough information to decide.

Each hint also carries a confidence level. A direct pattern match gives high confidence: a missing-module message, a “cannot import name” message, or a `.so` failure. Softer phrasing, such as an “install” or “required” hint, gives medium. An ambiguous local module, or a message carrying too little information to decide, gives low.

5.3.2 Recovering Source Context

Three sources of context are used, ordered by reliability. The first is a legacy execution table from an earlier pipeline run. Because this table was not produced by the same execution as the current dataset and has no timestamp, it is treated only as supporting evidence.

A legacy row is accepted only when its final reported error matches the current failure. Matching only the notebook or the failing module name is not enough. For example, a cell may import both `pandas` and `sklearn`, but fail on `pandas` in one execution and on `sklearn` in another. Treating those as the same failure would attach the wrong source context.

In practice, 162 of the 214 dataset rows (76%) have a matching notebook ID in the legacy table, but only about 40% of those records contain the correct failing module in the stored traceback. For this reason, legacy context is used only as a low-confidence hint and is never trusted on its own.

The second tier, used only when explicitly requested, fetches the notebook and its dependency files directly from GitHub. It uses the repository’s recorded commit where available, and falls back to the `main` or `master` branch otherwise. The third tier falls back to the metadata already present from dataset preparation (error type, message, cell index, failing module), when no source can be recovered at all. This is unavoidable here: the repositories cloned during the original Docker pipeline run were never kept on this machine, so no local copy of any notebook’s source exists unless it is fetched again.

5.3.3 Limitations

A few limitations follow directly from the above, and are worth stating plainly, because they affect how confidently later components can rely on this context. Remote fetching is not pinned to the exact commit that originally failed, unless that commit happens to still be resolvable. So a freshly fetched notebook can, in rare cases, differ slightly from the one that actually produced the recorded error. `error_cell_index` indexes into the notebook’s full cell list, so a notebook edited since the original run can shift cell positions and misalign the index. Any unexpected per-row failure (a malformed row, a database error) falls back to a bare metadata-only record, rather than aborting the whole batch – this keeps one bad row from blocking the other 213.

5.4 LLM Explanation Component

LLMExplainer (Section 4.3.2) turns a classified failure into a plain-language account of what went wrong. It is the component most directly tied to the thesis title, and its role is deliberately narrow: it explains a failure, it never proposes how to fix one.

5.4.1 Role and Separation from Repair

The prompt sent to the model instructs it to describe the failure in plain, non-technical language, using only the information supplied, and explicitly forbids it from suggesting a fix – deriving a repair is RAGRepairAgent’s job, kept structurally separate (Section 5.5). This separation serves two purposes. First, it keeps each component’s failure mode contained: an explanation that misjudges a detail is a readability problem, while a repair suggestion that misjudges a package name or version is executed inside a container, so the two carry very different risk and deserve very different validation. Second, it lets LLMExplainer’s scope be wider than RAGRepairAgent’s: the pip-only repair scope defined in Section 3.2.3 restricts

which rows may be *repaired*, not which rows may be *explained* (Section 3.2.4). A researcher whose notebook fails because of a missing system library still receives a plain-language explanation of the failure, even though the current repair layer cannot act on that particular case.

The same two principles carry over to the second repair round. When a repaired notebook fails again on a new error, that error is reclassified and explained as a record of its own before its repair eligibility is evaluated, so the reader receives an account of every error the pipeline encountered, not only the first (Section 5.7). The explainer is called with the same prompt, schema, and retry rule in both rounds; the round only changes which record it is given.

5.4.2 Model and Runtime Selection

The explanation and repair components share one open, locally-run model, rather than each using a different one. **Gemma-2 9B** was selected as the primary model because PLLM [2], the closest prior system, reports Gemma-2 9B with RAG as its own best-performing configuration for exactly this kind of dependency task. Using the same model for both explanation and repair keeps a later comparison between configurations clean: any difference in outcome can be attributed to the prompt or the retrieval design, not to a hidden difference in which model produced which part of the result. CodeLlama-13B and DeepSeek-Coder (7B and 13B) were kept as fallback candidates, motivated by Szalontai et al. [22] and a broader empirical comparison of code-specialised models [21]. Both report that smaller, code-specialised models can match or beat larger general-purpose ones on repair tasks. Neither fallback was needed: Gemma-2 9B performed adequately in the sanity checks run on the development split. A separate experiment later replaced Gemma with a much larger open model to test how much the choice mattered (Section 6.6.15).

Ollama was chosen as the serving runtime because it gives simple local access to a quantised model without setting up a GPU-serving stack, which is appropriate for a component running one notebook at a time rather than serving high-throughput traffic. A hardware check confirmed `gemma2:9b` runs on the available machine using 6.3 GB of memory, entirely on CPU, with a 4096-token context window – workable, though slow, which later became relevant to a timeout decision described in Section 5.5.

5.4.3 Structured Explanation Output

An explanation is read by a person, but it also has to be logged, validated, and processed by later stages. Free prose cannot be checked automatically for the fields those stages need. The explainer therefore returns a single JSON object, validated against a fixed schema (`schemas/explanation.schema.json`) with exactly six fields:

- `summary` – a short, plain-language statement of what failed;
- `root_cause` – why it failed, in the same plain language;

- **evidence** – a list of the concrete signals the explanation is grounded in (the error type, the error message, the root-cause hint);
- **failing_module** – the module named in the explanation;
- **explanation_confidence** – high, medium, or low; and
- **limitations** – what the explanation could not determine, for example because only metadata, not source code, was available.

The plain-language requirement is enforced inside the fields rather than abandoned: **summary** and **root_cause** are what a reader sees, and the prompt still constrains them to short, non-technical sentences. The surrounding structure exists for the pipeline, not for the reader. This follows the same pattern used for the repair prompt (Section 5.5).

The schema’s **explanation_confidence** field is named differently from the classifier’s own **confidence** field (Section 5.3) because the two measure different things and can legitimately disagree. A classifier can be highly confident it matched a known pattern while the model is only moderately confident it has explained the underlying cause correctly.

5.4.4 Prompt, Validation, and Retry

The prompt instructs the model to write in plain, non-technical language, to use only the information supplied, and never to suggest a fix. Using only supplied information means never inventing a version number, a file name, or a cause not visible in the error. The runner sends this prompt to **gemma2:9b** through Ollama (or, in the model-sensitivity run of Section 6.6.15, to Qwen through the same provider interface with the same prompt), parses the response as JSON, and validates it against the schema. **Each request has a 120-second timeout.** On invalid JSON, a schema violation, a timeout, or an unavailable model, the runner retries once with a corrective prompt. If the second attempt also fails, the failure itself is logged, with its category (**timeout**, **model_unavailable**, or a validation failure), rather than discarded, so no row silently disappears from the results file. The evaluation in Chapter 6 keeps these categories apart: a request that timed out is a runtime failure of the model service, not a malformed model output, and the two are reported separately.

Each logged result keeps three things apart: the input metadata from the classification stage, the model and prompt configuration used, and the generated explanation with its validation status, latency, and retry count. Separating them means a later reader cannot confuse the original error text with text the model wrote about it. It also lets the result-logging stage (Section 5.8) join an explanation back to its originating notebook by **notebook_execution_id** alone.

5.4.5 A Real Explanation Record

The following committed record illustrates the schema above on a real, already-classified failure: `notebook_execution_id` 8, a `ModuleNotFoundError` for `sklearn`.

summary: "The notebook execution failed because Python could not find the module 'sklearn'."

root_cause: "The notebook imports 'sklearn', which is not available in the execution environment. The classifier also marks this as an import/distribution-name mismatch, meaning the import name may not directly match the package distribution name."

evidence: ["The error type is `ModuleNotFoundError`.", "The error message says: No module named 'sklearn'.", "The `root_cause_hint` is `import_distribution_name_mismatch`."]

failing_module: "sklearn"

explanation_confidence: "high"

limitations: "Only metadata is available, so the exact failing cell and dependency files cannot be inspected."

This record was generated with the `few_shot` prompt strategy. It succeeded on the first attempt and required `98,029 ms` on the CPU-only hardware described above. The example is included only to illustrate the structure of a real explanation record; it `should not be interpreted as evidence of explanation quality` across the full dataset. The few-shot prompt contains two fixed examples in `prompts/dependency_explanation_v1.txt`: this `sklearn/ModuleNotFoundError` case and the `scipy/cumtrapz` case discussed in Section 5.9. Both examples come from the 13-row development split and remain unchanged for every LLM call. No example from the reserved evaluation split is included in the prompt.

5.5 PyPI-Grounded Repair Generation

RAGRepairAgent (Section 4.3.3) does not install anything. Its only job is to answer one question – *what repair should be attempted?* – and to answer it safely: never inventing a package name, never inventing a version number, and never handing a later component a raw string to execute in a shell. This section explains how that is achieved.

5.5.1 Overall Flow

classified dependency failure (usable)

- > identify the failing import
- > resolve the import name to a PyPI distribution name
- > query the PyPI registry for that distribution
- > filter the returned versions to a small, safe candidate set

- > for a version-incompatibility failure, intersect with curated compatibility evidence
- > give the LLM only this grounded candidate set
- > validate the LLM's proposal against the same evidence
- > if valid, deterministically build the exact install command

The design principle behind this flow is that the language model is never the only thing standing between an error message and a command that will later run inside a container. Four separate layers each have exactly one job, and each layer only trusts what the previous layer has already verified.

5.5.2 Layer 1: Deterministic Retrieval

Before any network request, the failing Python import name must be resolved to an installable PyPI distribution name. The two are frequently different: `sklearn` installs as `scikit-learn`, `cv2` as `opencv-python`, `skimage` as `scikit-image`, `umap` as `umap-learn`, `pkg_resources` as `setuptools`, and `Bio` as `biopython`. A small, explicit, hand-curated mapping table (`config/package_mapping.yaml`) holds every case the retriever is allowed to resolve. An import name absent from this table is never assumed to also be its own package name. Instead, the retriever reports a `mapping_unknown` status, without making any PyPI request at all. This rule matters concretely for this thesis's own dataset: `sklearn` and `pkg_resources` are, respectively, the second and first most common failing modules (Table 3.3). Getting this mapping right or wrong affects a large share of the dataset, not an edge case.

Once resolved, the distribution's release metadata is retrieved from the official PyPI JSON Simple API (`GET /simple/{distribution}/`), normalising the name first according to PEP 503. This endpoint was chosen deliberately over the older project JSON endpoint's `releases` field, which PyPI's own documentation marks as deprecated for new integrations; the newer endpoint is the one PyPI recommends going forward. The returned files are grouped by release version, and a safe candidate set is computed by dropping unparseable files, pre-releases, and releases where every file is yanked, and by checking each release's declared `requires-python` against the runtime the notebook actually executed under.

That runtime version is a fixed constant, `"3.10"` (`config/rag_repair.yaml`), not looked up per notebook. This is not an approximation of convenience: the Docker pipeline builds every repository's container from the same template, with the base image hardcoded to `python:3.10-slim`, so every notebook this component processes genuinely ran under the same interpreter version. The notebook's own recorded kernel metadata is deliberately not used as a substitute, because it reflects what the notebook's original author had installed, which is frequently stale or missing, not what the Docker container the notebook actually failed in provides.

The final candidate list is capped to five versions, newest first, to keep the amount of version metadata placed in the prompt bounded and readable.

5.5.3 Why PyPI Metadata Is Not Enough

For a `wrong_version` failure – a “cannot import name” error – knowing that a release exists and supports the right Python version is not enough. PyPI’s own Simple API specification (PEP 691) only exposes, per file, its filename, URL, hashes, required Python version, size, upload time, and yanked status. It says nothing about which functions or classes a given release actually provides. Proving that, for example, `scipy.integrate.cumtrapz` still exists in a specific SciPy release requires consulting that project’s own documentation, not PyPI.

This gap is closed with a small, hand-curated compatibility registry. Each entry records one import symbol, the version range in which an official source confirms that symbol still exists, and a link to that source. Table 5.1 lists the three patterns curated for this dataset, all covering symbols that were removed in a major SciPy or NumPy release.

Symbol	Compatible with	Official source
<code>scipy.integrate.cumtrapz</code>	<1.14.0	SciPy 1.14.0 release notes, “Expired deprecations”
<code>scipy.sparse.sputils.isshape</code>	<1.14.0	SciPy 1.14.0 release notes, cross-checked against SciPy’s source repository
<code>numpy.VisibleDeprecationWarning</code>	<2.0.0	NumPy 2.0.0 release notes, “NumPy 2.0 Python API removals”

Table 5.1: Curated API-compatibility evidence used for `wrong_version` repairs.

The registry is intentionally small and manually curated. It contains verified compatibility rules only for the specific wrong-version API failures observed in this thesis dataset. Each rule was checked against an official source. If a wrong-version failure is not covered by the registry, the system does not guess a compatibility rule and therefore abstains from proposing a version-based repair. The registry is therefore dataset-specific rather than a general-purpose compatibility database. Its purpose is to provide reliable evidence for the wrong-version patterns present in the evaluated corpus.

Layer 2: What the Model Decides

After Layer 1 has resolved the package and retrieved the available PyPI evidence, the repair model chooses a repair from that grounded information. It receives the classified failure, including the error type and message, failing module, subtype, and root-cause hint. Where available, the failing cell source is also included. Layer 1 additionally provides the resolved distribution name, the Python target, the compat-

ible candidate releases, and, for wrong-version failures, the applicable compatibility constraint.

The model does not receive the raw PyPI response. Instead, it sees only the candidate set already filtered by the deterministic retrieval layer. It also does not receive the LLMExplainer output or repository dependency files such as `requirements.txt`; the latter are used later when FixApplicator reconstructs the execution environment.

The model returns a structured repair proposal with three possible actions: `install`, `pin.version`, or `none`. For `install`, it identifies the distribution to install. For `pin.version`, it additionally selects a version from the supplied candidate set. The `none` action represents abstention when the available evidence does not support a repair.

5.5.4 Layers 3 and 4: Validation and Command Construction

Before a repair proposal is used, it first passes schema validation and is then checked against the evidence retrieved in Layer 1. **Five deterministic grounding checks** are applied:

- `action` must be one of the three allowed values: `install`, `pin.version`, or `none`.
- A proposed `install.name` must exactly match the distribution resolved by the retriever.
- A proposed `version` must appear in the supplied candidate list.
- An `install` action for a `missing.package` additionally requires at least one Python-compatible candidate.
- A `pin.version` action for a `wrong.version` additionally requires verified compatibility evidence.

The proposal is also checked for unsafe string values, such as whitespace, leading dashes, or shell metacharacters. If any validation check fails, the proposal is rejected and the action is set to `none`. The system does not replace an invalid package or version with a guessed alternative.

Only validated proposals are converted into executable commands. The command is constructed deterministically by the pipeline rather than generated by the LLM. For example, an installation produces `python -m pip install <package>`, while a version pin uses `<package>==<version>`. This ensures that unrestricted model-generated text is never executed directly.

If the model response cannot be parsed, fails schema or grounding validation, times out, or the model is unavailable, the component retries once. The reason for the failure is included in the retry prompt. If the second attempt also fails, the component returns `action: none`, and no repair is applied.

5.5.5 Why Repository Dependency Files Are Not Used as Repair Evidence

Repository dependency files such as `requirements.txt` are useful for reconstructing an environment, but they are not reliable enough to serve as direct repair evidence. A declared dependency file does not necessarily describe the exact package state in which the recorded failure occurred. It may be incomplete, outdated, unpinned, or may itself contain the version that caused the incompatibility.

The files are also inconsistent across repositories. Some projects use `requirements.txt`, while others use `setup.py`, `pyproject.toml`, multiple requirement files, or no explicit dependency file at all. Making repair generation depend on one of these files would therefore provide different amounts and qualities of evidence for different notebooks.

For these reasons, the repair decision is based on evidence that can be checked independently of the repository’s declared environment: the observed failure, the resolved PyPI distribution, available releases, and compatibility evidence where applicable. Repository dependency files are used later by `FixApplicator` when rebuilding the baseline environment in which the proposed repair is tested.

This separation gives the two kinds of information different roles. Dependency files describe how the repository expects its environment to be constructed, whereas the repair evidence is used to determine how the observed dependency failure should be addressed.

5.5.6 Runtime Configuration

RAGRepairAgent shares its model with the explanation component: `gemma2:9b` through Ollama, temperature 0.1, top-p 0.9, and at most 700 generated tokens. The request timeout is 300 seconds. This value was set from measurement. In a pilot run on the CPU-only hardware described above, the same repair request timed out twice under an initial 120-second limit and then succeeded at the first attempt under a 300-second limit, on the same machine with the same model (Section 6.2). The model was healthy throughout; 120 seconds was simply too short for CPU-only generation.

The retrieval client enforces a 0.5-second wait between PyPI requests and at most one retry on an HTTP 429 response, so a batch run over many rows cannot overload the public registry.

5.5.7 The Implemented Result Shape

Each persisted result record carries three groups of fields. `notebook_execution_id` is its only join key back to the rest of the dataset. An `eligibility` block records whether the row was usable, excluded, or invalid. A nested `retrieval_result` block holds everything Layer 1 found. The validated outcome is stored as `final_action`,

`final_install_name`, and `final_version`, together with the constructed argument list and its human-readable `command` string.

An overall `status` field summarises the attempt with one of three values. "success" marks a grounded proposal. "abstained" marks a legitimate none, whether the model chose it or a failed check forced it. "failed" means every attempt was exhausted without a usable response. The evaluation in Chapter 6 counts abstentions from this three-way distinction.

The fix object stores only the repair information and the `notebook_execution_id` used as a join key. Repository and notebook metadata are not duplicated in this object. Before applying a repair, FixApplicator uses the join key to recover the corresponding repository and notebook information from the original dataset (Section 5.6).

5.6 Fix Application and Notebook Re-execution

FixApplicator answers the question RAGRepairAgent leaves open: *does the proposed repair actually work?* It contains no language-model call. It is a small, deterministic pipeline that applies an already-validated proposal, re-runs the notebook, and classifies what happened.

```
validated RAGRepairAgent proposal
-> re-join with the classification dataset
    by notebook_execution_id
-> re-validate install_name/version and rebuild the command
-> rebuild a Docker environment from the original recipe
-> apply the fix inside that environment
-> re-run the notebook top to bottom
-> classify the outcome
```

5.6.1 Recovering Notebook Identity

A RAGRepairAgent result is joined to the rest of the dataset by `notebook_execution_id` alone; it carries no repository ID, notebook name, or repository URL. FixApplicator therefore first looks up each record's `notebook_execution_id` in the classification dataset (`data/context-classification/dependency_error_contexts.jsonl`) to recover this identity. If the ID is not found, or the matching record lacks one of these fields, FixApplicator records the attempt as an infrastructure failure (`apply_error`, failure stage "join"). Filling in a missing field with a default value could silently clone or run the wrong repository, so no guess is made.

5.6.2 Re-validating the Persisted Command

FixApplicator does not execute the persisted `argv` as it is, even though RAGRepairAgent already validated the proposal. It re-checks `install_name` and `version` with the same safety check RAGRepairAgent uses, and rebuilds the argument list from scratch with the same function. If the rebuilt list differs from the persisted one, the record is rejected before anything runs. This second check enforces the same safety rule independently. When the two components agree, the proposal has not been altered between the time it was written and the time FixApplicator reads it.

5.6.3 Rebuilding the Docker Environment

The upstream Docker pipeline does not preserve its per-repository containers, so there is no original container to reuse. FixApplicator therefore rebuilds the environment for every repair attempt from the same recipe, as Chapter 4 specifies: the same base image (`python:3.10-slim`), the same dependency-installation procedure, and the same notebook execution command (`jupyter nbconvert --to notebook --execute --allow-errors`) that the reference pipeline uses.

The installation procedure is where the repository’s own dependency files enter the pipeline. A `requirements.txt` at the repository root, when present, is installed entry by entry; an entry that fails to install is skipped rather than aborting the build. Afterwards, each `setup.py` whose location the upstream repository metadata records is installed from its directory. These files define the baseline environment in which the fix is tested; they are not evidence for the fix itself (Section 5.5.5). The `setup.py` locations come from the upstream metadata rather than from scanning the clone. In the final evaluation runs, that metadata lookup returned no data (Section 6.6.2), so the baseline there consisted of the root-level `requirements.txt` where the repository had one.

The execution command is reused intentionally instead of writing a new execution loop: it is the exact mechanism that produced the original dataset, so reusing it keeps the comparison fair. The rebuilt environment is meant to keep environmental differences small, not to be an exact copy. The repository is cloned again, and packages the repository does not pin resolve to whatever PyPI serves at run time. Only the one notebook that failed is executed, not every notebook in the repository, and it is always executed from the beginning. A notebook keeps state across cells, so a partial re-run from the failing cell cannot show that a fix works.

Two implementation details differ from the reference Docker pipeline. First, FixApplicator keeps the Docker build context separate from the cloned repository. This prevents the repository’s full Git history from being sent to the Docker daemon during the build. The repository is still mounted into the container when the notebook is executed, so this does not change the notebook source available at run time. Second, the temporary repair container runs as the default root user instead of being mapped to the host user’s ID. This keeps the execution setup simple and allows the already validated installation command to be executed without modification. The

container is disposable and is used only for validating the repair.

5.6.4 Commit Pinning

Where the upstream pipeline’s own database records a commit for a repository, FixApplicator checks out that exact commit before building anything, rather than staying on the repository’s current default branch. A public GitHub repository can change after the original dataset was collected; without pinning, a later fix attempt could be tested against a different version of the notebook than the one that produced the original failure. The reference pipeline does not pin the commit itself: it clones only the latest state of the default branch, even though it records a commit. Here the implementation goes further than the system it builds on. If the recorded commit cannot be checked out, FixApplicator reports a specific infrastructure failure rather than falling back silently to the current branch.

The commit lookup is optional. If the metadata database is unavailable, FixApplicator continues without commit pinning and records that the checkout was skipped. In that case, the repository is cloned from its current default branch. This fallback was used in the final evaluation runs because the metadata database configured for FixApplicator was not accessible from the WSL environment. As a result, the evaluated repositories were not pinned to their original recorded commits. Section 6.6.2 discusses this limitation and its effect on reproducibility.

5.6.5 Fix Application Safety

The install command is executed only as an explicit list of arguments passed directly to a subprocess, never as a shell string. Every external process this component invokes, git and Docker included, goes through an injectable runner as an argument list. A static check of the source code confirms that `shell=True`, `os.system`, `eval`, and `exec` appear nowhere in it. If the fix installation fails inside the container, the container script stops immediately and the notebook is not executed, because running an unfixed notebook would validate nothing.

5.6.6 Outcome Classification

Exactly three outcomes are possible, listed in Table 5.2. They match the contract fixed in Chapter 4; no fourth value is introduced.

A `still_failing` result additionally records whether the new error is the same as the original one. The check is conservative: the error is marked as the same only when the exception type matches *and* the original failing module name still appears in the new message. The check is designed to under-claim sameness rather than over-claim it, so a fix that resolved the original problem and uncovered a different one is not reported as having done nothing. This distinction matters a great deal in practice, as Chapter 6 shows.

Outcome	Meaning
<code>fixed</code>	The re-executed notebook produces no error output anywhere.
<code>still_failing</code>	The fix installed and the notebook re-ran, but an error remains – possibly the same error as before, possibly a different one further into the notebook.
<code>apply_error</code>	The repair could not be meaningfully tested: a dataset join failure, an invalid repair-proposal record, a clone or checkout failure, a Docker build failure, a failed fix installation, a timeout, or a missing output notebook.

Table 5.2: FixApplicator outcome values.

A row whose proposal was `"none"`, or whose upstream status was not `"success"`, is not treated as an attempt. It is recorded with a separate `status: "skipped"` field rather than a fourth outcome value. This keeps the three-value contract intact while still separating “nothing was attempted” from “an attempt was made and failed”.

5.6.7 Why a Real Pilot Was Necessary

The FixApplicator test suite passed before the component was tested with real Docker executions. However, the first real pilot revealed three environment-specific problems. Docker output was decoded incorrectly, the generated entry-point script had incompatible line endings, and cleanup failed for some read-only files created by `git clone` on Windows. These problems were fixed, and regression tests were added for each of them.

This showed why real execution tests were necessary. Mocked tests can verify the internal logic of FixApplicator, but they do not reproduce all behaviour of Docker, Git, the operating system, and subprocess execution. Therefore, the pilots in Chapter 6 use real executions in addition to the automated test suite.

5.6.8 Environment Isolation and What Is Out of Scope

Every attempt uses a freshly named work directory and container. Both are removed afterwards, whether the attempt succeeded, failed, or timed out. Docker may reuse a built image across attempts through its own build cache; this is expected and does not break isolation. The guarantee that matters is a fresh clone and a fresh container per attempt, not a fresh image layer every time.

FixApplicator returns one self-contained result per attempt. It has no orchestrator and no two-round loop, and it does not persist its results to a database or knowledge graph. ResultLogger persists the results (Section 5.8), and the orchestrator,

described next, carries a newly exposed error into the bounded second round.

5.7 Orchestration and the Bounded Second Round

The orchestrator, `scripts/run_pipeline.py`, drives the components above over a dataset split in one unattended pass, calling each through its own interface. Its control flow was fixed in Section 4.4. This section describes how that flow is realised, in particular how a newly exposed error is handled, because the evaluation in Chapter 6 measures exactly this behaviour.

5.7.1 One Pass per Notebook

For each record of the split, the orchestrator classifies the record and calls LLM-Explainer on it. If the record is repair-eligible, it calls RAGRepairAgent, and if the agent produces a grounded proposal, it calls FixApplicator. Every call for one record shares one run identifier. Round 1 is the same for every record. A second round is considered only when three conditions hold: Round 1 reached FixApplicator, the notebook came back `still_failing`, and the conservative same-error check (Section 5.6) reports a genuinely different error. A notebook that still fails on its original error, or whose repair could not be tested, receives no second round, because repeating the same repair would test nothing new.

5.7.2 Reclassifying and Explaining a Newly Exposed Error

A genuinely new error is turned into a record of its own. The orchestrator runs the same two classification stages that every original row went through (Sections 5.2 and 5.3) on the new error type and message. The result is a *round-2 record* with its own failing module, subtype, scope status, and root-cause hint. The record is explained first. A helper, `explain_round_record()`, calls the same explainer on the round-2 record with the same prompt template, schema, and one-retry rule as Round 1, and tags the resulting explanation with its round. Two properties of this step matter for the evaluation.

First, the explanation is produced for every newly exposed error, whatever its scope. A new `missing_package` error is explained and then repaired. A new `system_library` error, a new `mapping_unknown` error, or a non-dependency error is explained, and the record then stops there because it is not repair-eligible. Explanation scope therefore stays wider than repair scope in Round 2, exactly as it does in Round 1.

Second, the explanation is independent of the repair. RAGRepairAgent receives the classified round-2 record and the PyPI evidence retrieved for it, never the explanation text. A failed explanation call does not block repair either. If the explainer times out, the model is unavailable, or the call raises unexpectedly, the helper records a failed explanation with its failure category and returns normally. An otherwise

eligible record then proceeds to RAGRepairAgent and FixApplicator as usual, and the evaluation counts it as a repair round with a failed explanation.

Repair eligibility is evaluated only after the explanation is recorded, with the same rules as in Round 1. An eligible round-2 record is passed to RAGRepairAgent as a target in its own right, with its own retrieval, proposal, and validation. Two rounds is a hard cap (`MAX_ROUNDS_HARD_CAP = 2`). An error exposed by the Round-2 re-execution is recorded in the Round-2 outcome but is not reclassified, explained, or repaired. No record therefore carries more than two explanations or two repair rounds.

5.7.3 Why Round 2 Reapplies Round 1's Fix

The isolation guarantee of Section 5.6 has a direct consequence for the second round: no container survives between Round 1 and Round 2 of the same notebook. If Round 2 rebuilt a fresh container and applied only its own fix, the notebook would be tested against *original environment + Round 2*, without Round 1's already-validated fix. A `fixed` or `still_failing` verdict from that container would show only whether the second repair alone was enough. The question Round 2 is meant to answer is whether both repairs together make the notebook run.

The orchestrator therefore re-runs Round 1's already-validated install command inside Round 2's freshly rebuilt container, immediately before it applies Round 2's own fix. The notebook is re-executed against *original environment + Round 1 + Round 2*. Every Round-2 outcome then reflects what it claims to validate, and no container has to persist between rounds: the fresh-container-per-attempt guarantee still holds for every Round-2 attempt, and only the fixes applied inside that fresh container change.

5.7.4 What Is Recorded

The orchestrator writes two things. The first is a per-notebook JSONL trace: one line per notebook (187 in the evaluation split), holding the classification, the Round-1 explanation, the second-round decision, and one entry per repair round in which RAGRepairAgent was invoked. A reclassified new error's explanation is stored with the decision that produced it, so it is present whether or not a second round followed. The top-level explanation of a trace line is always the original failure's, so a trace from a single-round run has the same fields in the same place.

The second is the `repair_attempts` table, written through ResultLogger (Section 5.8) with one row per repair-decision entry: normally a RAGRepairAgent invocation, and for an original out-of-scope record a synthesised abstention entry written without an agent call (Section 5.8.1). An abstention is a row too, with `action = none` and no outcome. A Round-1 row carries the explanation, model, and prompt strategy of the original failure; a Round-2 row carries those of the newly exposed error it targeted. A newly exposed error that was explained but was not repair-eligible never reaches RAGRepairAgent and therefore has no Round-2 row: its explanation

exists in the trace only. In the final evaluation the table holds 235 rows per run, 187 for Round 1 and 48 for Round 2, of which 73 carry an executed fix application. The table’s schema is unchanged by any of this: the same eighteen columns hold either round’s values.

5.8 Result Logging and Knowledge-Graph Enrichment

ResultLogger (Section 4.3.5) classifies, explains, proposes, and applies nothing. It joins the JSONL output files of the preceding components, one file per component, into one persistent `repair_attempts` row per repair-decision entry, and makes that row exportable for the knowledge-graph mapping. Three files implement it. `scripts/result_logger.py` covers the SQLite side. `scripts/export_repair_attempts_csv.py` and `mapping/rml_mapping/repair_attempts.rml.ttl` cover the knowledge-graph side.

5.8.1 Joining Four Outputs

RAGRepairAgent’s repair-proposal output is the driving input: ResultLogger writes exactly one row per entry in it, in order. An invocation that abstains is an entry too, so the number of entries is the number of repair decisions taken, not the number of classified or explained errors. Repair eligibility is checked before RAGRepairAgent is called (Section 5.7), and the two rounds then differ in what an ineligible record produces. For an original record outside repair scope, the orchestrator synthesises an entry in the same abstained result shape without any model call, reusing the agent’s own eligibility check and result skeleton (`build_excluded_repair_stub` in `scripts/run_pipeline.py`), so that every original record still contributes one row. A newly exposed error is instead reclassified and explained, and reaches RAGRepairAgent only if it is itself repair-eligible; a non-repairable new error stops there, its explanation existing in the trace only and producing no row. In the final evaluation every one of the 187 original records was repair-eligible, so no synthesised entry occurs in it: 50 newly exposed errors were explained, 48 became Round-2 invocations, and all $187 + 48 = 235$ entries are genuine invocations, giving 235 rows.

The other three inputs are joined to that driving record in three different ways, because the data does not offer one uniform key. The classification dataset is joined by `notebook_execution_id`; a miss is a hard error, as it is for FixApplicator’s own re-join (Section 5.6). The explanation output is also joined by `notebook_execution_id`. A coherent run produces at most one explanation per notebook, and the loader refuses to continue if it finds two. The fix-application output is joined by *position*: FixApplicator stamps each result with the line index of the proposal it processed, and ResultLogger uses that index. Position is needed because `notebook_execution_id` can legitimately repeat across proposal lines. The pilot output of Section 6.2 contains a timed-out attempt at notebook 8 followed by a successful re-run of the same notebook. Joining by id would not tell the two

outcomes apart; joining by position does, and the two attempts become two rows, as Section 4.5.2 requires.

5.8.2 Building One Row

`build_repair_attempt_row()` fills the eighteen columns fixed in Section 4.5.2 from these four inputs. `failing_module` and `subtype` come from the classification dataset. `explanation`, `llm_model`, and `prompt_strategy` come from the explanation output; under the orchestrator, a Round-2 row takes them from the explanation of the newly exposed error (Section 5.7.4). The `fix` and `outcome` fields come from the fix-application record when one exists, because that record reflects what was actually applied and observed. For an abstained proposal, the `fix` fields fall back to the proposal’s own values, and `outcome` and the new-error fields stay `NULL`. `rationale` always comes from the proposal, because `FixApplicator` makes no repair decision of its own.

`run_id` and `created_at` identify the run that produced the row. When the orchestrator drives every stage in one pass, as in the final evaluation, it threads one shared `run_id` through every call, so every row of a run carries the same identifier (Section 4.4). Only when a component is run on its own does a row without an outcome fall back to the repair agent’s run identifier.

5.8.3 Preserving Structured Evidence

Two of the eighteen columns hold a serialized JSON document rather than a single reduced value. The `explanation` column holds the complete explanation object: its `summary`, `root_cause`, evidence list, `explanation_confidence`, and `limitations`, exactly as validated against `schemas/explanation.schema.json` (Section 5.4.3). It is not reduced to the `summary` field alone. The `pypi_evidence` column holds the complete `retrieval_result` object produced by `RAGRepairAgent`: the resolved distribution name, every candidate version considered, the compatibility evidence and its source citation where one was used, and any warning raised during retrieval. This is more than the three-field illustration shown in Section 4.5.1. The retrieval object is serialized whenever the retrieval record exists, which includes a retrieval-stage abstention: a `mapping_unknown` row still carries a retrieval object whose status and warnings record why package resolution failed, so every one of the 235 rows of the final evaluation holds `pypi_evidence`. A column is `NULL` only when the corresponding upstream field has no value at all, such as the `explanation` of a record whose explanation call produced no response. Neither column ever holds an empty string or an empty JSON object.

This preserves detail that a later evaluation might need, without going back to the original explanation and repair-proposal files, which the batch that produced them may not have kept. A candidate version that PyPI offered but the model did not choose, or a compatibility source the proposal relied on, remains readable from the row that used it, as evidence and as provenance.

5.8.4 From SQLite to CSV

SQLite remains the store of record for evaluation and provenance. The exporter, `scripts/export_repair_attempts_csv.py`, adds a CSV export because the existing FAIR Jupyter knowledge-graph mappings (`mapping/rml_mapping/*.rml.ttl`, including `executions.rml.ttl`) all read a CSV file through Morph-KGC’s `ql:CSV` logical source, never a live database connection. The CSV is a bridge to that existing tooling, not a replacement for the table it is exported from.

`repair_attempts` stores only `notebook_execution_id` (Section 4.5.2), but the knowledge graph identifies a notebook by `notebook_id`. The exporter resolves `notebook_id` for each row by re-joining `notebook_execution_id` against the same classification dataset ResultLogger requires, using the loader FixApplicator already uses for this kind of recovery (Section 5.6). A `notebook_execution_id` missing from that dataset raises an error naming the offending `repair_attempts.id`, rather than exporting a row with no notebook link. By construction this case should never arise, because `log_repair_attempts()` already required every id it logged to resolve against the same dataset. The exported columns are `id` and `notebook_id`, followed by all eighteen `repair_attempts` columns unchanged. `notebook_execution_id` is carried into the CSV for traceability even though the mapping below does not reference it.

5.8.5 The RML Mapping

`mapping/rml_mapping/repair_attempts.rml.ttl` is modelled directly on the existing `executions.rml.ttl`, with the same prefix block and the same `rr:TriplesMap/rr:PredicateObjectMap` structure. It defines two triples maps over the exported CSV.

The first map gives each repair attempt an IRI, `https://w3id.org/reproducible/repairattempt_{id}`, typed both `repr:RepairAttempt` and `prov:Activity`. This follows the vocabulary strategy fixed at design time (Section 4.5.3): reuse an existing term where one fits, and add a new `repr:` term otherwise. The failing module, the subtype, the explanation, and every field of the proposed fix and its outcome are attached to this IRI as literal properties, each with its own new `repr:` predicate:

<code>failing_module</code>	<code>repr:failingModule</code>	<code>pypi_evidence</code>	<code>repr:pypiEvidence</code>
<code>subtype</code>	<code>repr:subtype</code>	<code>outcome</code>	<code>repr:fixOutcome</code>
<code>explanation</code>	<code>repr:explanation</code>	<code>llm_model</code>	<code>repr:llmModel</code>
<code>action</code>	<code>repr:fixAction</code>	<code>prompt_strategy</code>	<code>repr:promptStrategy</code>
<code>install_name</code>	<code>repr:installName</code>	<code>round</code>	<code>repr:round</code>
<code>version</code>	<code>repr:fixVersion</code>	<code>run_id</code>	<code>repr:runId</code>
<code>command</code>	<code>repr:fixCommand</code>		
<code>rationale</code>	<code>repr:fixRationale</code>		

Two further fields reuse predicates that the existing executions mapping defines for a cell execution’s own exception and message. The new error type maps to `repr:exception` and the new error message to `repr:msg`. These describe the *post-repair* error, not the original one. The original failure’s exception and message triples already exist in the graph, written by the executions mapping, so the same kind of fact is reused for a second observation. Finally, `created_at` maps to `prov:endedAtTime`, explicitly typed `xsd:dateTime`. This is the one place where the mapping departs from the untyped-literal style of the rest of the graph, because the PROV-O specification gives `prov:endedAtTime` an `xsd:dateTime` range and the `created_at` values are already ISO-8601 strings.

A column that is NULL in `repair_attempts`, such as an abstained attempt’s `install_name` or the `outcome` of a row that has none, becomes an empty field in the exported CSV. Morph-KGC then generates no triple for the corresponding predicate on that row, rather than an empty-string literal. Missing evidence is therefore absent from the graph rather than misrepresented as an observed empty value.

The second triples map links the notebook to the repair attempt. Its subject template, `https://w3id.org/reproduceme/notebook_{notebook_id}`, has exactly the same shape as the subject template of `notebooks.rml.ttl`, so the two resolve to the same node once loaded into the same graph. No join and no edit to the existing mapping is required. The map emits one triple, `repr:hadRepairAttempt repr:repairattempt_{id}`, which attaches the repair history to a notebook resource the graph already contains.

This export-and-map chain is the whole of the knowledge-graph side: ResultLogger writes SQLite, the exporter writes CSV, and Morph-KGC turns the CSV into RDF through this one mapping file. Section 6.4 reports the mapping’s pilot validation and its subsequent execution, unchanged, over the complete 235-row table of the final evaluation run.

5.8.6 Why No Crosswalk Table Is Needed

The link above only works if the Docker pipeline’s `notebook_id` identifies the same notebook as the `notebook_id` of the existing FAIR Jupyter knowledge graph. That graph was generated from the earlier, conda-based pipeline (Section 3.2.1), not from the Docker pipeline this thesis builds on. Whether the two pipelines’ identifiers coincide was an open question when the mapping was designed, and it was settled by checking; Section 6.4 reports the check and its result. The two pipelines share the same `notebook/repository` primary keys for the dataset this thesis operates on. The mapping can therefore link a repair attempt directly to its existing notebook resource via `notebook_id`, without a separate crosswalk table between the Docker pipeline and the knowledge graph.

5.9 A Worked Example: One Record Through the Pipeline

The preceding sections describe each component on its own terms. This section traces one real record through every stage, to make the data flow concrete. Every value shown comes from the recorded classification, explanation, repair-proposal, and fix-application outputs of the pilot runs reported in Sections 6.2, 6.3 and 6.5. The record is `notebook_execution_id` 174, the `scipy/cumtrapz` case already introduced in Table 5.1.

1. Classification. The notebook `examples/notebooks/MoltenSalt.Comparison.ipynb` in `zincware/MDSuite` fails with `ImportError: cannot import name 'cumtrapz' from 'scipy.integrate'`. Dataset preparation and context extraction record the failing module as `scipy`, the refined subtype as `wrong_version`, the scope status as `usable`, the root-cause hint as `version_or_api_incompatibility`, and the classifier’s confidence as high.

2. Explanation. `LLMExplainer`, run on this record by the orchestrator pilot of Section 6.5, returned a schema-valid explanation on its first attempt: *summary* "The notebook failed because it tried to import 'cumtrapz' from 'scipy.integrate', but that name was not available."; *root cause* "This is likely caused by a package API or version incompatibility, where the notebook expects a function that is not present in the installed SciPy package version."; *failing module* `scipy`; *confidence* `medium`, because the installed SciPy version could not be confirmed from the error alone. Section 5.4.5 shows the full six-field record for a different pilot notebook. Nothing in this explanation is passed on to the repair step that follows.

3. Retrieval. `RAGRepairAgent` resolves `scipy` to itself (no name mapping needed) and retrieves five Python-3.10-compatible candidates from the real PyPI Simple API: 1.13.1, 1.13.0, 1.12.0, 1.11.4, and 1.11.3. Because the subtype is `wrong_version`, the compatibility-evidence registry (Table 5.1) is also consulted and resolves `compatible_specifier`: "`<1.14.0`", citing the SciPy 1.14.0 release notes' "Expired deprecations" entry for `cumtrapz`.

4. Repair proposal. Intersecting the candidate list with the compatibility constraint, the model proposes action: "`pin_version`", `install_name`: "`scipy`", `version`: "`1.13.1`", with rationale "cumtrapz was removed in SciPy 1.14.0; 1.13.1 is the newest candidate version below that boundary, confirmed Python-3.10-compatible." Both schema and grounding validation pass on the first attempt, and the command is built deterministically as `python -m pip install scipy==1.13.1`.

5. Fix application. FixApplicator re-joins the record to recover its repository and commit (eadda45f96874bd6d7eacba89c57daba76db1c7d), rebuilds the Docker environment, installs a real `requirements.txt` and one `setup.py` package, then applies the pin. Installation succeeds, and the notebook is re-executed top to bottom.

6. Observed result. The original `cumtrapz` import error is gone, and execution proceeds further into the notebook. This is direct evidence that the version pin resolved the targeted incompatibility. The notebook then fails again on an unrelated `ModuleNotFoundError: No module named 'tf_keras'`. The conservative `same_as_original_error` check marks this as `false`: it is a different problem, not evidence that the fix did not work. This is the pattern Section 6.3.1 discusses for the whole pilot: repairing one dependency can reveal another, and this single attempt has not made the notebook fully runnable.

In two-round mode, this new error does not simply end the record. The orchestrator reclassifies `tf_keras` as `missing_package/usable`, explains it as a record of its own (Section 5.7.2), and then evaluates its repair eligibility. The record is eligible, so RAGRepairAgent is called on it. Retrieval returns `mapping_unknown`, because `tf_keras` has no entry in the package-mapping table, and the second round abstains without a model call (Section 6.5). The reader still receives an explanation of the second error; the repair layer, by design, does not guess at a package it cannot resolve. Section 6.6.10 shows an evaluation-split record where the second round does reach a grounded repair, with the Round-2 explanation that precedes it.

7. Result logging. Applying the join logic of Section 5.8.2 to this record's classification, proposal, and outcome yields one `repair_attempts` row: `failing_module: "scipy", subtype: "wrong_version", action: "pin_version", install_name: "scipy", version: "1.13.1", outcome: "still_failing", new_error_type: "ModuleNotFoundError", new_error_message: "No module named 'tf_keras'", and round: 1`. Because a matching fix-application record exists, `run_id` and `created_at` are taken from that FixApplicator run, as described above. In two-round mode, the Round-2 invocation of step 6 is logged as well: the `tf_keras` abstention becomes a second row for the same notebook with `round: 2, action: "none", no outcome, and a pypi_evidence object whose status is mapping_unknown` (Section 5.7.4). This specific Round-1 row was not produced by a committed Result-Logger run; the mapping's pilot validation (Section 6.4) used a different combination of records. The row is shown here as the mechanical result of the documented join logic applied to this record's validated outputs, not as a row already in the database.

8. Knowledge-graph link. Exporting the Round-1 row and running it through `repair_attempts.rml.ttl` attaches it to the existing notebook resource for `notebook_id 428`, the notebook's confirmed identifier (Section 5.8.6). The RDF below shows this Round-1 row only; the Round-2 abstention row would become a second `repr:RepairAttempt` resource linked from the same notebook:

```
repr:notebook_428 repr:hadRepairAttempt repr:repairattempt_{id} .
repr:repairattempt_{id} a repr:RepairAttempt, prov:Activity ;
repr:failingModule "scipy" ;
repr:fixAction "pin_version" ;
repr:fixVersion "1.13.1" ;
repr:fixOutcome "still_failing" ;
repr:exception "ModuleNotFoundError" ;
repr:msg "No module named 'tf_keras'" .
```

The {id} placeholder is left as a template, as in Section 5.8.5, because this row has not been assigned a primary key by an actual insert. This worked example does not represent a repair success. It shows what Section 6.3.1 establishes across the whole pilot: the implemented chain carries a dependency failure through explanation, grounded repair, re-execution, and structured logging, and it records that the first repair removed its targeted dependency error and exposed another dependency error afterwards.

5.10 Design Decisions and Trade-offs

The sections above explain each decision next to the component it affects. Table 5.3 collects the eleven decisions with the largest effect on how the results in Chapter 6 should be read. For each decision it states the alternative that was not taken and the cost of the choice that was. Several of these costs show up directly in the evaluation, so the table is worth reading before that chapter. It introduces no new decision; every row is justified in the section cited.

Table 5.3: The design decisions with the largest effect on how the evaluation should be interpreted, each stated as decision, alternative, reason, and cost.

Decision	Alternative	Why chosen	Trade-off
Deterministic classification before any LLM call	Let a model judge scope and subtype	A decision that gates automated repair must be auditable and reproducible (Section 5.2)	Fixed rules miss cases a model might read correctly; 4 of 49 manual labels disagree (Section 6.6.12)
Open, locally deployable model (Gemma-2 9B via Ollama)	A proprietary hosted API	Open-model requirement NFR1, no hosted-API fee, and the primary run’s inputs stay on the local machine (Section 5.4)	CPU-only local serving with high latency, which timed out on 27 explanation calls in the final Gemma run (Section 6.6.3)

continued on the next page

5 Methodology and Implementation

Table 5.3, continued from the previous page.

Decision	Alternative	Why chosen	Trade-off
PyPI grounding through an explicit import-to-distribution mapping, frozen for the evaluation	Free-form repair from the model’s memory, or a mapping extended as gaps appear	Every proposal names a verified package and, for version pins, a version from the grounded candidate set; the frozen table prevents tuning on the reserved split (Sections 5.5.2 and 6.6.2)	Abstention whenever the mapping cannot resolve the import: 162 of 235 invocations (Section 6.6.6)
Hand-curated compatibility registry for wrong_version	Automated changelog or API-history mining	PyPI metadata cannot prove that a symbol exists in a release (Section 5.5.3)	Three entries, scoped to this dataset; unseen incompatibilities can only be abstained on
Repository dependency files rebuild the baseline environment only; they are not repair evidence	Hand the declared package list to the repair model	Declared files may be incomplete, stale, unpinned, or contain the failing version; PyPI evidence is verifiable (Section 5.5.5)	Version hints an author did record go unused; a declared pin that conflicts with the proposal surfaces only at re-execution (Section 7.2)
The model never writes the install command	The model returns the command string	Free-text output must not reach a subprocess; the command is built deterministically after validation (Section 5.5)	Only the three structured actions can be expressed
Newly exposed Round-2 error explained before its eligibility is checked	Explain only errors that will be repaired	The reader gets an account of every error encountered; explanation and repair stay independent (Section 5.7.2)	One extra explanation call per new error (50 in the final run)

continued on the next page

5 Methodology and Implementation

Table 5.3, continued from the previous page.

Decision	Alternative	Why chosen	Trade-off
Maximum two repair rounds	Open-ended iteration until no error remains	A bounded, reproducible evaluation budget (Section 5.7.3)	A notebook that exposes another dependency error after Round 2 remains unfixed within the two-round budget. (Section 6.6.10)
Rebuild the environment on every attempt, with commit pinning where metadata is reachable	Reuse one long-running container across attempts	Clean isolation between attempts, same recipe as the upstream run (Section 5.6.3)	Slower, and in the final runs the metadata path was unreachable, so default branches were cloned (Section 6.6.2)
One <code>repair_attempts</code> row per repair-agent invocation, with the full explanation and PyPI evidence kept as JSON	Log only applied repairs, or flatten the evidence	Every denominator in the evaluation can be reconstructed from the table, and abstentions are visible (Section 5.8)	A larger table in which 162 of 235 rows carry no outcome
Separate targeted-error-resolution and full-recovery metrics	One binary success metric	Distinguishes “the local repair worked” from “the whole notebook runs” (Section 6.6.5)	A more complex evaluation to report and to read

6 Validation and Evaluation

This chapter reports the evidence behind every claim this thesis makes about its own system, in increasing order of scale.

It starts with the automated test suite, which checks each component’s logic in isolation against mocked external systems. It then reports the real-system pilots: small, hand-picked runs that check whether the same components behave correctly against the real PyPI registry, a real Ollama model, real Docker execution, real GitHub repositories, and the real Morph-KGC mapping engine. Section 6.5 extends these to the orchestrator driving several components together in one unattended pass.

Section 6.6 then reports the thesis’s final evaluation: the final frozen implementation of the same orchestrator, run over the complete 187-row evaluation split (Chapter 3) with Gemma-2 9B. Its explanation results (Section 6.6.3) together with the human study in Section 6.7 answer RQ1; its repair results (Section 6.6.4 to Section 6.6.14) answer RQ2, keeping targeted-error resolution and full notebook recovery apart. Section 6.6.15 repeats the evaluation with the language model replaced by Qwen3.6-35B-A3B-MLX-8bit and answers RQ3. Section 6.8 states what all of this does and does not license concluding.

6.1 Automated Testing

The repository’s test suite contains **711 passing tests**, organised by the component they exercise (Table 6.1). This measures software correctness, not repair quality or explanation quality. It confirms that each component’s logic behaves as designed against controlled inputs. It says nothing about how well the system performs on real notebooks at scale.

Dataset preparation has no dedicated automated test file of its own. Instead, its output is checked a different way: the counts in Chapter 3 are recomputed directly from its output files. They agree exactly.

Across these areas, the tests specifically check:

- rejection of malformed or schema-invalid model output;
- rejection of a repair proposal that invents a package name or a version outside the retrieved evidence;
- the shell-safety checks from Section 5.6, including a static check that `shell=True`, `os.system`, `eval`, and `exec` never appear anywhere in the components that build or run a command;

Area	Tests	What is checked
Dataset preparation, context extraction, classification	9	Root-cause hints, confidence levels, legacy-hint acceptance (Section 5.3).
Explanation component	33	Prompt rendering, explanation schema validation, retry on failure (Section 5.4).
PyPI retrieval, compatibility evidence, repair-proposal validation	214	Import-name mapping, candidate filtering, compatibility registry, grounding validation, argument-list construction (Section 5.5).
Fix application and Docker orchestration	87	Command re-validation, dataset re-join, outcome classification, cleanup (Section 5.6).
Orchestration and the bounded second round	66	Sequencing, shared run id, Round-2 trigger, reclassification and explanation of a new error, hard cap, cumulative environment (Section 5.7).
Result logging, CSV export, knowledge-graph tooling	36	Joins and duplicate handling, SQLite persistence, CSV export, notebook-identity alignment (Section 5.8).
Evaluation manifest, metrics, integrity checks, manual-validation tooling	214	Hashing and provenance, frozen metric definitions, explanation metrics, the 18 integrity checks, ground-truth scoring (Section 6.6.2).
Model-provider abstraction (Ollama, Kiste)	10	Provider dispatch shared by the Gemma and Qwen runs (Section 6.6.15).
Human-evaluation tooling	42	Pool sampling, survey reshaping, descriptive statistics, Cronbach’s alpha (Section 6.7.3).
Total	711	

Table 6.1: Automated tests by area. Counts are the collected test functions per area; all 711 pass.

- the three-outcome classification logic, including the conservative same-error check;
- cleanup behaviour under both success and failure; and
- for ResultLogger, rejection of an ambiguous duplicate explanation record, and correct separation of two repair attempts that share the same `notebook_execution_id` into two distinct rows.

All of these tests run against mocked subprocesses, a mocked Ollama endpoint, and mocked HTTP responses. This is deliberate, and it is also a limitation. A mocked test can prove that a component’s logic is internally consistent. It cannot prove that the component behaves correctly against a real operating system, a running Docker environment, or a real network; Section 5.6.7 showed three real bugs that a mocked test could not have caught. The pilots below check exactly that.

6.2 RAGRepairAgent Against Real PyPI and a Real Model

6.2.1 Retrieval Against Real PyPI

The five import names used in the original retrieval-design check (`sklearn`, `umap`, `pkg_resources`, `scipy`, `dms_variants`) were re-checked against the real `retrieve()` entry point and the live PyPI registry. All four mapped names resolved to their real PyPI distributions, with plausible, current candidate versions. `dms_variants` has no verified mapping; it returned `mapping_unknown`, and no PyPI request was made. This confirms under real conditions that an unmapped import never reaches the network.

A second check, made during the implementation of RAGRepairAgent, extended the mapping table by two entries. `cv2` and `skimage` are among the frequent failing modules of the working dataset (Table 3.3), and neither had an entry in `config/package_mapping.yaml`, so the retriever returned `mapping_unknown` for both. Their distributions are generic, well-known import-to-distribution correspondences, not information derived from any evaluation outcome. The entries `cv2: opencv-python` and `skimage: scikit-image` were added on 2026-08-23, checked against the real PyPI registry, and confirmed by a regression check that the existing `sklearn` mapping still resolved correctly. The mapping table has not changed since: it was frozen with the rest of the configuration on 2026-09-12 (Section 6.6.2), before the final runs, and no entry was added in response to a final evaluation result (Section 6.6.13).

6.2.2 Repair Proposals Against a Real Model

A small pilot ran `rag_repair_agent.py`’s real entry point, using a real local Ollama model, in four pilot runs:

- An excluded, `system_library` record correctly abstained with zero PyPI or Ollama calls, confirming the eligibility gate works under real conditions.
- A `missing_package` record (`sklearn`) timed out twice under the initial 120-second limit, then succeeded at the first attempt once the limit was raised to 300 seconds. This is the evidence behind the timeout change reported in Section 5.5.
- The same record, rerun at 300 seconds, produced a schema-valid, grounded proposal: `install scikit-learn`.
- A `wrong_version` record (the `scipy.integrate.cumtrapz` case from Table 5.1, traced in full in Section 5.9) produced a grounded `pin_version` proposal for `scipy==1.13.1`, using the real compatibility-evidence intersection on the first attempt.

6.2.3 What This Pilot Does and Does Not Show

The pilot shows that real PyPI retrieval, real import-name mapping, and real model proposal generation work together across RAGRepairAgent’s retrieval-to-proposal flow. It also shows that a timeout leads to a retry and then to a safe abstention, not to a hang or an invented result. It does not show an installed package, a modified environment, or a re-executed notebook, because none of that happens at this step. Validating a proposal is FixApplicator’s job, covered next.

6.3 FixApplicator Against Real Docker Execution

A second pilot ran `apply_and_validate()` for real. It built and ran containers against real GitHub repositories, reusing the already-validated repair proposals from the pilot above. During this pilot, the upstream sibling database was locked by another process, so one commit lookup fell back to its documented “unavailable” behaviour. This was a real, unplanned exercise of that fallback path.

Cases 4 and 5 were chosen deliberately, after cases 1–3, to give the pilot the best possible chance of reaching a completely clean `fixed` result. Both repositories have no declared requirements file, no `setup.py`, and exactly one notebook. This minimises the chance of a second, unrelated dependency gap.

6.3.1 Key Observation: Fixing One Dependency Can Reveal Another

In each of the four non-synthetic repair cases, the applied fix removed the targeted error and execution then failed on a different problem. Case 3 was the controlled infrastructure-failure case and did not reach repair execution. This happened across four repositories, two fix actions, and packages with no shared code. It is not a defect

Case	Original error	Fix applied	Result
1 – sklearn	ModuleNot- FoundError: sklearn	install scikit-learn	Original error gone; still_failing on a later, unrelated No module named 'pandas'.
2 – scipy	ImportError: cumtrapz	pin scipy==1.13.1	Original error gone; still_failing on a later, unrelated No module named 'tf_keras'.
3 – controlled failure	synthetic: non-existent repository	n/a	apply_error, failure stage clone; no Docker artefacts created.
4 – cv2	ModuleNot- FoundError: cv2	install opencv-python	Original error gone; still_failing on a missing system library (libxcb.so.1), a known characteristic of opencv-python on a minimal base image.
5 – sklearn (2nd repo)	ModuleNot- FoundError: sklearn	install scikit-learn	Original error gone; still_failing on a later, unrelated No module named 'tensorflow'.

Table 6.2: Real-Docker pilot cases. Case 3 is a deliberately synthetic infrastructure-failure test, not a dataset row.

in FixApplicator’s classification logic, which is unit-tested separately and reports a clean **fixed** result as soon as one occurs. It is a property of the dataset. The Docker pipeline’s error classification records only the *first* import failure a notebook hits. A real research notebook can depend on several packages that are each missing. Once the first one is repaired, execution goes further and can reach the next one, which was invisible until then. The conservative **same_as_original_error** check (Chapter 5) told “a different, new problem” apart from “the fix did not work” in every one of these cases.

This pilot produced no completely clean **fixed** outcome, even though it was designed to find one. A clean outcome remains achievable in principle on a single-dependency notebook. The two-round repair mode carries the new error of a **still_failing** result into reclassification and explanation and, if the error is repair-eligible, into a second repair attempt. It was demonstrated for real on cases 1 and 2 above (Section 6.5): in both, the second round triggered correctly and then abstained rather than proposing anything ungrounded. Section 6.6 shows that this pattern also occurs frequently in the full evaluation: many successful targeted repairs removed the original dependency error but exposed a later failure.

6.4 Result Logging and Knowledge-Graph Enrichment

6.4.1 Notebook-Identity Alignment

The RML mapping in Section 5.8.5 links a repair attempt to the notebook it repairs. This only works if the Docker pipeline’s **notebook_id** and **repository_id** identify the same notebooks and repositories as the existing FAIR Jupyter knowledge graph. That graph was generated from the earlier, conda-based pipeline (Section 3.2.1), not from the Docker pipeline this thesis builds on, so the alignment could not be assumed. It was checked directly. All **214** rows of the classification dataset (Section 5.3) were cross-checked against the knowledge graph’s own **repositories.csv** and **notebooks.csv** source files, comparing each row’s resolved repository path and notebook filename for an exact string match. All 214 rows matched on both **repository_id**→**repository** and **notebook_id**→**name**, with zero missing and zero mismatched records.

Within the inspected dataset, the Docker pipeline and the FAIR Jupyter knowledge graph therefore use the same **repository/notebook** primary keys for the same notebooks. A repair attempt can link directly to its existing notebook resource via **notebook_id** (Section 5.8.6), and no crosswalk table between the two pipelines is needed. This result establishes alignment for the inspected FAIR Jupyter source dataset. It does not extend to a knowledge-graph deployment built from a different data snapshot.

6.4.2 Executing the RML Mapping with Morph-KGC

The RML mapping described in Section 5.8.5 was also executed, with Morph-KGC, the mapping engine the existing `run_fairjupyter_kg.sh` build script uses for every other table in the knowledge graph. A `repair_attempts.csv` file was assembled from the real classification, explanation, and repair-proposal pilot records reported in Section 6.2. No live fix-application batch output existed yet, so one row used a labelled synthetic FixApplicator record, so that the outcome and post-repair error fields could also be exercised in the generated RDF.

Running the mapping over this file produced valid Turtle with no RML or Turtle syntax error. The resulting RDF was then inspected directly. A SQLite column left NULL, such as an abstained attempt’s `install_name`, produced no triple for the corresponding predicate, never an empty-string literal. `repr:exception` and `repr:msg` appeared only on the one row that carried a post-repair error. `prov:endedAtTime` was present, typed `xsd:dateTime`, on every row.

The generated `repr:hadRepairAttempt` links were also checked against real notebook resources rather than against the CSV’s `notebook_id` column alone. The existing, unmodified `notebooks.rml.ttl` was run against the real `notebooks.csv`. It produced 627,127 triples, confirming that this file executes as the rest of the knowledge graph expects. The notebook IRIs the repair mapping links to were present there, typed `repr:Notebook` and carrying the expected `pav:retrievedFrom` link to their repository. All validation output was written to a scratch location; the FAIR Jupyter knowledge-graph checkout itself was left untouched.

This pilot shows that the mapping produces syntactically valid RDF for real data, links correctly to existing notebook resources for the records tested, and handles missing values and the PROV-O time literal correctly.

This pilot check used a small, real sample built for the purpose. The same mapping was afterwards run, unchanged, over the complete `repair_attempts` table of the final evaluation; the next subsection reports that run.

6.4.3 Full-Scale Materialisation of the Final Evaluation Run

Once the final evaluation of Section 6.6 had been frozen, its `repair_attempts` table was exported with the existing exporter (Section 5.8.4) and materialised with the existing, unmodified mapping (Section 5.8.5) through Morph-KGC, invoked as one process for this one mapping file, exactly as the upstream `run_fairjupyter_kg.sh` build script invokes every mapping. The table is the SQLite database of run `i10-eval-gemma-round2-explanations-20260918T161356Z`; it holds 235 rows, 187 for Round 1 and 48 for Round 2. No component, mapping, configuration, or result value was changed for this step, and no evaluation run was repeated: the chain pilot-validated above was run over the full table as it stood.

The export was checked before the mapping ran. All 235 SQLite rows appear in the CSV exactly once. Every one of the 18 table columns round-trips byte for byte, and all 1,222 NULL cells become empty fields. All 443 serialised explanation

and PyPI-evidence documents survive unchanged. Every row’s `notebook_id` was re-derived independently from the classification dataset and matched. Morph-KGC then generated 3,478 triples; Table 6.3 lists what they contain.

Content	Class or predicate	Triples
Repair-attempt resources, one per row	<code>rdf:type repr:RepairAttempt</code> and <code>rdf:type prov:Activity</code>	2×235
Notebook $\rightarrow$ repair links, to 187 distinct notebooks	<code>repr:hadRepairAttempt</code>	235
Present on every row	<code>repr:failingModule</code> , <code>repr:subtype</code> , <code>repr:fixAction</code> , <code>repr:pypiEvidence</code> , <code>repr:llmModel</code> , <code>repr:promptStrategy</code> , <code>repr:round</code> , <code>repr:runId</code> , <code>prov:endedAtTime</code>	9×235
Explanation (27 Round-1 rows have none, Section 6.6.3)	<code>repr:explanation</code>	208
Executed fix applications	<code>repr:installName</code> , <code>repr:fixCommand</code> , <code>repr:fixRationale</code> , <code>repr:fixOutcome</code>	4×73
Version pins (wrong_version repairs)	<code>repr:fixVersion</code>	18
Post-repair error present	<code>repr:exception</code> , <code>repr:msg</code>	2×70
Total		3,478

Table 6.3: Triples generated by the unmodified `repair_attempts.rml.ttl` from the 235-row `repair_attempts` table of the final Gemma run. Every per-predicate count equals the number of rows in which the corresponding column is non-NULL; a NULL column yields no triple.

The generated graph was then validated with 24 explicit pass/fail checks against the SQLite table as ground truth, not against Morph-KGC’s own report. The output parses as both N-Triples and Turtle. There is exactly one `repr:RepairAttempt` resource per SQLite row, each also typed `prov:Activity`, and no duplicate repair-attempt IRI. Every resource is the object of exactly one `repr:hadRepairAttempt` link, and the subject of that link is the notebook IRI of the row’s own `notebook_id`. For every column of every row, the graph holds exactly one literal equal to the SQLite value when the column is non-NULL, and no triple when it is NULL. The graph

contains no empty-string literal, and the per-predicate counts in Table 6.3 equal the table’s non-NULL column counts exactly. `repr:fixOutcome` therefore appears on exactly the 73 rows with an executed fix application, and `repr:exception` and `repr:msg` on exactly the 70 rows whose re-execution ended in a post-repair error. `prov:endedAtTime` is present on all 235 resources, typed `xsd:dateTime`, and parses as a timestamp on every one. The `repr:round` literal partitions the resources into 187 Round-1 and 48 Round-2 resources, and each of the 48 Round-2 notebooks also carries a Round-1 resource.

The notebook side was checked as in the pilot, now at full scale. The unmodified upstream `notebooks.rml.ttl` was run as its own Morph-KGC process over the real `notebooks.csv` and `repositories.csv`. It produced the same 627,127 triples as before, and every one of the 187 notebook IRIs the repair mapping links to exists there as an `rdf:type repr:Notebook` resource with its `pav:retrievedFrom` repository link. Over the union of the two outputs (630,605 triples), a SPARQL query that starts from a typed notebook, follows `repr:hadRepairAttempt`, and requires a typed repair attempt reaches all 235 repair attempts from all 187 notebooks. All 24 checks passed. The exported CSV, the generated triples, the Morph-KGC configuration and log, the check script, and its report are stored next to the run’s raw output. The upstream knowledge-graph checkout was again only read, never written, and the generated triples were not loaded into the public FAIR Jupyter triple store; Section 7.1 treats that deployment as future work. One vocabulary detail is inherited rather than chosen: the mapping binds the `prov:` prefix to the namespace IRI that every upstream FAIR Jupyter mapping declares (<http://www.w3.org/ns/prov-o#>), so the generated `prov:Activity` and `prov:endedAtTime` terms follow the existing graph’s convention rather than the W3C PROV namespace.

With this run, objective O6 is met at evaluation scale. It is met by an artefact distinct from the one that meets O4: the benchmark table of Section 6.6.2 and the graph generated from it describe the same 235 repair attempts, one as rows and the other as linked RDF resources.

6.5 The Orchestrator End to End

A further pilot ran the orchestrator itself (`scripts/run_pipeline.py`, Section 4.4) end to end, from classification through result logging, including the bounded two-round repair loop. It ran against real Ollama, real PyPI, and real Docker execution for two development-split records. Both cases reproduce the ones reported in Section 6.3 and extend them with a real Round-2 attempt, as Table 6.4 sets out.

Round 1 of both cases reproduced the FixApplicator-only pilot results in Table 6.2 exactly, now driven by the orchestrator instead of by hand. Both cases carry one shared `run_id` across both rounds, confirmed directly in the resulting SQLite rows. Each round produced its own `repair_attempts` row (Section 5.8).

Neither Round-2 attempt in this small pilot produced a successful second repair. Both triggered correctly and abstained correctly, for two different reasons. The

Case	Round 1	Round 2 trigger	Round 2 result
A – sklearn (id 8)	install scikit-learn; still failing on No module named 'pandas'	Fired: pandas reclassifies as missing_package/ usable	RAGRepairAgent was called against pandas and abstained after two invalid-JSON responses. The model produced JSON that echoed PyPI's own "unparseable filename" warnings for pandas's many legacy Windows release artifacts, which were embedded in the prompt's evidence block. This is a genuine model failure mode; FixApplicator never reached Docker.
B – scipy/ cumtrapz (id 174)	pin scipy==1.13.1; still failing on No module named 'tf_keras'	Fired: tf_keras reclassifies as missing_package/ usable	RAGRepairAgent was called against tf_keras and abstained: PyPI retrieval returned mapping_unknown because tf_keras has no entry in config/package_mapping.yaml. This confirms that the documented case cannot currently be repaired end to end; no mapping entry was added to force a different outcome. FixApplicator never reached Docker.

Table 6.4: Real orchestrator pilot: both cases exercise a genuine Round-2 trigger followed by a RAGRepairAgent abstention, each for a different recorded reason.

mocked test suite (Section 6.1) separately checks that the second-round mechanism works end to end for cases where RAGRepairAgent does propose a fix and FixApplicator does apply it: the trigger conditions, the reclassification and explanation of the new error, the cumulative environment, the shared run identifier, and the two-round cap. Section 6.6 reports how the second round behaved across the full evaluation split. Round 2 fired for 48 notebooks and produced a real second repair proposal for 12 of them. As in this pilot, none of those 12 second attempts produced a completely clean `fixed` result.

Table 6.4 focuses on validating the second-round repair-control flow. Round-2 explanation generation is evaluated separately as part of the final integrated pipeline in Section 6.6.

6.6 Final Evaluation on the Reserved Evaluation Split

The previous sections report results from smaller pilot runs used to validate individual components and the integrated pipeline. This section presents the final evaluation on the reserved evaluation split introduced in Chapter 3. Unlike the pilots, the complete frozen pipeline was executed unattended across the full evaluation set using real external services and Docker execution.

The final run completed successfully and passed all 18 post-run integrity checks described in Section 6.6.2. The following subsections describe the evaluation design, report the explanation and repair results, and analyse the behaviour of the complete pipeline. A separate model-sensitivity evaluation is presented in Section 6.6.15.

6.6.1 Evaluation Design

The orchestrator was invoked with a two-round repair budget (`max_rounds = 2`) over all 187 evaluation rows. For every notebook, Round 1 classifies and explains the original failure. If retrieval can ground a repair proposal, the pipeline passes it to FixApplicator, which applies the repair and re-executes the notebook. If Round 1 reached FixApplicator, came back `still_failing`, and the notebook now fails on a genuinely new error, that error is reclassified by the same classifier used for every original row and explained as a record of its own, whatever its scope (Section 5.7). Round 2 is attempted only when the reclassified error is itself pip-repair-eligible. It runs the same RAGRepairAgent and FixApplicator components, targeting only the newly exposed error, inside a freshly rebuilt container. That container first reproduces Round 1’s already-validated fix before Round 2’s own fix is applied; Section 5.7.3 explains why this step is necessary. Two rounds is the hard cap; no third round exists.

Two populations therefore appear in the results below and should not be confused. Explanation results are counted per *encountered error*: the 187 original failures, plus every newly exposed error that was reclassified after Round 1, whether or not it was then repaired. Repair results are counted per *repair-agent invocation* and per

notebook. A Round-2 explanation call is not a repair-agent invocation and enters no repair metric.

As explained in Section 4.4, Round 1 and Round 2 are not two separate experiments. Both conditions are read from the same run, so Ollama’s non-zero sampling temperature cannot turn ordinary run-to-run variation into an apparent Round-2 effect. The comparison that follows is between the outcome after Round 1 and the final outcome after up to two rounds of the same execution, not between two independently sampled runs.

Each of the 187 rows is a notebook execution already known to fail with a dependency error before this evaluation began. Re-executing a notebook after a repair can expose a dependency error that was not visible in the original dataset row – simply because execution now reaches further into the notebook than it did before. Section 6.6.14 returns to what this means for interpreting the results below.

6.6.2 Reproducibility Manifest and Evaluation Freeze

The final evaluation was performed only after the implementation, prompts, package mapping, and evaluation split had been frozen. A machine-readable evaluation manifest records the source revision, model configuration, evaluation split, and SHA-256 hashes of the files that define the pipeline. This makes it possible to trace the reported results back to the exact configuration used to produce them.

Both final model runs used the same frozen pipeline implementation and the same 187 evaluation records. They were executed from the same implementation commit (46d90aa4) with identical pipeline-component hashes. The runs differ only in the language model and the model- and provider-specific serving configuration described in Section 6.6.15.

Repository revision during re-execution. One limitation affects how the re-execution results should be interpreted. FixApplicator supports checking out the repository commit recorded by the upstream pipeline (Section 5.6). During the final evaluation, however, the metadata database configured for FixApplicator could not be accessed from the WSL environment because its configuration contained a Windows-style path. The commit lookup therefore returned no commit, and the evaluated repositories were cloned from their current default branches instead.

Of the 73 real FixApplicator attempts, 71 record that commit checkout was skipped; the remaining two failed during cloning before a checkout could be attempted. The same metadata lookup also provides the locations of repository `setup.py` files. Because this metadata was unavailable, those files were not installed during the final runs. Root-level `requirements.txt` files were still used where available (Section 5.6.3).

The final evaluation should therefore not be interpreted as a commit-pinned reproduction of the historical executions. A repository may have changed between the original failure and the evaluation run. This limitation applied equally to both model runs, so their comparison in Section 6.6.15 uses the same execution policy.

The raw results are kept separate from the derived evaluation metrics. Each run produces a per-notebook JSONL trace and a `repair_attempts` SQLite database (Section 4.5.2). The evaluation tables and summary metrics are computed from these raw outputs rather than from manually adjusted intermediate values. The same SQLite database is also used for the full knowledge-graph materialisation reported in Section 6.4.3.

Before any metrics are reported, the raw output is checked by 18 integrity tests. These checks verify the expected notebook population, the Round-1 and Round-2 structure, shared run identifiers, the relationship between newly exposed errors and their Round-2 records, and consistency with the frozen configuration. Both final evaluation runs passed all 18 checks. Figure 6.1 summarises this evaluation chain.

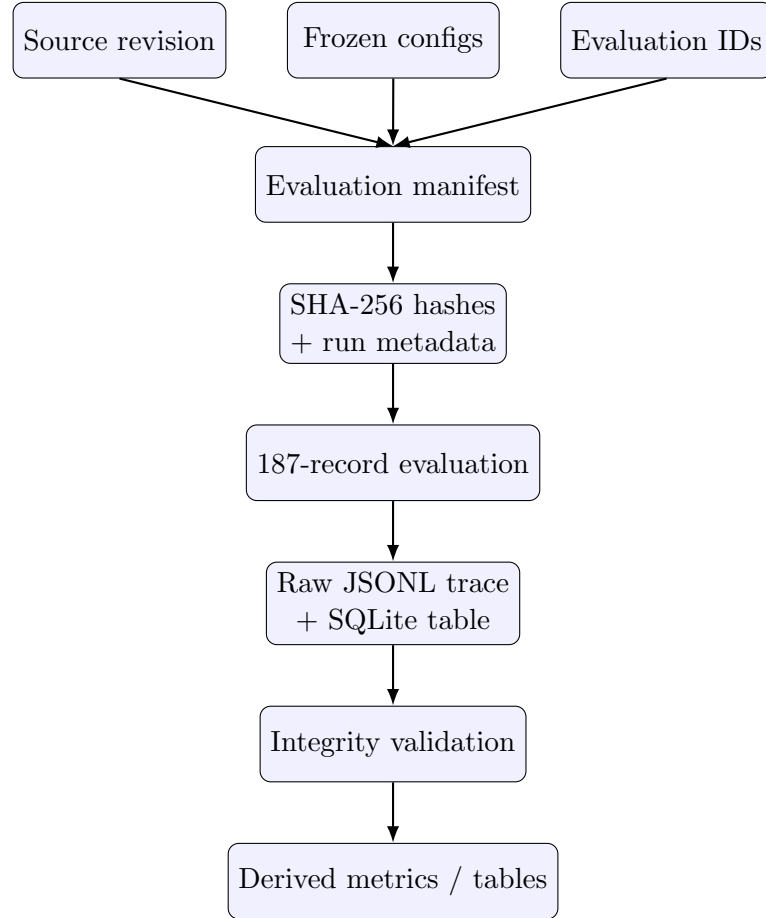


Figure 6.1: Traceability of the final evaluation. The frozen source revision, configuration, and evaluation IDs are recorded before execution. The resulting raw outputs are validated before the reported metrics and tables are computed.

6.6.3 Explanation Completion and Schema Validity

LLMExplainer is measured before the repair results, in the order the pipeline runs, with two separate questions that are easy to conflate. The first is *completion*: did the explanation call return a response that could be processed at all, within the one-retry budget? A call that timed out at the explainer’s 120-second limit, or that met an unavailable model service, produced no model output to judge. Such a record is a *runtime failure*, attributed to the model service and the machine it ran on, not to the model’s output. The second question is *schema validity among completed responses*: of the responses that did arrive, how many validated against the six-field explanation schema (Section 5.4.3)? Only this second figure says anything about the quality of what the model produced. A single combined “validity” number would let a slow machine look like a model that writes malformed JSON, so the two are kept apart throughout.

Both are counted per encountered error, one record each. Retries are attempts, not records: a record that needed the corrective second attempt counts once, and the attempt counts appear separately. Table 6.5 gives the Gemma-2 9B results for three populations: the 187 original failures, the newly exposed errors reclassified after Round 1, and the two combined.

Gemma-2 9B	Original	Round 2	Combined
Explanation records	187	50	237
Completed	160	50	210
Completion rate	85.6%	100%	88.6%
Runtime failures	27	0	27
timeout	26	0	26
model unavailable (HTTP 500)	1	0	1
Schema-valid among completed	160/160	50/50	210/210
Schema validity among completed	100%	100%	100%
Explanation LLM attempts incl. re-tries	222	50	272
Records needing the retry	35	0	35

Table 6.5: Explanation completion and schema validity for the final Gemma-2 9B run, counted per encountered error. “Original” covers the 187 original failures of the evaluation records. “Round 2” covers the 50 newly exposed errors reclassified after Round 1, including the two that were explained but were not repair-eligible. A runtime failure is a call that produced no processable response within one retry; it is not a schema-invalid model output. Explanation calls are never counted as repair-agent invocations.

Three points follow from the table. First, every response Gemma completed was schema-valid: 210 of 210, across both rounds. No completed response was malformed JSON or missing a required field. Second, the 27 records without an explanation are all runtime failures of the local, CPU-only Ollama service: 26 requests exceeded the

120-second timeout on both attempts, and one met an HTTP 500 from the service. These are a property of the evaluation machine and the model’s speed on it, not of the model’s output. Third, the Round-2 population is complete. Round 1 exposed and reclassified 50 new errors: 48 `missing_package`, one `system_library`, and one non-dependency `IndexError` classified `out_of_scope`. All 50 received an explanation on the first attempt. Only the 48 `missing_package` records were repair-eligible and went on to Round 2 (Section 6.6.6); the other two were explained and stopped there, and their explanations exist in the run trace only, with no `repair_attempts` row. Of the 50, 49 are dependency-related errors and 48 were repair-eligible; the 50 is a count of explained records, not of repairable ones.

Schema validity is a structural check and nothing more. Whether these explanations are understandable, detailed enough, and perceived as correct by a reader is the question the human evaluation in Section 6.7 answers, for the original-failure explanations only.

6.6.4 Primary Repair Results

Full notebook recovery means a notebook re-executes from beginning to end with no error. No evaluation notebook achieved this, in either round: Round-1 repair success is 0/187, the final success rate after up to two rounds is 0/187, and Round 2 therefore added no fully fixed notebook. Full recovery is the answer to RQ2’s second half. Its first half, whether the targeted dependency error was removed, is a different question with a different denominator, and Section 6.6.5 answers it at the level of individual repair attempts.

6.6.5 Targeted Dependency-Error Resolution

Full notebook recovery requires every dependency problem in a notebook to be gone after re-execution. A narrower, attempt-level question asks whether the one dependency error that a repair attempt targeted was removed, regardless of whether the notebook then failed on something else. A repair attempt counts as resolving its targeted error in two cases: the notebook becomes `fixed`, or it stays `still_failing` but the original error is gone and the conservative `same_as_original_error` check (Chapter 5) confirms that the notebook moved on to a genuinely different error.

Across the 73 real FixApplicator attempts in this evaluation (61 in Round 1, 12 in Round 2), 62 resolved the error they targeted: 50 of the 61 Round-1 attempts and all 12 Round-2 attempts. The targeted dependency-error resolution rate is therefore 84.9% (62/73). This is an attempt-level figure, not a notebook-level one, and it should not be confused with the 0% full-recovery figure above; Figure 6.2 (Section 6.6.9) shows both levels side by side. No notebook completed successfully within two rounds, but the large majority of individual repair attempts removed the specific dependency error they addressed. Re-execution then frequently exposed a following failure. This is the pattern observed at pilot scale in Section 6.3.1 and traced through one real record in Section 5.9.

6.6.6 Abstention: Round 1, Round 2, and Final State

In a large share of its invocations, RAGRepairAgent declined to propose a repair, so no repair attempt followed. This is abstention. It is distinct from an infrastructure fault and from a failed repair: when the system lacks grounded package information, it refuses to generate an unsupported fix rather than guessing.

In Round 1, RAGRepairAgent abstained on 126 of the 187 rows (67.4%, the Round-1 abstention rate). All 126 cases have the same cause: PyPI retrieval returned `mapping_unknown`, meaning that RAGRepairAgent’s static import-to-distribution mapping (Chapter 5) could not resolve the failing import to a supported PyPI package. In all 126 cases the language model was never called; the abstention happens one layer earlier, at retrieval.

Round 2 does not simply repeat this figure. Of the 187 rows, 50 had a real Round-1 attempt that came back `still_failing` on a genuinely new error. All 50 were reclassified and explained (Section 6.6.3). The 48 newly exposed `missing_package` errors were repair-eligible; the two others, a missing system library and a non-dependency `IndexError`, fell outside repair scope and received no second round. Table 6.6 shows what happened to the 48 eligible notebooks. Only 12 received a second repair proposal and a real second FixApplicator attempt. The remaining 36 abstained again. As in Round 1, every one of these 36 abstentions is a `mapping_unknown` case: the newly exposed error’s import name had no entry in the package mapping either.

Combining both rounds, 162 of the 187 evaluation rows (86.6%) ended in abstention as their final classification: the 126 Round-1 abstentions plus the 36 Round-2 abstentions above. The Round-1 abstention rate (67.4%) and the final-state abstention rate (86.6%) are reported as two separate figures because they answer different questions. The first asks how often the very first repair attempt could not get past package resolution. The second asks how often the notebook’s terminal repair decision was an abstention: 126 notebooks stopped at Round 1 without any proposal, while 36 received a Round-1 proposal and repair, exposed a new error, and then abstained on that newly exposed error in Round 2. All 162 final-state abstentions, in both rounds, have the same underlying cause: `mapping_unknown`. The dominant reason a repair attempt did not happen, in whichever round, is therefore package-mapping coverage, not the language model or the grounding logic.

6.6.7 Proposal Quality and Proposal Coverage

RAGRepairAgent was invoked 235 times across both rounds (187 in Round 1, 48 in Round 2). Of those invocations, 73 reached the point of calling the language model. The remaining 162 abstained beforehand, for the `mapping_unknown` reason described above. All 73 responses the model produced passed schema validation, and all 73 also passed grounding validation against the retrieved PyPI evidence. Whenever RAGRepairAgent’s retrieval step supplied a resolved candidate distribution, the model’s proposal was therefore schema-valid and grounded. This gives a proposal validity rate of 100% (73/73), a grounding pass rate of 100% (73/73 schema-valid

Stage	Count	Meaning
Newly exposed error reclassified and explained	50	Round 1 reached FixApplicator and came back <code>still_failing</code> on a genuinely new error; every such error received a Round-2 explanation.
Explained but not repair-eligible	2	One <code>system_library</code> and one <code>out_of_scope</code> error; explanation kept in the trace only, no second round.
Round-2 eligible	48	The newly exposed error is itself repair-eligible (<code>missing_package</code>).
Round-2 proposal generated	12	RAGRepairAgent reached the language model and produced a proposal for the newly exposed error.
Round-2 abstained (<code>mapping_unknown</code>)	36	Retrieval could not resolve the newly exposed error’s import name; the model was never called.
Round-2 FixApplicator attempts	12	Every generated proposal was applied and the notebook re-executed.
Round-2 fixed	0	No second attempt produced a completely clean result.
Round-2 still failing	12	Every attempted second repair left the notebook failing on some error.

Table 6.6: The second round, from the 50 newly exposed errors to the 12 second repairs actually applied. Every newly exposed error was explained; only repair-eligible ones proceeded. Round-2 eligibility does not by itself mean a second repair was applied: for three of every four eligible notebooks, package resolution abstained before RAGRepairAgent could propose anything.

proposals), and a grounded proposal rate among LLM invocations of 100% (73/73).

These three figures are conditional on the language model having been called. They say nothing about overall proposal coverage. Counting all 235 repair-agent invocations, including the 162 that abstained before reaching the model, only 73 ended in a grounded proposal: an overall grounded proposal coverage rate of 31.1% (73/235). The contrast between these two figures is central to interpreting this system. Its output is schema-valid and grounded whenever it produces a proposal. But the static package-mapping coverage decides whether the model is called at all, and it is the binding constraint on how often the model gets that chance.

6.6.8 Infrastructure Failures

Two of the 187 evaluation rows (1.1%) ended in an infrastructure failure: an environment or tooling fault, unrelated to repair quality, that prevented a fair trial of the method (Chapter 5). These cases are reported separately from abstentions, method failures, and `still_failing` outcomes, so that their external cause remains visible. They nevertheless remain in the 73-attempt denominator used for targeted-error resolution. This follows the same infrastructure-versus-method distinction already used throughout the pilots earlier in this chapter.

The rule behind this distinction is conservative. A case is classified as an infrastructure failure only when there is concrete evidence of an external execution or environment problem: a clone failure, a Docker build failure, a timeout, or a similar tooling fault. When the cause of a failure is ambiguous, it is kept as a method failure. Otherwise an ambiguous case could be absorbed into “infrastructure” and excluded from the method’s own results, which would make the reported repair figures look better than what was observed.

6.6.9 Final Outcome Breakdown

Figure 6.2 shows the combined picture directly, separating the attempt-level view already established in Sections 6.6.5 to 6.6.7 (Panel A) from the notebook-level final classification (Panel B). Table 6.7 gives the same notebook-level breakdown in numbers.

6 Validation and Evaluation

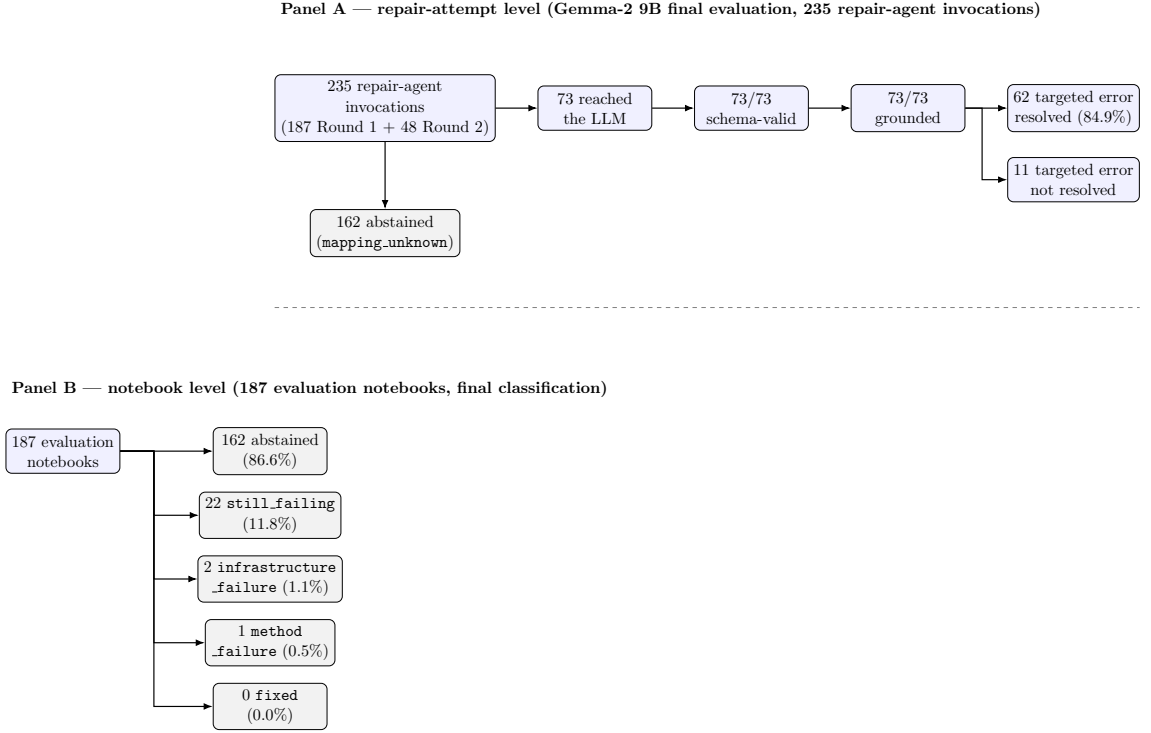


Figure 6.2: Two levels of the same final evaluation, for the frozen Gemma-2 9B run. Panel A follows every repair-agent invocation; Panel B classifies the 187 notebooks by their final outcome. The panels deliberately do not share one funnel, because they count different populations: 235 invocations against 187 notebooks.

Final category	Count	Share of 187
fixed	0	0.0%
still_failing	22	11.8%
abstained (126 Round 1 + 36 Round 2)	162	86.6%
infrastructure_failure	2	1.1%
method_failure	1	0.5%
excluded	0	0.0%
Total	187	100%

Table 6.7: Final classification of all 187 evaluation rows, after up to two repair rounds.

6.6.10 End-to-End Repair Case Study: Notebook 163

The aggregate results above do not show what happens inside an individual notebook. This subsection therefore follows one evaluation record through both repair rounds. Notebook execution id 163 was selected because both attempted repairs removed the dependency error they targeted, while the notebook still did not complete successfully. It therefore illustrates the distinction between targeted-error resolution and full notebook recovery.

The record corresponds to `notebooks/jDR_AJIVE.Template.ipynb` in `baprice/AJIVE-jDR-Integration`. Its original failure is an incompatibility involving `scipy.sparse.sputils.isshape`, one of the wrong-version patterns covered by the compatibility evidence in Table 5.1. Table 6.8 traces the complete two-round execution.

The execution exposed at least three dependency problems in sequence. The first repair removed the `isshape` incompatibility, after which the missing `pandas` dependency became visible. The second repair removed that error, after which the missing `matplotlib` dependency became visible. No further repair was attempted because the two-round budget had been exhausted.

This case illustrates why targeted-error resolution and full notebook recovery are reported separately. At the repair-attempt level, both attempted fixes succeeded in removing the errors they targeted. At the notebook level, the final outcome remains `still_failing` because another dependency error remained after Round 2. A single binary success measure would not capture this distinction.

Stage	Observed result
Original failure	<code>ImportError: cannot import name 'isshape' from 'scipy.sparse.sputils'.</code>
Classification	<code>failing_module: "scipy", subtype: "wrong-version", scope_status: "usable", with confidence high.</code>
Root-cause hint	<code>version_or_api_incompatibility.</code>
Explanation	Gemma-2 9B explains that the notebook tried to import <code>issshape</code> from <code>scipy.sparse.sputils</code> , but that name was not available. The explanation confidence is <code>medium</code> because the installed SciPy version cannot be determined from the recorded error alone.
PyPI evidence (Round 1)	The failing module resolves directly to the <code>scipy</code> distribution. The compatibility registry requires a version below 1.14.0, based on the same SciPy release-note evidence summarised in Table 5.1.
Repair proposal (Round 1)	<code>pin_version scipy==1.13.1</code> . The proposal is schema-valid and grounded in the retrieved evidence.
Round-1 result	<code>still_failing</code> . The original <code>issshape</code> error is no longer present. Execution instead reaches a new <code>ModuleNotFoundError: No module named 'pandas'.</code> The Round-1 targeted error is resolved.
Round-2 reclassification	The new error is classified as <code>failing_module: "pandas", subtype: "missing-package", and scope_status: "usable".</code>
Round-2 explanation	Gemma-2 9B explains that execution failed because Python could not find the <code>pandas</code> module. The explanation is schema-valid with confidence <code>high</code> . As in Round 1, the explanation is not passed to RAGRepairAgent.
Round-2 proposal	The new error is repair-eligible, so Round 2 is triggered. RAGRepairAgent proposes <code>install pandas</code> . The proposal is again schema-valid and grounded.
Round-2 result	<code>still_failing</code> . The <code>pandas</code> error is no longer present. Execution instead reaches another <code>ModuleNotFoundError: No module named 'matplotlib'.</code> The Round-2 targeted error is also resolved.
Final notebook outcome	<code>still_failing</code> . The two-round budget is exhausted before the newly exposed <code>matplotlib</code> error can be repaired. The error is recorded, but no third repair round is attempted.

Table 6.8: Evaluation-split notebook execution id 163 traced through classification, explanation, two repair rounds, and the final outcome. Both attempted repairs remove their targeted dependency error, but the notebook remains failing after the two-round budget is exhausted.

6.6.11 Results by Subtype

Split by the subtype recorded during classification (Chapter 5), full notebook recovery is 0/169 (0%) for `missing_package` and 0/18 (0%) for `wrong_version`, the two subtypes within repair scope (Chapter 3). Read this full-recovery figure together with the attempt-level targeted-error resolution rate in Section 6.6.5, not instead of it. A 0% full-recovery rate for a subtype does not mean individual repairs of that subtype’s errors were ineffective. It only means that no notebook containing one was left completely error-free within the two-round budget.

6.6.12 Manual Validation of the Classifier

To check `ErrorClassifier`’s output against ground truth, rather than only against its own internal logic, a stratified sample of 49 records was frozen before this evaluation ran and labelled by hand. The sample has all 10 records the classifier had assigned `system_library` and all 4 it had assigned `mapping_unknown` (the two subtypes excluded from repair scope), plus 20 of the records it had assigned `missing_package` and 15 it had assigned `wrong_version`, drawn mainly from the evaluation split.

On the binary scope decision (usable versus excluded), the classifier scored 33 true positives, 2 false positives, 0 false negatives, and 14 true negatives against the manual labels. This gives an accuracy of 95.9%, a precision of 94.3%, a recall of 100%, and an F1 score of 97.1%. On the four-way subtype decision, 45 of the 49 sample rows matched the manual label exactly: a naive accuracy of 91.8%. This figure describes the classifier’s accuracy on this deliberately stratified sample, which oversamples the two rarer, excluded subtypes. It is reported as sample-level accuracy, not as a population-wide estimate. A population-wide estimate would need an independent estimate of each subtype’s true share of the full 214-row dataset; without one, it would mean using the classifier’s own output to validate itself. Table 6.9 breaks the subtype decision down per class.

Subtype	Precision	Recall	F1
<code>missing_package</code>	0.800	1.000	0.889
<code>wrong_version</code>	1.000	0.882	0.938
<code>system_library</code>	1.000	1.000	1.000
<code>mapping_unknown</code>	1.000	0.667	0.800

Table 6.9: Per-subtype precision, recall, and F1 on the 49-record manual validation sample.

Finally, the extracted failing-module name matched the manually confirmed name exactly for all 49 sample rows (100%). Extraction accuracy is reported as an exact-match rate, not as precision/recall/F1, because the space of possible module names is unbounded; a confusion matrix over it would not be meaningful.

6.6.13 Manual Validation of PyPI Distribution Resolution

A separate, small diagnostic sample of 20 import names was manually checked against the real PyPI registry. It covers every name already present in the package mapping, plus the most frequently exercised names that are not. Of the 20, 17 were names a human reviewer could confidently resolve to one specific distribution, or confirm as unresolvable. The remaining 3 were left unresolved on purpose, because no single correct distribution could be determined by hand either. Against those 17 manually resolvable cases, RAGRepairAgent’s retrieval step resolved the correct distribution in 7 (41.2%).

This is a small diagnostic sample, not a population-wide measurement of package-resolution accuracy. It is, however, consistent with the abstention results in Section 6.6.6. A package mapping that resolves fewer than half of a manually checked diagnostic sample of import names has the kind of coverage gap that would be expected to produce a high `mapping_unknown` abstention rate at full evaluation scale, which is what was observed. The two pieces of evidence are consistent with each other; this does not prove that one caused the other.

Table 3.3 gives concrete cases of this gap. `ipy2` (15 occurrences), `cana` (12), `celloracle` (12), `keras` (7), and `statsmodels` (4) are all frequent failing modules in the working dataset, and none of the five has an entry in `config/package_mapping.yaml` (Section 5.5.2). Every one of them therefore resolves to `mapping_unknown` at retrieval time, even where the correct PyPI distribution shares the import’s own name. Some of the five might resolve correctly if attempted, but the mapping table was never told about them, so RAGRepairAgent abstains before finding out.

The mapping table was not extended to cover these five names, or any other name surfaced by the final evaluation, once this pattern became visible. Extending it would have changed a repair-eligibility rule in direct response to the reserved evaluation split’s own outcome, which is exactly the kind of tuning this evaluation is designed to avoid (Section 6.6.2). The limitation is also not specific to the size of this particular table. Any manually curated mapping needs continued maintenance to track newly published or renamed packages, and it cannot resolve an import name it has never been told about, whatever its starting size. This is the same structural limitation already identified in the static knowledge-graph approaches this thesis positions itself against (Section 2.2).

6.6.14 What Limits Full Recovery

The results above separate three populations. Every proposal the language model produced was schema-valid and grounded (73 of 73). Most real repair attempts removed the error they targeted (62 of 73, Section 6.6.5). No notebook ran end to end (0 of 187, Section 6.6.4). Two factors, one acting before the model’s decision and one after it, explain the gap between the last two.

The first is package-mapping coverage, a deliberate system behaviour. Retrieval

resolved the failing import for 73 of 235 repair-agent invocations and abstained on the other 162 rather than guess an unsupported package name (Section 6.6.6). The second is a property of the notebooks themselves. Many repaired notebooks exposed further dependency errors in sequence, so removing one error frequently exposed the next (Section 6.3.1), and the two-round cap determines how far along such a chain the system can follow. The results do not show how many errors any notebook has in total, only that two rounds did not reach the end of the chain for any of them.

6.6.15 LLM Model Sensitivity Analysis

The final evaluation above uses one specific open model, Gemma-2 9B, served locally through Ollama. A natural question follows: how much of the reported behaviour comes from that specific model, and how much from the pipeline around it? This is RQ3. To answer it, the same repair layer was run over the same 187 evaluation notebooks with the language model replaced. Everything else was held fixed: the pipeline structure, the prompts and few-shot examples, the JSON schemas, the classifier, PyPI retrieval and the package mapping, grounding validation, FixApplicator, the Round-2 trigger and explanation logic, the evaluation records, and the repair policy. Both runs record the same Git commit and byte-identical component hashes in their manifests (Section 6.6.2). Only model- and provider-specific serving settings differed: the model name, the provider (local Ollama against a remote endpoint), and the calibrated Qwen output-token budget described next. The comparison is therefore controlled at the level of the pipeline; not every configuration field was identical.

Setup. The replacement model is Qwen3.6-35B-A3B-MLX-8bit, served through a remote OpenAI-compatible endpoint [using Kiste](#). A small provider abstraction lets the pipeline call either backend through the same interface; Gemma’s own configuration and code path are untouched. Qwen needed one setting change: a larger completion budget. At Gemma’s setting of 700 tokens, Qwen’s responses were often cut off before they finished. A calibration run over five development-split notebooks, never part of the evaluation split, tested budgets of 1400, 2800, and 4096 tokens. Truncation still occurred at 1400 and once at 2800, but not at 4096. The budget of 4096 was therefore the smallest of the three tested values that eliminated observed truncation; smaller untested budgets might also have worked. The Qwen run is identified as `i11-eval-qwen-round2-explanations-20260919T131359Z`. It processed all 187 records and passed all 18 integrity checks.

Explanation results. Table 6.10 sets the two models side by side on the explanation metrics of Section 6.6.3, with completion and schema validity kept apart. Qwen completed every explanation record on the first attempt: all 187 original failures and all 50 newly exposed Round-2 errors, 237 of 237, with 237 explanation calls, no retry, no timeout, no model-service failure, and no schema-invalid response. Gemma completed 210 of 237, and every one of those 210 was schema-valid. The two models therefore do not differ in the validity of what they produced; they differ in whether a response arrived at all. Gemma’s 27 missing records are timeouts and

one service error of the local, CPU-only Ollama deployment. This is a runtime-reliability difference under the evaluation environment: a large model on a remote serving endpoint against a small model on a laptop CPU. It is not evidence that one model explains better, and the human evaluation in Section 6.7 did not rate Qwen’s explanations.

Explanation metric	Gemma-2 9B	Qwen3.6-35B-A3B
Original failures completed	160/187 (85.6%)	187/187 (100%)
Round-2 errors completed	50/50 (100%)	50/50 (100%)
Combined completed	210/237 (88.6%)	237/237 (100%)
Schema-valid among completed	210/210 (100%)	237/237 (100%)
Runtime failures (timeout / service)	27 (26 / 1)	0
Explanation LLM attempts incl. retries	272	237
Records needing the retry	35	0

Table 6.10: Explanation completion and schema validity for both models, counted per encountered error (187 original failures + 50 newly exposed Round-2 errors = 237 records in each run). A runtime failure is a call that returned no processable response within one retry. Explanation calls are separate from the repair-agent calls of Table 6.11.

Repair results. Table 6.11 compares the two models on the repair metrics used for the Gemma final evaluation above, computed the same way for both. Every aggregate figure is identical.

Identical aggregates do not mean identical decisions, so the two runs were also compared notebook by notebook from their raw traces. On 174 of the 187 notebooks, every repair-relevant field matched: eligibility, retrieval status, the proposed action, install name and version, the FixApplicator outcome, the newly exposed error, the Round-2 decision, the Round-2 proposal and outcome, and the final classification. The remaining 13 notebooks differ in exactly one field: the version chosen in the Round-1 `pin_version` proposal. These are execution ids 189 to 201, all `wrong_version` failures on `numpy.VisibleDeprecationWarning` from two repositories of the same group. Gemma chose `numpy==1.26.3` in all 13; Qwen chose `numpy==1.26.4` in all 13. The retrieval evidence given to both models was identical: the same five candidate versions, including both 1.26.3 and 1.26.4, and the same compatibility constraint `<2.0.0` from the curated registry (Table 5.1). Both choices are valid, grounded candidates within that constraint. This is therefore a genuine model-selection difference, not a retrieval or pipeline difference: given the same evidence, Gemma preferred the second newest compatible release and Qwen the newest.

The difference stopped there. In all 13 notebooks the pin installed successfully under both models, and re-execution then exposed the same new error (`No module`

Repair metric	Gemma-2 9B	Qwen3.6-35B-A3B
Round-1 abstentions	126/187 (67.4%)	126/187 (67.4%)
Final-state abstentions	162/187 (86.6%)	162/187 (86.6%)
Repair-agent invocations	235 (187 R1 + 48 R2)	235 (187 R1 + 48 R2)
Repair-LLM responses	73 (61 R1 + 12 R2)	73 (61 R1 + 12 R2)
Repair-LLM calls incl. retries	73	73
Proposal schema validity	73/73 (100%)	73/73 (100%)
Grounding validity	73/73 (100%)	73/73 (100%)
Grounded-proposal coverage	73/235 (31.1%)	73/235 (31.1%)
Targeted resolution, Round 1	50/61	50/61
Targeted resolution, Round 2	12/12	12/12
Targeted resolution, overall	62/73 (84.9%)	62/73 (84.9%)
Round-2 eligible / proposals / attempts	48 / 12 / 12	48 / 12 / 12
Full notebook recovery (final)	0/187	0/187
Final outcomes: abstained / still failing / infrastructure / method / fixed	162 / 22 / 2 / 1 / 0	162 / 22 / 2 / 1 / 0

Table 6.11: Gemma-2 9B vs. Qwen3.6-35B-A3B-MLX-8bit on the same 187 evaluation notebooks, same repair pipeline, same metric definitions. Denominators: abstentions and outcomes over 187 notebooks; validity and targeted resolution over the 73 repair attempts that reached the model; coverage over the 235 repair-agent invocations. Repair-LLM calls exclude explanation calls.

named `'cxsystem2'` or `'matplotlib'`, depending on the repository). The Round-2 reclassification, explanation, eligibility decision, proposal, and outcome were identical; the targeted Round-1 error counted as resolved in both; and the final classification was the same. Model choice produced a small difference in version selection for 13 NumPy repairs, but this did not change any measured downstream repair outcome in the evaluation set.

Interpretation. Replacing Gemma-2 9B with Qwen3.6-35B-A3B-MLX-8bit did not change any aggregate repair result. The models made a different repair decision for 13 of the 187 notebooks, but these differences affected only which compatible NumPy version was selected. They did not change any downstream execution outcome.

This result reflects the role of the deterministic retrieval and grounding layers. The model can choose only from package versions that were retrieved from PyPI and passed the applicable compatibility constraints. More than one version can satisfy these constraints, which is where the two models differed in this evaluation.

The two runs also had the same Round-1 abstention count of 126 out of 187. These abstentions occurred before the repair model was called because the package mapping could not resolve the failing import. Both models also resolved the targeted dependency error in 62 of the 73 FixApplicator attempts, while neither produced a fully recovered notebook within the two-round budget.

For these two model configurations, the main limits on repair coverage were therefore package-mapping coverage and notebooks that exposed further dependency errors after a repair. The results do not establish that one model is generally better than the other. Qwen completed all explanation calls, whereas the Gemma run had 27 runtime failures, but the models were served through different environments and with different provider-specific settings. This difference therefore cannot be attributed to the language model alone.

6.7 Human Evaluation of Explanation Quality

Section 6.6.3 showed that every explanation Gemma-2 9B completed was schema-valid. Schema validity is a structural check. It does not show whether an explanation is understandable, detailed enough, or useful to a reader, which is what objective O1 and the plain-language requirement NFR2 (Section 4.3.2) ask for. Only a human reader can judge that. This section therefore reports a small human study of perceived explanation quality.

The study rated a frozen set of 12 Gemma-2 9B explanations of *original* failures from the evaluation split. It is separate from the automated completion and schema-validity measurements of Section 6.6.3, and it did not rate any Round-2 explanation of a newly exposed error or any Qwen explanation. The design was fixed, and the survey administered, before any response was analysed.

6.7.1 Questionnaire

The questionnaire adapts the Explanation Satisfaction Scale of Hoffman et al. [9], a seven-item Likert scale that records a user’s own judgment of an explanation after reading it. Six of the seven items are kept, each adapted only in its domain reference; for example, “From the explanation, I know how the [tool] works” becomes “From this explanation, I understand why the notebook failed.” The adapted items are listed in Table 6.12. The seventh original item, whether the explanation “tells me how to use” the system, is excluded: LLMExplainer does not suggest what to do about a failure, because deriving a repair is RAGRepairAgent’s separate responsibility (Section 5.4.1).

The item closest to the original scale’s accuracy question, “This explanation seems correct, based on the error shown” (Q6), is reported throughout as *perceived correctness*. It records how correct the explanation appears to the reader, not whether it matches the notebook’s true root cause. The study does not measure objective correctness.

All six items use the same 1–5 scale (1 = strongly disagree, 2 = disagree, 3 = neither agree nor disagree, 4 = agree, 5 = strongly agree) after every example. The scale fits this thesis because it measures a reader’s experience of a single, static, plain-language explanation. The System Causability Scale of Holzinger et al. [10] was not adopted, because its items assume an interactive, expert-facing explanation interface.

6.7.2 Study Design and Participants

Example pool. Twelve explanations were drawn from the frozen 187-row evaluation split. None come from the 13-row development split (Section 5.4.5). Five evaluation-split records had previously been used for explanation-only smoke checking of the implemented explainer, after the prompt-design sample had been fixed and the design-time prompt checks had been carried out on the development split alone (Section 3.2); that check ran under the same explanation prompt version as the final evaluation records, and all five returned schema-valid output on the first attempt. Because they had nevertheless been inspected during development, they were conservatively excluded from the human-study sampling pool. Two further `missing_package` records were unavailable for sampling because their explanation call timed out in the run the pool was drawn from, so no schema-valid explanation existed to show a participant. The eligible sampling population is therefore 180 of the 187 evaluation records – 162 `missing_package` and 18 `wrong_version`, all seven excluded records being `missing_package` ones. All twelve examples were generated with Gemma-2 9B under the final explainer prompt, schema, and configuration, and the pool was frozen before any participant response was collected. Sampling was stratified by subtype, with the number of examples per subtype bounded by how many distinct error signatures the eligible population contains, and it did not consult repair outcomes or any other evaluation result. The eligible `wrong_version`

population has only three distinct (failing module, error message) signatures across its 18 rows, most of which share one of two repeated NumPy or SciPy messages; all three signatures are included once. The remaining nine positions come from `missing_package`, which offers 41 distinct failing modules across 162 eligible rows, and each of the nine has a different failing module. The pool therefore holds **9 missing_package** and **3 wrong_version** explanations, not a balanced 6/6 split.

Assignment. Each participant rated 6 of the 12 explanations. The 12 were arranged into four fixed survey variants so that every explanation appears in exactly two variants, which balances exposure without a shared anchor example. Participants were assigned to variants in round-robin order as they were recruited, and the order of the six explanations within a variant was randomised.

Procedure. For each explanation, a participant saw only the original error type and message together with the explanation’s `summary`, `root_cause`, `evidence`, and `limitations` fields, rendered as plain text and unedited. The repository name, the source code, the model’s `explanation_confidence`, and every outcome of the final evaluation, such as whether the notebook was repaired or whether a Round-2 attempt fired, were withheld, because they could bias the judgment of the explanation itself. The same six items followed every explanation in the same order. Two optional free-text questions at the end asked what was unclear or missing and what would make the explanations more helpful; these are supplementary feedback, not part of the scale.

Participants. The only requirement was general programming experience. Familiarity with Python or Jupyter was not required, and no machine-learning expertise was needed, because the explanations are written for exactly this audience (NFR2). A short, anonymous background section recorded self-reported programming experience, Python and Jupyter familiarity, and prior experience with package errors. The target was a convenience sample of 15–20 participants, sized for a descriptive study.

The survey ran as four LimeSurvey instances, one per variant, between 2026-09-15 and 2026-09-17, and produced 18 raw responses. Three were excluded before analysis: one respondent declined consent, and two left after the consent page with no rating filled in. The remaining **15 responses are complete**, with all 36 rating cells (6 explanations $\times$ 6 items) filled in; variants A–D contributed 3, 4, 4, and 4 complete participants. This gives **90 explanation evaluations** and **540 individual ratings**. Of the 15 participants, 10 described their programming experience as intermediate, 4 as advanced or professional, and 1 as beginner. Seven use Python regularly, seven had used it a little, and one had never used it. Eleven had used Jupyter Notebooks a few times, three never, and one regularly. All had encountered a missing-package or package-version error, 11 occasionally and 4 frequently.

6.7.3 Analysis and Limitations

For each of the six items, the analysis reports the number of ratings, the median, the interquartile range (IQR), the full 1–5 frequency distribution, and the percentage of ratings of 4 or 5 (%4–5). These statistics are computed overall, by subtype, and per explanation, exactly as planned before any response was collected. No inferential test is computed. Every statistic is produced by one analysis script directly from the raw survey exports, using functions that were written and unit-tested while the study was being designed.

Cronbach’s alpha is reported as a supplementary internal-consistency statistic only. It describes whether the six items move together in the collected data. It is not a measure of agreement between participants, it does not show that the six items measure a single construct, and it is not used to combine the items into one composite score. Item-level results remain the primary output.

Five limitations bound what the results can show. First, the study is a convenience sample of 15 participants, each rating 6 of 12 explanations. The 90 evaluations are clustered within 15 people, not 90 independent judgments, and the results describe this group and this pool, not programmers or explanations in general. Second, participants judged a rendered error message and explanation, not the original notebook and its code. Third, the ratings measure perceived quality; Q6 in particular records whether an explanation struck a reader as correct, not whether it matches the notebook’s true root cause. Fourth, no baseline or alternative explanation was shown, so the results describe this component in isolation. Fifth, the subtype comparison rests on 9 distinct `missing_package` explanations against 3 distinct `wrong_version` explanations, with 67 and 23 evaluations respectively, and is descriptive only. In addition, the study covers original-failure Gemma-2 9B explanations alone. The 50 Round-2 explanations and the 237 Qwen explanations of Sections 6.6.3 and 6.6.15 were evaluated automatically for completion and schema validity only, and no participant rated any of them. The broader threats to validity are collected in Section 6.8.

6.7.4 Results

Overall ratings. Table 6.12 reports the six items over all 90 explanation evaluations. Every item has a median of 4 or 5, and every item’s %4–5 is at least 84%. Pooling all 540 ratings gives a median of 5 and an IQR of 1, with 20 Strongly Disagree, 17 Disagree, 23 Neither agree nor disagree, 185 Agree, and 295 Strongly Agree responses: **88.9%** of all ratings were Agree or Strongly Agree. Figure 6.3 shows the six item-level distributions side by side.

Internal consistency. Cronbach’s alpha across the six items, computed over the 90 evaluations, is **0.928**. The six adapted items therefore moved together consistently in this dataset. As stated in Section 6.7.3, this value is not an inter-rater agreement statistic, and the item-level table remains the primary result.

Item	n	Median	IQR	%4–5
Q1. I understand why the notebook failed.	90	5	1	92.2%
Q2. This explanation is satisfying.	90	4	1	84.4%
Q3. This explanation has enough detail.	90	5	1	91.1%
Q4. This explanation seems complete.	90	4	1	91.1%
Q5. This explanation is useful for understanding what went wrong.	90	5	1	87.8%
Q6. This explanation seems correct, based on the error shown (<i>perceived correctness</i> , Section 6.7.1).	90	5	1	86.7%

Table 6.12: Results for the six adapted Explanation-Satisfaction-Scale items, over all 90 explanation evaluations (15 participants $\times$ 6 rated explanations each). IQR is the interquartile range on the 1–5 scale; %4–5 is the share of that item’s 90 ratings that were Agree or Strongly Agree.

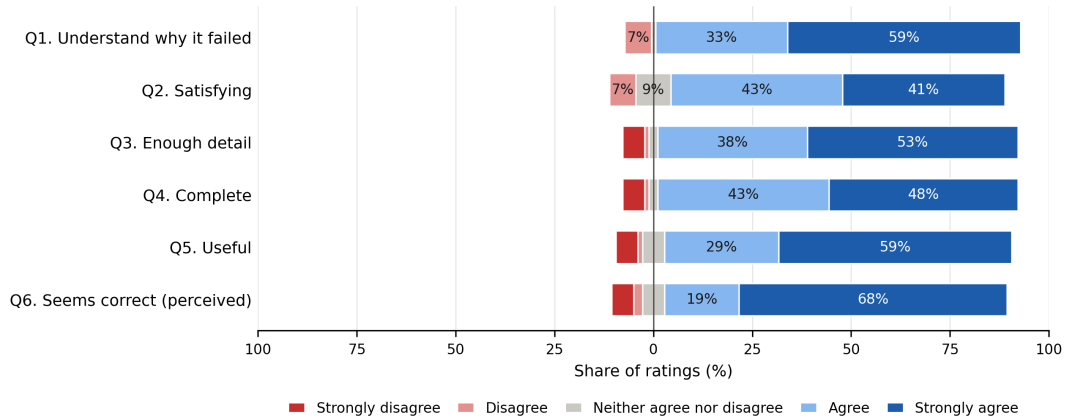


Figure 6.3: Distribution of participant ratings for the six explanation-quality items, over all 90 explanation evaluations.

Results by subtype and by explanation. Table 6.13 compares the two subtypes descriptively. Both sit close together on the positive side of the scale, and neither is rated clearly better than the other; the unequal sizes follow from the pool composition (Section 6.7.2). At the level of individual explanations, every one of the 12 had a median of 5, and the per-explanation %4–5 ranged from 78.6% to 100%. Exposure was close to balanced: six explanations were rated by 7 participants and six by 8, because the four variants collected 3, 4, 4, and 4 complete participants and each explanation appears in two variants.

Subtype	Evaluations	Ratings	Median	IQR	%4–5
missing_package	67	402	5	1	89.6%
wrong_version	23	138	5	1	87.0%

Table 6.13: Descriptive comparison of the two error subtypes, pooling all six items. “Evaluations” counts participant–explanation pairs ($67 + 23 = 90$) and “Ratings” counts those times six items ($402 + 138 = 540$).

Qualitative feedback. Five of the 15 participants answered at least one of the two free-text questions, giving six answers in total. Two themes recur. Several comments were unqualified positive assessments, such as “Nothing. Everything is clear” and that the explanations “were very helpful and also provide enough information on understanding exactly what is working wrong.” Several others asked for more causal or technical detail: one participant asked for “more detailed explanation about the error message”, and another wrote that some explanations “repeated the error message and could have explained the cause a little more” and suggested adding “specific details, such as the correct package name or what kind of version incompatibility might be involved.” No broader thematic analysis is claimed from six comments.

Interpretation. Within the scope of this study, the results answer the reader-facing half of RQ1 positively. Participants, whose self-reported Python and Jupyter experience ranged from none to regular use, rated the explanations positively on every item. Understanding (Q1) scored highest at 92.2%, followed by detail and completeness (Q3 and Q4, 91.1%), usefulness (Q5, 87.8%), and perceived correctness (Q6, 86.7%). Satisfaction (Q2) was the weakest item at 84.4%. It is still clearly positive, but it is the item to which the free-text requests for more causal, package, and version detail most plausibly apply. The spread across items is about eight percentage points, so this ordering describes emphasis within a generally positive set of ratings rather than a sharp separation. The limitations of Section 6.7.3 apply: the ratings are perceived quality from a 15-participant convenience sample, with no alternative explanation method for comparison. Within those limits, the plain-language explanations LLMExplainer produced were understandable, useful, and perceived as

correct by the readers in this study, with satisfaction the weakest item and the one the free-text feedback pointed at.

6.8 Threats to Validity

This section collects the limitations that affect how the evaluation as a whole should be read. Each point is stated once, with a reference to the section that discusses it in full.

6.8.1 Internal Validity

- **LLM sampling is not deterministic.** Both models run at a non-zero temperature (Sections 5.5.2 and 6.6.15), so a repeated prompt need not reproduce the same output. The Round-1-versus-Round-2 comparison reads both conditions from one run, so sampling variation cannot appear as a Round-2 effect (Section 6.6.1). Each model’s run in the Gemma/Qwen comparison is likewise a single run.
- **The two models were served differently.** Gemma ran on local, CPU-only Ollama; Qwen ran on a remote OpenAI-compatible endpoint with a larger output-token budget (Section 6.6.15). Serving hardware, latency, and queuing are therefore confounded with model choice. Gemma’s 27 explanation runtime failures against Qwen’s none are reported as a property of the serving setup, not as a model-quality difference. The 13 notebooks with a different Round-1 version choice are the opposite case: the retrieval evidence was identical, so that difference is attributed to the models.
- **Repositories were not commit-pinned.** FixApplicator could not reach its configured metadata database in the final evaluation environment, so every repository was cloned from its current default branch rather than checked out at the originally failing commit (Section 6.6.2). A repository changed by its authors in the meantime could behave differently on re-execution. The same fallback policy applied to both model runs, which reduces this threat to the model comparison, although a repository could in principle have changed between the two run dates. The absolute repair figures describe the repositories as they were on the run dates.
- **Infrastructure failures are kept apart from repair-method failures.** A clone, build, or container failure (Section 5.6.3) is recorded as `apply_error` and counted as an infrastructure failure only when there is concrete evidence of an external fault; an ambiguous case stays a method failure (Section 6.6.8). Infrastructure failures are labelled separately from method failures so that their external cause remains visible. They nevertheless remain unresolved cases in

the 73-attempt targeted-resolution denominator, so this classification does not inflate the reported 62/73 resolution rate.

- **The evaluation split was reserved and the configuration frozen.** The 187-row split was reserved at dataset-preparation time (Section 3.2). No prompt, mapping entry, or repair rule was designed against it, and the package mapping was not extended after the abstention pattern became visible (Section 6.6.13). Both final runs executed from one recorded commit under a hash-checked configuration (Section 6.6.2).

6.8.2 Construct Validity

RQ2 asks whether the targeted dependency error is resolved and whether the notebook recovers; RQ1 asks whether the explanations are understandable to a reader. The metrics of this chapter measure these concepts with the following qualifications.

- **Targeted-error resolution is not full notebook recovery.** An attempt can remove exactly the dependency error it targeted and still leave the notebook failing on another (Sections 6.6.5 and 6.6.10). The two figures are therefore reported side by side, together with proposal validity and grounding validity, rather than combined into one success score (Figure 6.2); a single score would hide the gap on which the central finding depends.
- **A schema-valid, grounded proposal is not necessarily a correct repair.** Schema and grounding validity (Section 6.6.7) show only that the model’s output has the right shape and cites evidence RAGRepairAgent retrieved. Whether the fix works is decided by FixApplicator’s re-execution alone.
- **Explanation completion and schema validity are runtime and structural measures.** A timed-out explanation call is a runtime failure with no output to judge, not a malformed response, and the two are kept apart (Section 6.6.3). Schema validity of a completed response says nothing about whether a reader finds it understandable or useful, which is what O1 and NFR2 ask for.
- **The human evaluation measures perceived quality.** It answers the reader-facing question for a frozen pool of Gemma original-failure explanations (Section 6.7). Its correctness item records whether an explanation struck a reader as correct, not whether it matches the true root cause (Section 6.7.3).

6.8.3 External Validity

- **One dataset, one failure class.** Every record comes from the Docker-based FAIR Jupyter pipeline over PubMed-Central-linked notebooks (Section 3.2.1), and only `ModuleNotFoundError` and `ImportError` failures are

considered (Section 3.2.2). The results say nothing about other notebook collections or about logic or data errors.

- **Repair is pip/PyPI-based.** Every automated fix is a `pip install` or a version pin for a `missing_package` or `wrong_version` failure (Section 3.2.3). `system_library` failures, which need system-level installation, are outside repair scope by design (Figure 3.1), and an import the package mapping cannot resolve is abstained on rather than repaired (Section 6.6.6). Nothing here shows how the approach would fare in a Conda-managed environment or another package ecosystem.
- **Two open LLM configurations were evaluated.** Gemma-2 9B and Qwen3.6-35B-A3B-MLX-8bit are the only models run through the full evaluation (Section 6.6.15). Their agreement describes these two models under this grounded design; other open or proprietary models could behave differently.
- **The human study is small and narrow in scope.** It is a convenience sample of 15 participants whose Python and Jupyter experience ranged from none to regular use (Section 6.7.4), and it rated only the frozen pool of Gemma original-failure explanations, not Round-2 or Qwen explanations (Section 6.7.3). Its ratings describe how this group judged this pool, not how programmers in general would.

6.8.4 Conclusion Validity

- **Denominators are small.** The evaluation split has 187 notebooks, and only 73 of the 235 repair-agent invocations reached the language model and FixApplicator (Section 6.6.6). Every proposal-quality and targeted-resolution figure is therefore a proportion over 73 attempts, and subgroup figures such as the 12 Round-2 attempts or the 18 `wrong_version` rows rest on even fewer cases. A different 187-row sample could give different exact percentages.
- **The Gemma/Qwen comparison is descriptive.** It compares two single runs of the same 187 notebooks (Section 6.6.15) without a significance test. The identical aggregates are consistent with the grounding layer strongly constraining the available repair choices, but they are not evidence that either model is generally better, and the two need not agree where a candidate set leaves more choice.
- **The two-round cap bounds conclusions about longer repair chains.** Section 6.6.10 shows a notebook that exposed a third dependency error after two successful targeted repairs. The evaluation shows that two rounds were not enough for such notebooks; it cannot say what a third round would have found, or whether further errors lay behind the third (Section 5.7.3).

- **The human-study statistics are descriptive.** The 90 explanation evaluations come from 15 participants who each rated 6 of 12 examples, so the per-item percentages and Cronbach’s alpha are computed over ratings clustered within 15 people, not 90 independent raters. The subtype comparison rests on 9 and 3 distinct examples (Section 6.7.2), and no inferential claim is made.

None of these points overturns the results reported in this chapter. They mark the boundary of what the results can claim, so that Chapter 8 states no more than this evaluation supports.

6.9 Chapter Summary

This chapter reported several kinds of evidence, in increasing order of scale. The automated test suite (Section 6.1, 711 tests) shows that every implemented component behaves as designed against controlled, mocked inputs. The component-by-component pilots (Sections 6.2 to 6.4) and the small orchestrator pilot (Section 6.5) showed that each component, and the orchestrator that connects them, also behaves correctly against its real external system: a real Ollama model, the live PyPI registry, real Docker execution, real GitHub repositories, and the real Morph-KGC mapping engine. These pilots surfaced concrete problems that only a real pilot could have found: a timeout set too low, two missing package mappings, and three environment-specific bugs.

Section 6.6 then reported the final evaluation: the final frozen implementation, run with Gemma-2 9B over the 187-row evaluation split with a two-round budget. For RQ1, Gemma completed 160 of the 187 original-failure explanations and all 50 explanations of newly exposed Round-2 errors; every one of the 210 completed responses was schema-valid, and the 27 missing records were timeouts and one service error of the local model service. The human study (Section 6.7) adds the reader’s view for the original-failure explanations: 15 participants, 540 ratings, 88.9% Agree or Strongly Agree, every item with a majority-agree median. For RQ2, the two levels of the repair result stay apart: 62 of the 73 real repair attempts (84.9%) removed the error they targeted, while 0 of 187 notebooks ran end to end. Every one of the 73 proposals the model produced was schema-valid and grounded, but only 73 of 235 repair-agent invocations reached the model at all, because a `mapping_unknown` package-resolution gap accounted for all 162 abstentions. Many repaired notebooks also exposed further dependency errors in sequence, and the two-round cap does not follow such a chain to its end (Sections 6.6.10 and 6.6.14). Two rows ended in an infrastructure failure and one in a method failure; these categories are reported separately in the final-outcome breakdown, while the infrastructure cases remain unresolved in the 73-attempt targeted-resolution denominator.

For RQ3, Section 6.6.15 repeated the evaluation with Qwen3.6-35B-A3B-MLX-8bit in place of Gemma-2 9B, with the pipeline held fixed and only model- and

provider-specific serving settings changed. Every aggregate repair figure was identical. A notebook-by-notebook comparison found one structured difference, on 13 notebooks: both models pinned `numpy` from the same grounded candidate list, Gemma to 1.26.3 and Qwen to 1.26.4, with no effect on installation, on the errors exposed next, on Round 2, or on any final classification. Qwen completed all 237 explanation records where Gemma completed 210. Because the model and its serving environment changed together, that difference is reported as an observation about the evaluation setup, not as evidence about model architecture.

The final Gemma run’s `repair_attempts` table, one row per repair-agent invocation of the full split, 235 in all, was also exported and run through the existing RML mapping unchanged (Section 6.4.3). This produced 3,478 triples, one repair-attempt resource per row, each linked to the notebook resource that the FAIR Jupyter knowledge graph uses for the repaired notebook, and all 24 validation checks passed in union with the existing notebook graph. The benchmark table (O4) and the materialised repair graph (O6) therefore describe the same 235 repair attempts; the repair graph has not been loaded into the public FAIR Jupyter triple store.

Two things this chapter does not show are worth naming. It does not report a repair success rate for the whole 214-row dataset, because 14 rows fall outside pip-only repair scope by design (Chapter 3). And it does not compare the system against a baseline: a comparison against a non-grounded repair agent remains future work (Chapter 7).

Section 6.8 states what this evaluation does and does not license concluding, across internal, construct, external, and conclusion validity, so the summary above is read with its boundaries attached.

7 Future Work

The evaluation in Chapter 6 answers the research questions of this thesis, but also identifies several directions for further work. These concern the use of the knowledge-graph enrichment, wider repair coverage, comparison with a repair baseline, and a broader human evaluation.

7.1 Building on the Knowledge-Graph Enrichment

Objectives O4 and O6 are both met for the final evaluation, by two distinct artefacts that describe the same repair attempts. The `repair_attempts` table of the final Gemma run holds one row per repair-agent invocation, 235 in all (187 Round-1 and 48 Round-2 rows), with the full explanation and PyPI evidence of every invocation. That table, with its CSV export, is the structured benchmark dataset of O4. The existing RML mapping, run over that export, materialised the same 235 attempts as 3,478 triples, each linked to the notebook resource of the FAIR Jupyter knowledge graph that identifies the repaired notebook, and the result passed every validation check in union with the existing notebook graph (Section 6.4.3). That repair graph is the enrichment of O6 at evaluation scale; it has not yet been loaded into the public FAIR Jupyter triple store.

The first extension is deployment. The enriched triples exist as files next to the run that produced them. Loading them into the FAIR Jupyter triple store, beside the graph the upstream pipeline already publishes, would make repair history queryable at the same public SPARQL endpoint as the execution records it extends. Because the repair mapping is one more `.rml.ttl` file of the shape the upstream build script already iterates over, this requires running that script with the mapping and the export in place, not new tooling. Materialising a second run next to the first, such as the Qwen run of Section 6.6.15, would first require the repair-attempt IRI template to carry the run identifier as well as the row id, because each run's table numbers its rows from one.

The second is richer use of the graph. The checks of Section 6.4.3 ask structural questions: does every attempt link to a notebook, and does every executed attempt carry an outcome. The same graph can also answer the questions for which the summary tables of Chapter 6 were computed separately: which notebooks received a Round-2 attempt and after which Round-1 outcome, which failing modules abstained, and how the attempts of one run compare with those of another once both are loaded. This is possible because the round, the model, and the run identifier are recorded on every attempt. Written as SPARQL queries over the union of the

upstream graph and the repair triples, these questions could be re-asked without returning to the SQLite table, and the repair history could be combined with the execution, module, and repository facts the graph already holds about the same notebooks.

The third is a richer vocabulary. The mapping records the proposed package, version, and PyPI evidence as literals on the repair attempt. Modelling the installed distribution and its release as resources of their own, and linking each repair attempt to them, would connect repair provenance to package and version entities that other datasets could share. A query could then ask, for instance, which notebooks were repaired by pinning the same NumPy release. Repair actions this thesis deferred, such as replacing an import (Section 7.2), and any further repair rounds would need no change to the mapping: its existing predicates already carry the action value and the round number.

7.2 Widening Repair Coverage

The single largest limit on this system’s repair coverage is the static import-to-distribution mapping. It left 162 of 235 repair-agent invocations abstaining before the language model was reached (Section 6.6.6), and the model-sensitivity experiment showed this limit is shared across both models tested (Section 6.6.15). Extending the mapping table is the obvious first step, but it does not remove the underlying problem: any hand-curated table needs continued maintenance and cannot resolve a name it has never been told about. A more durable approach would resolve import names against a maintained external source, and fall back to abstention only when that source genuinely cannot answer.

Two further limits are narrower but real. The compatibility-evidence registry that supports `wrong_version` repairs holds three hand-checked entries (Table 5.1), which is enough for this dataset and no more. Deriving such evidence automatically from release notes or from a project’s own API history would generalise it. The repair scope is pip-only by design (Section 3.2.3), which leaves the 10 `system_library` records unrepairable and rules out fixing an API rename inside a notebook’s own source. A related extension concerns the repository’s declared dependency files, which the pipeline currently uses only to rebuild the baseline environment (Section 5.5.5). They could additionally be shown to the repair model as non-authoritative context, for instance to flag that a proposed pin conflicts with a version the repository declares, while PyPI evidence remains the only source from which a version may be drawn.

A third question the evaluation raised but could not answer is how much a larger repair budget would help. Section 6.6.10 shows a notebook that exposed a third dependency error after two successful targeted repairs. Whether a third or fourth round would have recovered it, or whether notebooks of this kind have a long tail of dependency problems, is unmeasured: no run beyond two rounds was attempted.

Two further points concern how the final runs were executed rather than what they

repaired. First, the commit-pinning mechanism of FixApplicator was not in effect in the final evaluation environment (Section 6.6.2), so the evaluated repositories were cloned at their default-branch state. A rerun with the metadata database reachable from the execution environment would test each repository at the commit that originally failed, and would show whether any of the 187 outcomes depends on changes the repository’s authors made since. Second, the Gemma explainer lost 27 of 237 explanation records to timeouts of the local, CPU-only model service (Section 6.6.3). A larger timeout, or serving the same model on faster hardware, would separate how much of that gap is the machine and how much is the model; the present results cannot.

7.3 Baseline Comparison

This thesis reports how its own repair layer performs, not how it performs relative to an alternative. The most informative comparison would be against the same agent with PyPI grounding removed, so that the contribution of grounding could be isolated from the contribution of the model. Such a comparison could reuse the existing evaluation design without changing the dataset or the metric definitions of Chapter 6. The ungrounded baseline has not been built and the comparison has not been run, so this thesis makes no claim about grounding’s measured benefit.

7.4 Extending the Human Evaluation

The human evaluation reported in Section 6.7.4 is a convenience sample of 15 participants, each rating six explanations from a frozen pool of 12, all of them Gemma explanations of original failures. It supports a descriptive picture only (Section 6.7.3). The Round-2 explanations of newly exposed errors and the Qwen explanations were evaluated automatically for completion and schema validity only, and rating them with the same instrument is the most direct extension. Three further extensions would each answer a question the present study cannot.

With an appropriate sampling and analysis design, a larger sample could support inferential comparisons rather than descriptive claims. A replication with expert raters, judging explanations against each notebook’s true root cause, would measure objective correctness, which this study does not: Q6 captures only whether an explanation struck a reader as correct. Showing participants an alternative explanation, whether from a different model or from a non-LLM template, would place these results on a comparative scale.

The free-text feedback points to one concrete improvement worth testing directly. Several participants asked for more about the likely cause and less restatement of the error message (Section 6.7.4). Whether a revised prompt raises the satisfaction item without harming the others could be tested with the existing instrument and example pool.

8 Conclusion

This thesis set out to close a narrow gap. The FAIR Jupyter reproducibility pipeline can detect that a notebook failed because of a missing or incompatible Python package, but it does not explain that failure and does not try to repair it. Chapter 2 showed that the pieces needed to close this gap already existed separately: failure detection, automated dependency inference, and LLM-based program repair. Among the systems reviewed, none combined all three inside a reproducibility pipeline, grounded in live package data and paired with a plain-language explanation.

The repair layer built here does that. A deterministic classifier decides scope and subtype before any language model is involved. An explanation component turns a raw error message into a plain-language, schema-validated account of what went wrong. Its scope is deliberately wider than the repair scope: all 214 dependency-error records are eligible for an explanation, not only the 200 that fall inside the pip-only repair scope. A repair component proposes a fix using an open, locally deployable model, checked field by field against live PyPI metadata and a small curated compatibility registry, so a suggested package name or version is never a guess. A fix-application component applies that proposal inside a Docker environment rebuilt from the original recipe, re-runs the whole notebook, and classifies the result. When that re-execution exposes a new error, the layer classifies and explains it as well, and repairs it in one bounded second round if it is repairable; explanation and repair stay independent of each other throughout.

A result-logging component records one row per repair-decision entry, including abstentions, in a SQLite benchmark table; in the final evaluation every one of those entries was a repair-agent invocation. Exported to CSV and run through the existing RML mapping, the final run’s 235 rows became 3,478 RDF triples, in which each repair attempt is linked to the notebook resource that the FAIR Jupyter knowledge graph uses for the repaired notebook, validated in union with the existing notebook graph. The implementation is covered by 711 automated tests and was checked against real PyPI, a real Ollama model, real Docker execution, real GitHub repositories, and the real Morph-KGC mapping engine. The quantitative evaluation used the 187 records reserved as the evaluation split; the 13 development records were used only for prompt design and pilots.

The three research questions of Section 1.3.2 are answered below, each from the final frozen implementation run over that split.

RQ1: Explaining residual failures

Gemma-2 9B completed 160 of the 187 original-failure explanations and all 50 explanations of errors newly exposed by a first repair; every one of the 210 completed responses satisfied the explanation schema. The 27 missing records were timeouts and one service error of the local, CPU-only model deployment, not malformed output. Fifteen participants, with self-reported Python and Jupyter experience ranging from none to regular use, each rated six of twelve frozen original-failure explanations against a six-item questionnaire adapted from Hoffman et al. [9]. Of the 540 ratings, 88.9% were Agree or Strongly Agree, and every item reached a majority-agree median. Understanding scored highest; satisfaction scored lowest, matching free-text feedback that asked for more about the likely cause and less restatement of the error message. The answer to RQ1 is therefore positive within its scope: an open model produced structured explanations for every failure it was able to answer, and the readers in this study found them understandable, useful, and plausible. The ratings say nothing about objective correctness, about a comparison with an alternative method, or about the Round-2 and Qwen explanations, which no participant rated.

RQ2: Resolving the targeted error and recovering the notebook

The two halves of RQ2 have different answers, and they must be read together. Every one of the 73 repair proposals the language model produced was schema-valid and grounded in retrieved PyPI evidence, and 62 of the 73 FixApplicator attempts (84.9%) removed the specific dependency error they targeted. No notebook, however, re-executed cleanly from start to finish within the two-round budget: full recovery is 0 of 187. The first figure is per attempt, the second per notebook, and they are reported side by side throughout rather than combined into one success rate.

Two factors explain the distance between them. The first is a deliberate system behaviour: the repair agent’s static import-to-distribution mapping could not resolve the failing import in 162 of its 235 invocations, and it abstained rather than guess a package name it could not ground. The second is a property of the evaluated notebooks: many of them exposed further dependency errors in sequence, so repairing one import frequently exposed a second that had been invisible behind it; Section 6.6.10 traces a notebook that exposed a third dependency error after two successful targeted repairs. The two-round cap then determines how far along such a chain the system can follow. The answer to RQ2 is that the bounded pipeline resolves the targeted error in most of the attempts it makes, that it makes an attempt for fewer than a third of its invocations, and that resolving one dependency error did not restore complete execution within the two-round budget in this evaluation.

RQ3: Sensitivity to the choice of model

The same 187-notebook evaluation was repeated with Gemma-2 9B replaced by Qwen3.6-35B-A3B-MLX-8bit. The pipeline, prompts, schemas, retrieval, validation, evaluation records, and repair policy were held fixed; the model, its provider, and the calibrated Qwen output-token budget differed. Every aggregate repair figure was identical: the same 126 Round-1 abstentions, the same 73 grounded proposals, the same 62 targeted errors resolved, and zero full recoveries for both. The two runs were not identical in every decision. On 13 notebooks, all failing on the same removed NumPy symbol, both models pinned NumPy from the same grounded candidate list, Gemma to 1.26.3 and Qwen to 1.26.4. Both versions satisfied the retrieved compatibility constraint, both installed, and the notebooks then failed on the same next error under both models. Model choice produced a small difference in version selection for these 13 repairs, but it did not change any measured downstream repair outcome. Qwen also completed all 237 explanation records under its serving environment, whereas the local Gemma/Ollama run lost 27 to runtime timeouts. Because the model and the serving setup changed together, this observation does not isolate the model as the cause. The answer to RQ3 is that, under this pipeline, structured repair decisions were nearly insensitive to the choice between these two models and the measured repair outcomes were unaffected, while explanation completion differed between the two runs. The retrieval and grounding layer bounds a valid proposal to the resolved distribution and a small, PyPI-confirmed candidate set, so what remains for a model to decide is a choice among a few compatible versions. Neither model is shown to be generally better than the other, and other models could behave differently.

Closing remark

Taken together, the answer to the question this thesis opened **with is a qualified yes**. A dependency-related notebook failure can be explained in language a non-specialist reader finds understandable. The results show that specific dependency errors can often be repaired automatically with grounded, deterministically validated proposals from an open model, even though complete notebook recovery remained out of reach within the two-round budget. What this system cannot yet do is carry a real research notebook all the way back to a clean run, because many notebooks in the evaluation exposed additional dependency errors after an initial repair and because resolving an import name to a package remains the binding constraint. Two limits of the evaluation itself also stand: the repositories were re-executed at their default-branch state rather than at the originally failing commit, and the local model service did not answer every explanation request in time. Chapter 7 sets out what would have to change for that, and Section 6.8 states the boundaries within which every result above should be read.

Bibliography

- [1] Barker, M., Chue Hong, N.P., Katz, D.S., Lamprecht, A.L., Martinez-Ortiz, C., Psomopoulos, F., et al.: Introducing the FAIR principles for research software. *Scientific Data* 9, 622 (2022)
- [2] Bartlett, A., Liem, C., Panichella, A.: The last dependency crusade: Solving Python dependency conflicts with LLMs. In: 2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW). pp. 66–73. IEEE (2025)
- [3] Cheng, W., Hu, W., Ma, X.: Revisiting knowledge-based inference of Python runtime environments: A realistic and adaptive approach. *IEEE Transactions on Software Engineering* 50(2), 258–279 (2024)
- [4] Cheng, W., Zhu, X., Hu, W.: Conflict-aware inference of Python compatible runtime environments with domain knowledge graph. In: Proceedings of the 44th International Conference on Software Engineering (ICSE) (2022)
- [5] Dinh, T., et al.: The impact of prompt programming on function-level code generation. *arXiv preprint arXiv:2412.20545* (2024)
- [6] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Guo, Q., Wang, M., Wang, H.: Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2024)
- [7] Groto, K., Borzilov, A., Krivobok, M., Bryksin, T., Zharov, Y.: Debug smarter, not harder: AI agents for error resolution in computational notebooks. *arXiv preprint arXiv:2410.14393* (2024)
- [8] Groto, K., Titov, S., Zharov, Y., Bryksin, T.: Untangling knots: Leveraging LLM for error resolution in computational notebooks. *arXiv preprint arXiv:2405.01559* (2024)
- [9] Hoffman, R.R., Mueller, S.T., Klein, G., Litman, J.: Measures for explainable AI: Explanation goodness, user satisfaction, mental models, curiosity, trust, and human-AI performance. *Frontiers in Computer Science* 5, 1096257 (2023)
- [10] Holzinger, A., Carrington, A., Müller, H.: Measuring the quality of explanations: The system causability scale (SCS). comparing human and machine explanations. *KI - Künstliche Intelligenz* 34(2), 193–198 (2020)

BIBLIOGRAPHY

- [11] Horton, E., Parnin, C.: DockerizeMe: Automatic inference of environment dependencies for Python code snippets. In: Proceedings of the 41st International Conference on Software Engineering (ICSE) (2019)
- [12] Jupyter AI Contributors: Jupyter AI: A generative AI extension for Jupyter-Lab. <https://github.com/jupyterlab/jupyter-ai> (2023)
- [13] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems (NeurIPS) (2020)
- [14] Pimentel, J.F., Murta, L., Braganholo, V., Freire, J.: A large-scale study about quality and reproducibility of Jupyter notebooks. In: Proceedings of the 16th IEEE/ACM International Conference on Mining Software Repositories (MSR) (2019)
- [15] Reyes, F., Mahmoud, M., Bono, F., Nadi, S., Baudry, B., Monperrus, M.: Byam: Fixing breaking dependency updates with large language models. arXiv preprint arXiv:2505.07522 (2025)
- [16] Samuel, S., König-Ries, B.: ReproduceMeGit: A visualization tool for analyzing reproducibility of Jupyter notebooks. In: Provenance and Annotation of Data and Processes (IPAW 2021). pp. 201–206. Lecture Notes in Computer Science, Springer (2021)
- [17] Samuel, S., Mietchen, D.: Dataset of a study of computational reproducibility of Jupyter notebooks from biomedical publications. Zenodo dataset, <https://doi.org/10.5281/zenodo.8226725> (2023)
- [18] Samuel, S., Mietchen, D.: Computational reproducibility of Jupyter notebooks from biomedical publications. *GigaScience* 13, giad113 (2024)
- [19] Samuel, S., Mietchen, D.: FAIR Jupyter: A knowledge graph approach to semantic sharing and granular exploration of a computational notebook reproducibility dataset. *Transactions on Graph Data and Knowledge* (2024)
- [20] Samuel, S., Mietchen, D., Lo, H., Gaedke, M.: Containing the reproducibility gap: Automated repository-level containerization for scholarly Jupyter notebooks. arXiv preprint arXiv:2604.01072 (2026)
- [21] Sun, J., Li, F., Qi, X., Zhang, H., Jiang, J.: Empirical evaluation of large language models in automated program repair. arXiv preprint arXiv:2506.13186 (2025)
- [22] Szalontai, B., et al.: Fine-tuning CodeLlama to fix bugs. In: Proceedings of the International Conference on Recent Innovations in Computing (ICRIC). Lecture Notes in Electrical Engineering, vol. 1195. Springer (2024)

BIBLIOGRAPHY

- [23] Tang, X., Gao, J., Xu, J., Sun, T., Song, Y., Ezzini, S., Ouédraogo, W.C., Klein, J., Bissyandé, T.F.: SynFix: Dependency-aware program repair via Relation-Graph analysis. In: Findings of the Association for Computational Linguistics (ACL 2025). pp. 4878–4894. Vienna, Austria (2025)
- [24] Tao, Y., et al.: Retrieval-augmented code generation: A survey with focus on repository-level approaches. arXiv preprint arXiv:2510.04905 (2025)
- [25] Wang, J., Kuo, T.Y., Li, L., Zeller, A.: Assessing and restoring reproducibility of Jupyter notebooks. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE) (2020)
- [26] Wang, J., Li, L., Zeller, A.: Restoring execution environments of Jupyter notebooks. In: Proceedings of the 43rd International Conference on Software Engineering (ICSE) (2021)
- [27] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D.: Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems (NeurIPS) (2022)
- [28] Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., et al.: The FAIR guiding principles for scientific data management and stewardship. *Scientific Data* 3, 160018 (2016)
- [29] World Wide Web Consortium: PROV-O: The PROV ontology. W3C Recommendation, <https://www.w3.org/TR/prov-o/> (2013)
- [30] Xia, C.S., Zhang, L.: Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA) (2024)
- [31] Yang, B., Cai, Z., Liu, F., Le, B., Zhang, L., Bissyandé, T.F., Liu, Y., Tian, H.: A survey of LLM-based automated program repair: Taxonomies, design paradigms, and applications. arXiv preprint arXiv:2506.23749 (2025)
- [32] Ye, H., Chen, W., Dou, W., Wu, G., Wei, J.: Knowledge-based environment dependency inference for Python programs. In: Proceedings of the 44th International Conference on Software Engineering (ICSE) (2022)